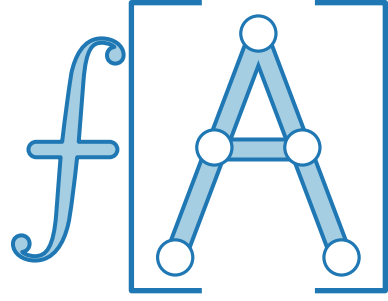




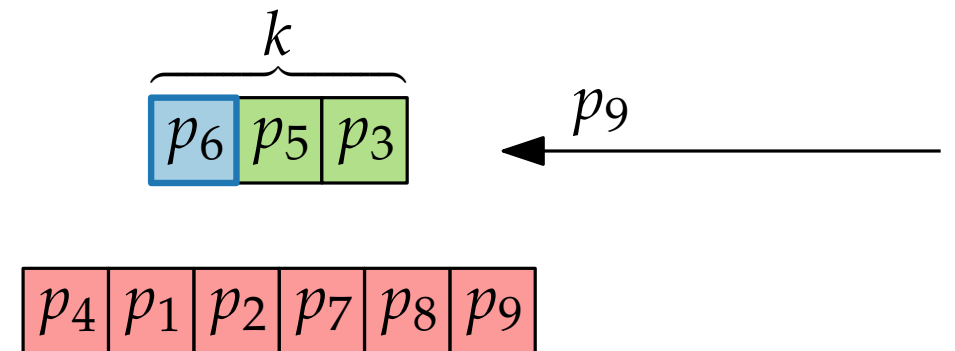
Fortgeschrittene Algorithmen

6. Vorlesung Online Algorithmen

Teil I: Ski-Verleih-Problem



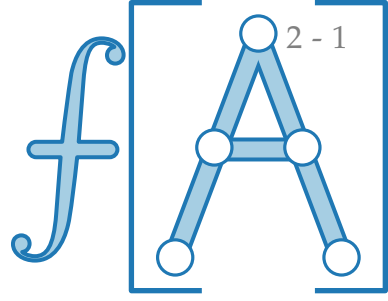
Philipp Kindermann



Wintersemester 2025/26

Einführung

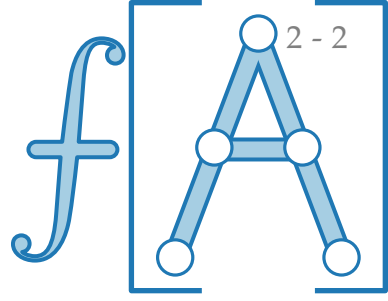
Die Winterferien stehen an...



Einführung

Die Winterferien stehen an...

... Zeit zum skifahren!

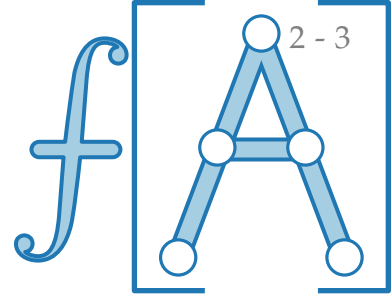


Einführung

Die Winterferien stehen an...



... Zeit zum skifahren!



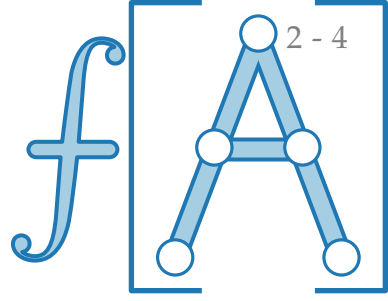
- Aber was, wenn wir nicht immer genug Schnee haben?

Ski-Verleih Problem

Die Winterferien stehen an...



... Zeit zum skifahren!



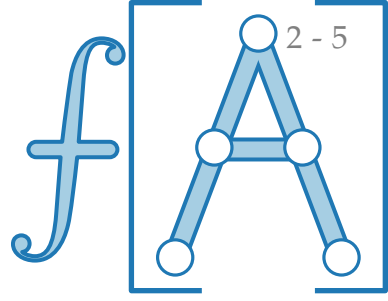
- Aber was, wenn wir nicht immer genug Schnee haben?
- Lohnt es sich, neue Ski zu **kaufen**?
- Oder sollten wir lieber welche **ausleihen**?

Ski-Verleih Problem

Die Winterferien stehen an...



... Zeit zum skifahren!



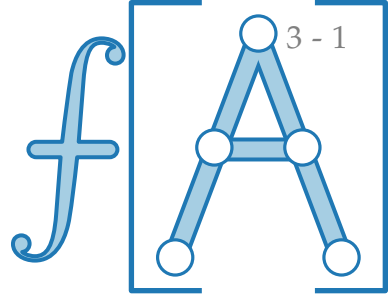
- Aber was, wenn wir nicht immer genug Schnee haben?
- Lohnt es sich, neue Ski zu **kaufen**?
- Oder sollten wir lieber welche **ausleihen**?
- Wir kennen das Wetter leider nicht im Voraus



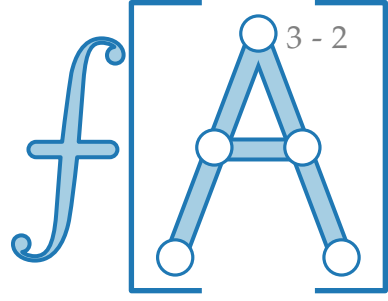
Problemstellung

Verhalten.

- An jedem Tag mit Schnee gehen wir skifahren.
Wir nennen das einen **guten** Tag.



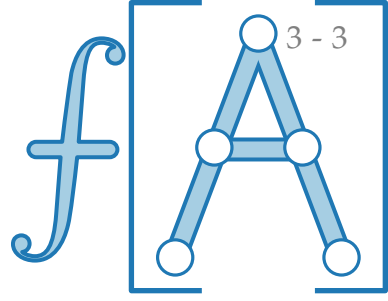
Problemstellung



Verhalten.

- An jedem Tag mit Schnee gehen wir skifahren.
Wir nennen das einen **guten** Tag.
- Jeden Morgen überprüfen wir, ob wir einen **guten** Tag haben. Früher geht das nicht.

Problemstellung



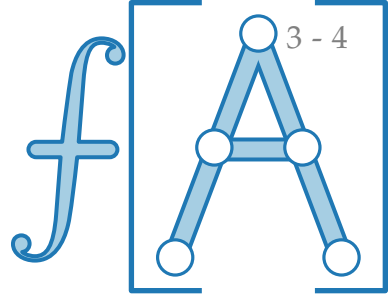
Verhalten.

- An jedem Tag mit Schnee gehen wir skifahren.
Wir nennen das einen **guten** Tag.
- Jeden Morgen überprüfen wir, ob wir einen **guten** Tag haben. Früher geht das nicht.

Kosten.

- Ski für 1 Tag ausleihen kostet 1€.

Problemstellung



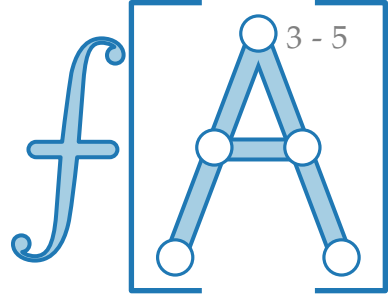
Verhalten.

- An jedem Tag mit Schnee gehen wir skifahren.
Wir nennen das einen **guten** Tag.
- Jeden Morgen überprüfen wir, ob wir einen **guten** Tag haben. Früher geht das nicht.

Kosten.

- Ski für 1 Tag ausleihen kostet 1€.
- Ski kaufen kostet M €, aber wir können sie behalten.

Problemstellung



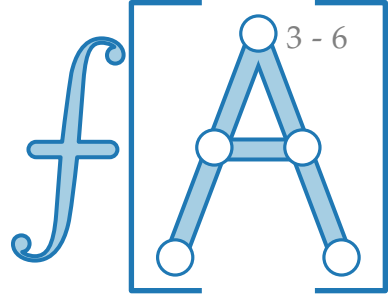
Verhalten.

- An jedem Tag mit Schnee gehen wir skifahren.
Wir nennen das einen **guten** Tag.
- Jeden Morgen überprüfen wir, ob wir einen **guten** Tag haben. Früher geht das nicht.

Kosten.

- Ski für 1 Tag ausleihen kostet 1€.
- Ski kaufen kostet M €, aber wir können sie behalten.
- Am Ende werden wir T **gute** Tage gehabt haben.

Problemstellung



Verhalten.

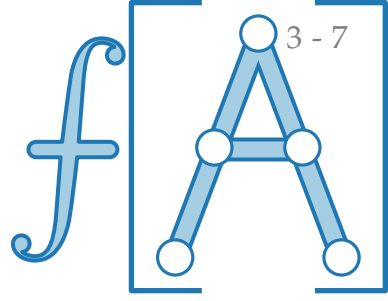
- An jedem Tag mit Schnee gehen wir skifahren.
Wir nennen das einen **guten** Tag.
- Jeden Morgen überprüfen wir, ob wir einen **guten** Tag haben. Früher geht das nicht.

Kosten.

- Ski für 1 Tag ausleihen kostet 1€.
- Ski kaufen kostet M €, aber wir können sie behalten.
- Am Ende werden wir T **gute** Tage gehabt haben.

(Wann sollen wir) Ski kaufen?

Problemstellung



Verhalten.

- An jedem Tag mit Schnee gehen wir skifahren.
Wir nennen das einen **guten** Tag.
- Jeden Morgen überprüfen wir, ob wir einen **guten** Tag haben. Früher geht das nicht.

Kosten.

- Ski für 1 Tag ausleihen kostet 1€.
- Ski kaufen kostet M €, aber wir können sie behalten.
- Am Ende werden wir T **gute** Tage gehabt haben.

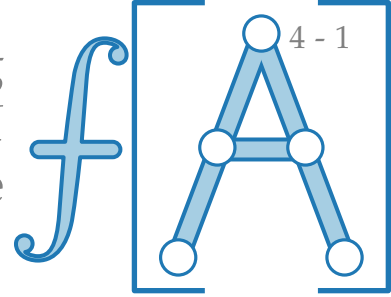
(Wann sollen wir) Ski kaufen?

Plan.

- Wir kennen T nicht,
- aber wollen eine Strategie entwickeln, wann wir Ski kaufen.

Strategien I und II

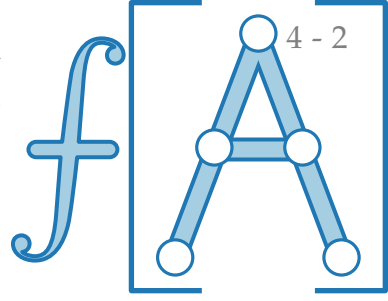
Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Strategien I und II

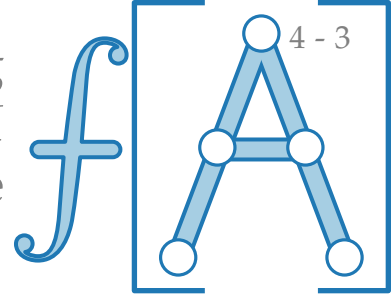
Strategie I: Kaufe am **ersten** guten Tag

Leihen kostet $1/\text{Tag}$
Kaufen kostet M
 T gute Tage



Strategien I und II

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage

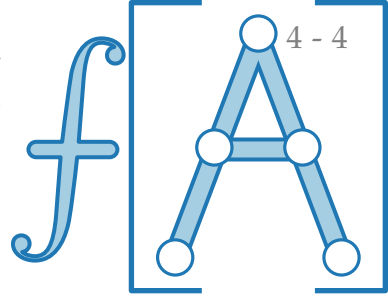


Strategie I: Kaufe am **ersten** guten Tag

- Angenommen, das ist der einzige gute Tag im gesamten Winter.

Strategien I und II

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage

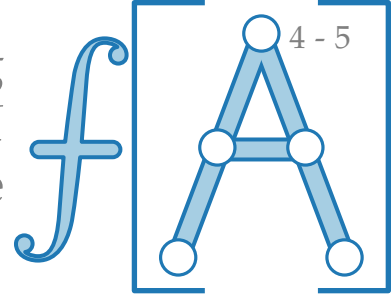


Strategie I: Kaufe am **ersten** guten Tag

- Angenommen, das ist der einzige gute Tag im gesamten Winter.
- Dann haben wir M bezahlt; im Optimalfall hätten wir geliehen und nur 1 bezahlt.

Strategien I und II

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage

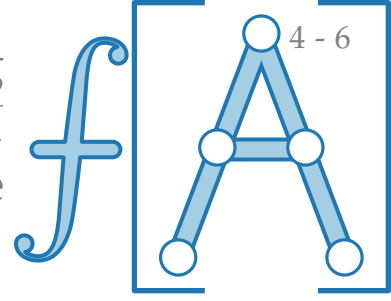


Strategie I: Kaufe am **ersten** guten Tag

- Angenommen, das ist der einzige gute Tag im gesamten Winter.
- Dann haben wir M bezahlt; im Optimalfall hätten wir geliehen und nur 1 bezahlt.
- Also ist Strategy I M mal so schlecht wie die optimale Strategie (im schlechtesten Fall) .

Strategien I und II

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage

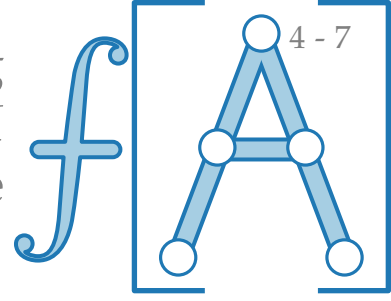


Strategie I: Kaufe am **ersten** guten Tag

- Angenommen, das ist der einzige gute Tag im gesamten Winter.
- Dann haben wir M bezahlt; im Optimalfall hätten wir geliehen und nur 1 bezahlt.
- Also ist Strategy I M mal so schlecht wie die optimale Strategie
(im schlechtesten Fall) .
→ für beliebig große M beliebig schlecht

Strategien I und II

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



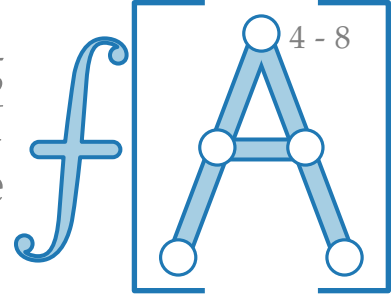
Strategie I: Kaufe am **ersten** guten Tag

- Angenommen, das ist der einzige gute Tag im gesamten Winter.
- Dann haben wir M bezahlt; im Optimalfall hätten wir geliehen und nur 1 bezahlt.
- Also ist Strategy I M mal so schlecht wie die optimale Strategie
(im schlechtesten Fall) .
→ für beliebig große M beliebig schlecht

Strategie II: **kaufe nie**, leihe immer

Strategien I und II

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Strategie I: Kaufe am **ersten** guten Tag

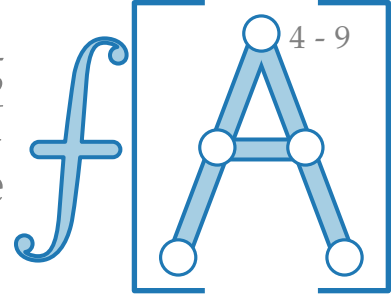
- Angenommen, das ist der einzige gute Tag im gesamten Winter.
- Dann haben wir M bezahlt; im Optimalfall hätten wir geliehen und nur 1 bezahlt.
- Also ist Strategy I M mal so schlecht wie die optimale Strategie (im schlechtesten Fall) .
➔ für beliebig große M beliebig schlecht

Strategie II: **kaufe nie**, leihe immer

- Angenommen, es gibt sehr viele gute Tage, d.h. $T > M$.

Strategien I und II

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Strategie I: Kaufe am **ersten** guten Tag

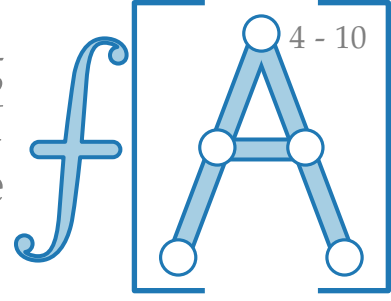
- Angenommen, das ist der einzige gute Tag im gesamten Winter.
- Dann haben wir M bezahlt; im Optimalfall hätten wir geliehen und nur 1 bezahlt.
- Also ist Strategy I M mal so schlecht wie die optimale Strategie (im schlechtesten Fall) .
→ für beliebig große M beliebig schlecht

Strategie II: **kaufe nie**, leihe immer

- Angenommen, es gibt sehr viele gute Tage, d.h. $T > M$.
- Dann haben wir T bezahlt.
Im Optimalfall hätten wir am Anfang gekauft und M bezahlt

Strategien I und II

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Strategie I: Kaufe am **ersten** guten Tag

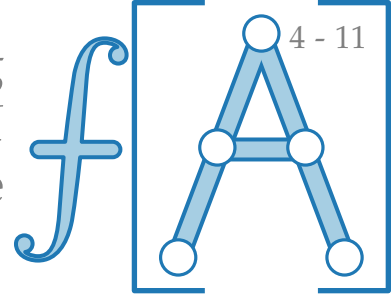
- Angenommen, das ist der einzige gute Tag im gesamten Winter.
- Dann haben wir M bezahlt; im Optimalfall hätten wir geliehen und nur 1 bezahlt.
- Also ist Strategy I M mal so schlecht wie die optimale Strategie (im schlechtesten Fall) .
➔ für beliebig große M beliebig schlecht

Strategie II: **kaufe nie**, leihe immer

- Angenommen, es gibt sehr viele gute Tage, d.h. $T > M$.
- Dann haben wir T bezahlt.
Im Optimalfall hätten wir am Anfang gekauft und M bezahlt
- Also ist Strategie II T/M mal so schlecht wie die optimale Strategie.

Strategien I und II

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Strategie I: Kaufe am **ersten** guten Tag

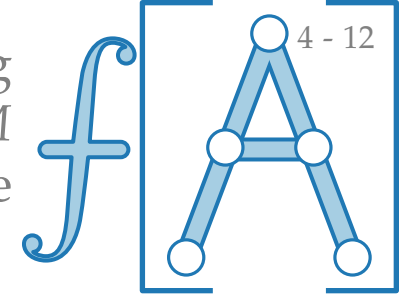
- Angenommen, das ist der einzige gute Tag im gesamten Winter.
- Dann haben wir M bezahlt; im Optimalfall hätten wir geliehen und nur 1 bezahlt.
- Also ist Strategy I M mal so schlecht wie die optimale Strategie (im schlechtesten Fall) .
➔ für beliebig große M beliebig schlecht

Strategie II: **kaufe nie**, leihe immer

- Angenommen, es gibt sehr viele gute Tage, d.h. $T > M$.
- Dann haben wir T bezahlt.
Im Optimalfall hätten wir am Anfang gekauft und M bezahlt
- Also ist Strategie II T/M mal so schlecht wie die optimale Strategie.
➔ für beliebig große T beliebig schlecht

Strategien I und II

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Strategie I: Kaufe am **ersten** guten Tag

- Angenommen, das ist der einzige gute Tag im gesamten Winter.
- Dann haben wir M bezahlt; im Optimalfall hätten wir geliehen und nur 1 bezahlt.
- Also ist **Strategy I M** mal so schlecht wie die optimale Strategie (im schlechtesten Fall).
→ für beliebig große M beliebig schlecht

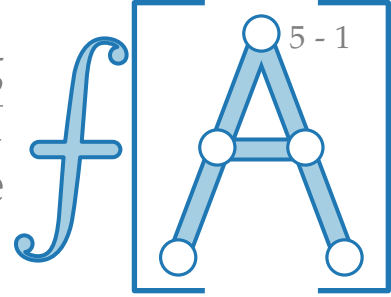
Strategie II: **kaufe nie**, leihe immer

- Angenommen, es gibt sehr viele gute Tage, d.h. $T > M$.
- Dann haben wir T bezahlt.
Im Optimalfall hätten wir am Anfang gekauft und M bezahlt
- Also ist **Strategie II T/M** mal so schlecht wie die optimale Strategie.
→ für beliebig große T beliebig schlecht

**kompetitives
Verhältnis**

Strategie III

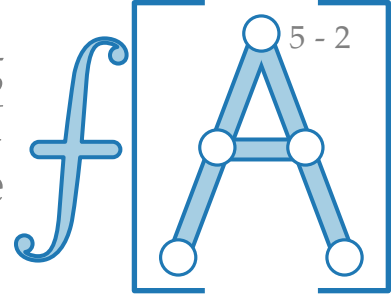
Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Strategie III

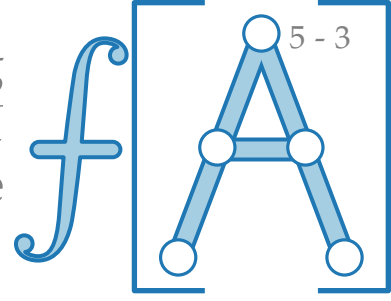
Gibt es eine Strategie, die nicht beliebig schlecht werden kann?

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Strategie III

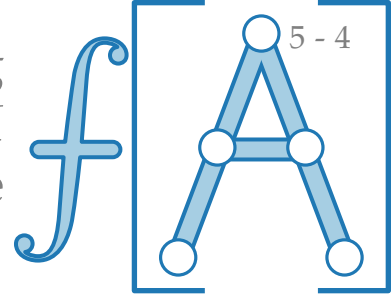
Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Gibt es eine Strategie, die nicht beliebig schlecht werden kann? – Ja!

Strategie III

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage

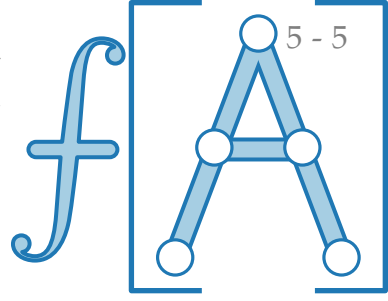


Gibt es eine Strategie, die nicht beliebig schlecht werden kann? – Ja!

Strategie III: kaufe am **M**-ten guten Tag

Strategie III

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



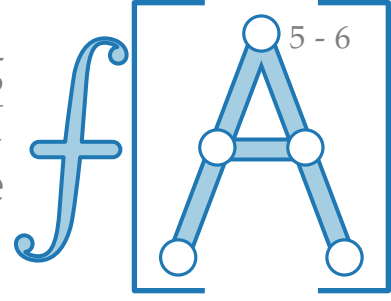
Gibt es eine Strategie, die nicht beliebig schlecht werden kann? – Ja!

Strategie III: kaufe am **M**-ten guten Tag

Beobachtung. Die optimale Lösung bezahlt $\min(M, T)$.

Strategie III

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Gibt es eine Strategie, die nicht beliebig schlecht werden kann? – Ja!

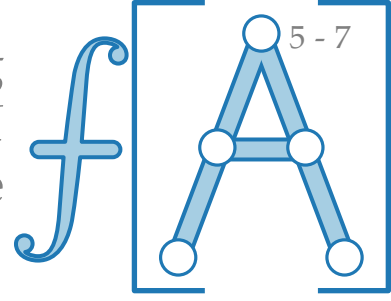
Strategie III: kaufe am **M**-ten guten Tag

Beobachtung. Die optimale Lösung bezahlt $\min(M, T)$.

- Wenn $T < M$, dann ist das kompetitive Verhältnis 1.

Strategie III

Leihen kostet $1/\text{Tag}$
Kaufen kostet M
 T gute Tage



Gibt es eine Strategie, die nicht beliebig schlecht werden kann? – Ja!

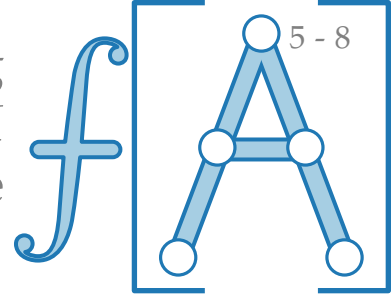
Strategie III: kaufe am **M**-ten guten Tag

Beobachtung. Die optimale Lösung bezahlt $\min(M, T)$.

■ Wenn $T < M$, dann ist das kompetitive Verhältnis 1. Sonst ist es $\frac{2M-1}{M} = 2 - \frac{1}{M}$

Strategie III

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Gibt es eine Strategie, die nicht beliebig schlecht werden kann? – Ja!

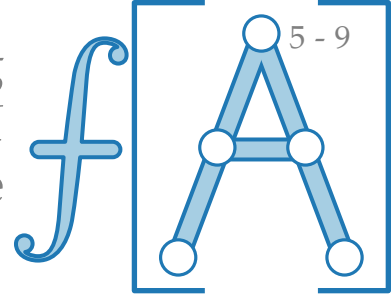
Strategie III: kaufe am **M**-ten guten Tag

Beobachtung. Die optimale Lösung bezahlt $\min(M, T)$.

■ Wenn $T < M$, dann ist das kompetitive Verhältnis 1. Sonst ist es $\frac{2M-1}{M} \stackrel{M \rightsquigarrow \infty}{=} 2 - \frac{1}{M}$.

Strategie III

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Gibt es eine Strategie, die nicht beliebig schlecht werden kann? – Ja!

Strategie III: kaufe am **M**-ten guten Tag

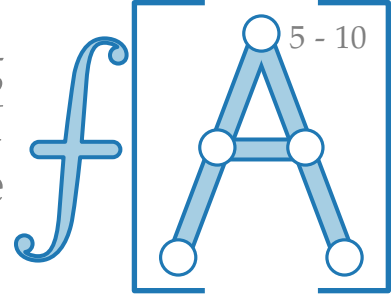
Beobachtung. Die optimale Lösung bezahlt $\min(M, T)$.

■ Wenn $T < M$, dann ist das kompetitive Verhältnis 1. Sonst ist es $\frac{2M-1}{M} \stackrel{M \rightsquigarrow \infty}{=} 2.$

$\Rightarrow$ Strategie III ist deterministisch und **2-kompetitiv**.

Strategie III

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Gibt es eine Strategie, die nicht beliebig schlecht werden kann? – Ja!

Strategie III: kaufe am **M**-ten guten Tag

Beobachtung. Die optimale Lösung bezahlt $\min(M, T)$.

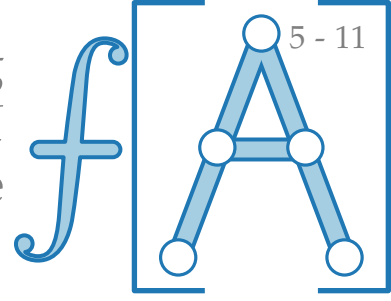
■ Wenn $T < M$, dann ist das kompetitive Verhältnis 1. Sonst ist es $\frac{2M-1}{M} \stackrel{M \rightsquigarrow \infty}{=} 2 - \frac{1}{M}$

$\Rightarrow$ Strategie III ist deterministisch und **2-kompetitiv**.

Satz. Keine det. Strategie ist besser als 2-kompetitiv (für $M \rightsquigarrow \infty$; im Allgemeinen: $2 - \frac{1}{M}$).

Strategie III

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Gibt es eine Strategie, die nicht beliebig schlecht werden kann? – Ja!

Strategie III: kaufe am **M**-ten guten Tag

Beobachtung. Die optimale Lösung bezahlt $\min(M, T)$.

■ Wenn $T < M$, dann ist das kompetitive Verhältnis 1. Sonst ist es $\frac{2M-1}{M} \stackrel{M \rightsquigarrow \infty}{=} 2 - \frac{1}{M}$

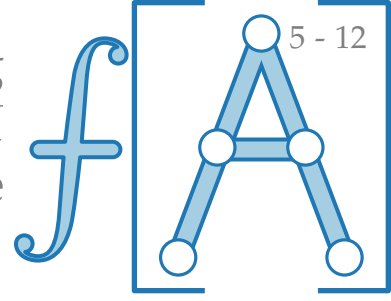
$\Rightarrow$ Strategie III ist deterministisch und **2-kompetitiv**.

Satz. Keine det. Strategie ist besser als 2-kompetitiv (für $M \rightsquigarrow \infty$; im Allgemeinen: $2 - \frac{1}{M}$).

Beweisidee.

Strategie III

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Gibt es eine Strategie, die nicht beliebig schlecht werden kann? – Ja!

Strategie III: kaufe am **M**-ten guten Tag

Beobachtung. Die optimale Lösung bezahlt $\min(M, T)$.

■ Wenn $T < M$, dann ist das kompetitive Verhältnis 1. Sonst ist es $\frac{2M-1}{M} \stackrel{M \rightsquigarrow \infty}{=} 2 - \frac{1}{M}$
 $\Rightarrow$ Strategie III ist deterministisch und **2-kompetitiv**.

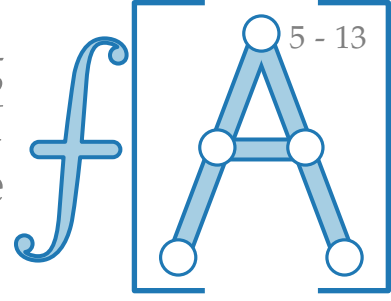
Satz. Keine det. Strategie ist besser als 2-kompetitiv (für $M \rightsquigarrow \infty$; im Allgemeinen: $2 - \frac{1}{M}$).

Beweisidee.

■ Jede det. Strategie kann so formuliert werden: *kaufe am X-ten guten Tag*.

Strategie III

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Gibt es eine Strategie, die nicht beliebig schlecht werden kann? – Ja!

Strategie III: kaufe am **M**-ten guten Tag

Beobachtung. Die optimale Lösung bezahlt $\min(M, T)$.

■ Wenn $T < M$, dann ist das kompetitive Verhältnis 1. Sonst ist es $\frac{2M-1}{M} \stackrel{M \rightsquigarrow \infty}{=} 2 - \frac{1}{M}$
 $\Rightarrow$ Strategie III ist deterministisch und **2-kompetitiv**.

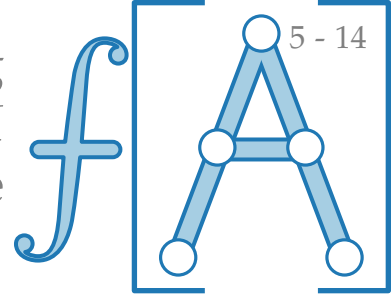
Satz. Keine det. Strategie ist besser als 2-kompetitiv (für $M \rightsquigarrow \infty$; im Allgemeinen: $2 - \frac{1}{M}$).

Beweisidee.

- Jede det. Strategie kann so formuliert werden: *kaufe am X-ten guten Tag*.
- Für $X \in \{0, \infty\}$ beliebig schlecht; nehme $X \in \mathbb{N}^+$ an. Beobachte: worst-case ist $T = X$.

Strategie III

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Gibt es eine Strategie, die nicht beliebig schlecht werden kann? – Ja!

Strategie III: kaufe am **M**-ten guten Tag

Beobachtung. Die optimale Lösung bezahlt $\min(M, T)$.

■ Wenn $T < M$, dann ist das kompetitive Verhältnis 1. Sonst ist es $\frac{2M-1}{M} \stackrel{M \rightsquigarrow \infty}{=} 2 - \frac{1}{M}$
 $\Rightarrow$ Strategie III ist deterministisch und **2-kompetitiv**.

Satz. Keine det. Strategie ist besser als 2-kompetitiv (für $M \rightsquigarrow \infty$; im Allgemeinen: $2 - \frac{1}{M}$).

Beweisidee.

- Jede det. Strategie kann so formuliert werden: *kaufe am X-ten guten Tag*.
- Für $X \in \{0, \infty\}$ beliebig schlecht; nehme $X \in \mathbb{N}^+$ an. Beobachte: worst-case ist $T = X$.

Kosten für deterministische Strategie

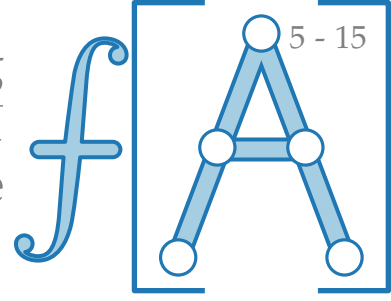
c_{det}

c_{OPT}

Kosten für optimale Strategie

Strategie III

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Gibt es eine Strategie, die nicht beliebig schlecht werden kann? – Ja!

Strategie III: kaufe am **M**-ten guten Tag

Beobachtung. Die optimale Lösung bezahlt $\min(M, T)$.

■ Wenn $T < M$, dann ist das kompetitive Verhältnis 1. Sonst ist es $\frac{2M-1}{M} \stackrel{M \rightsquigarrow \infty}{=} 2 - \frac{1}{M}$
 $\Rightarrow$ Strategie III ist deterministisch und **2-kompetitiv**.

Satz. Keine det. Strategie ist besser als 2-kompetitiv (für $M \rightsquigarrow \infty$; im Allgemeinen: $2 - \frac{1}{M}$).

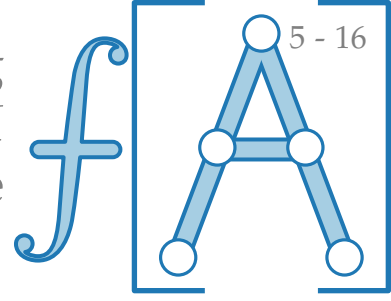
Beweisidee.

- Jede det. Strategie kann so formuliert werden: *kaufe am X -ten guten Tag.*
- Für $X \in \{0, \infty\}$ beliebig schlecht; nehme $X \in \mathbb{N}^+$ an. Beobachte: worst-case ist $T = X$.

■ $\frac{c_{\text{det}}}{c_{\text{OPT}}} =$

Strategie III

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Gibt es eine Strategie, die nicht beliebig schlecht werden kann? – Ja!

Strategie III: kaufe am **M**-ten guten Tag

Beobachtung. Die optimale Lösung bezahlt $\min(M, T)$.

■ Wenn $T < M$, dann ist das kompetitive Verhältnis 1. Sonst ist es $\frac{2M-1}{M} \stackrel{M \rightsquigarrow \infty}{=} 2 - \frac{1}{M}$
 $\Rightarrow$ Strategie III ist deterministisch und **2-kompetitiv**.

Satz. Keine det. Strategie ist besser als 2-kompetitiv (für $M \rightsquigarrow \infty$; im Allgemeinen: $2 - \frac{1}{M}$).

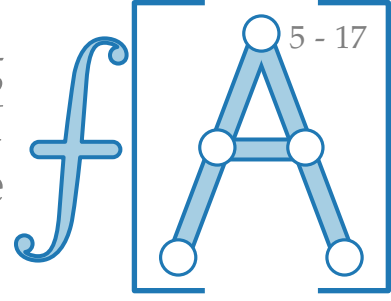
Beweisidee.

- Jede det. Strategie kann so formuliert werden: *kaufe am X -ten guten Tag.*
- Für $X \in \{0, \infty\}$ beliebig schlecht; nehme $X \in \mathbb{N}^+$ an. Beobachte: worst-case ist $T = X$.

■ $\frac{c_{\text{det}}}{c_{\text{OPT}}} = \frac{X-1+M}{X} \geq$

Strategie III

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Gibt es eine Strategie, die nicht beliebig schlecht werden kann? – Ja!

Strategie III: kaufe am **M**-ten guten Tag

Beobachtung. Die optimale Lösung bezahlt $\min(M, T)$.

■ Wenn $T < M$, dann ist das kompetitive Verhältnis 1. Sonst ist es $\frac{2M-1}{M} \stackrel{M \rightsquigarrow \infty}{=} 2 - \frac{1}{M}$
 $\Rightarrow$ Strategie III ist deterministisch und **2-kompetitiv**.

Satz. Keine det. Strategie ist besser als 2-kompetitiv (für $M \rightsquigarrow \infty$; im Allgemeinen: $2 - \frac{1}{M}$).

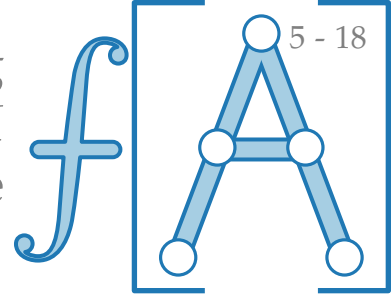
Beweisidee.

- Jede det. Strategie kann so formuliert werden: *kaufe am X -ten guten Tag.*
- Für $X \in \{0, \infty\}$ beliebig schlecht; nehme $X \in \mathbb{N}^+$ an. Beobachte: worst-case ist $T = X$.

■ $\frac{c_{\text{det}}}{c_{\text{OPT}}} = \frac{X-1+M}{M} \geq$

Strategie III

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Gibt es eine Strategie, die nicht beliebig schlecht werden kann? – Ja!

Strategie III: kaufe am **M**-ten guten Tag

Beobachtung. Die optimale Lösung bezahlt $\min(M, T)$.

■ Wenn $T < M$, dann ist das kompetitive Verhältnis 1. Sonst ist es $\frac{2M-1}{M} \stackrel{M \rightsquigarrow \infty}{=} 2 - \frac{1}{M}$
 $\Rightarrow$ Strategie III ist deterministisch und **2-kompetitiv**.

Satz. Keine det. Strategie ist besser als 2-kompetitiv (für $M \rightsquigarrow \infty$; im Allgemeinen: $2 - \frac{1}{M}$).

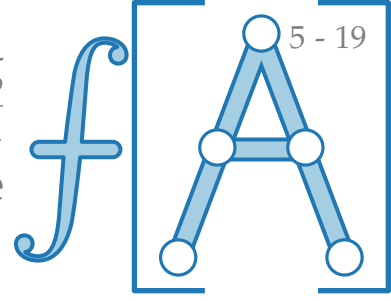
Beweisidee.

- Jede det. Strategie kann so formuliert werden: *kaufe am X -ten guten Tag.*
- Für $X \in \{0, \infty\}$ beliebig schlecht; nehme $X \in \mathbb{N}^+$ an. Beobachte: worst-case ist $T = X$.

■ $\frac{c_{\text{det}}}{c_{\text{OPT}}} = \frac{X-1+M}{X} \geq$

Strategie III

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Gibt es eine Strategie, die nicht beliebig schlecht werden kann? – Ja!

Strategie III: kaufe am **M**-ten guten Tag

Beobachtung. Die optimale Lösung bezahlt $\min(M, T)$.

■ Wenn $T < M$, dann ist das kompetitive Verhältnis 1. Sonst ist es $\frac{2M-1}{M} \stackrel{M \rightsquigarrow \infty}{=} 2 - \frac{1}{M}$
 $\Rightarrow$ Strategie III ist deterministisch und **2-kompetitiv**.

Satz. Keine det. Strategie ist besser als 2-kompetitiv (für $M \rightsquigarrow \infty$; im Allgemeinen: $2 - \frac{1}{M}$).

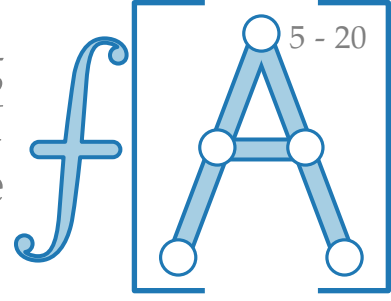
Beweisidee.

- Jede det. Strategie kann so formuliert werden: *kaufe am X -ten guten Tag.*
- Für $X \in \{0, \infty\}$ beliebig schlecht; nehme $X \in \mathbb{N}^+$ an. Beobachte: worst-case ist $T = X$.

$$\frac{c_{\text{det}}}{c_{\text{OPT}}} = \frac{X-1+M}{\min(X, M)} \geq$$

Strategie III

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Gibt es eine Strategie, die nicht beliebig schlecht werden kann? – Ja!

Strategie III: kaufe am **M**-ten guten Tag

Beobachtung. Die optimale Lösung bezahlt $\min(M, T)$.

■ Wenn $T < M$, dann ist das kompetitive Verhältnis 1. Sonst ist es $\frac{2M-1}{M} \stackrel{M \rightsquigarrow \infty}{=} 2 - \frac{1}{M}$
 $\Rightarrow$ Strategie III ist deterministisch und **2-kompetitiv**.

Satz. Keine det. Strategie ist besser als 2-kompetitiv (für $M \rightsquigarrow \infty$; im Allgemeinen: $2 - \frac{1}{M}$).

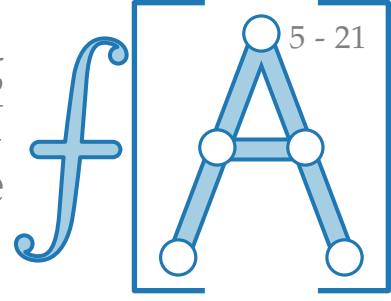
Beweisidee.

- Jede det. Strategie kann so formuliert werden: *kaufe am X -ten guten Tag.*
- Für $X \in \{0, \infty\}$ beliebig schlecht; nehme $X \in \mathbb{N}^+$ an. Beobachte: worst-case ist $T = X$.

$$\frac{c_{\text{det}}}{c_{\text{OPT}}} = \frac{X-1+M}{\min(X, M)} \geq \min \left(\overset{X < M}{\quad} \overset{X \geq M}{\quad} \right) =$$

Strategie III

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Gibt es eine Strategie, die nicht beliebig schlecht werden kann? – Ja!

Strategie III: kaufe am **M**-ten guten Tag

Beobachtung. Die optimale Lösung bezahlt $\min(M, T)$.

■ Wenn $T < M$, dann ist das kompetitive Verhältnis 1. Sonst ist es $\frac{2M-1}{M} \stackrel{M \rightsquigarrow \infty}{=} 2 - \frac{1}{M}$
 $\Rightarrow$ Strategie III ist deterministisch und **2-kompetitiv**.

Satz. Keine det. Strategie ist besser als 2-kompetitiv (für $M \rightsquigarrow \infty$; im Allgemeinen: $2 - \frac{1}{M}$).

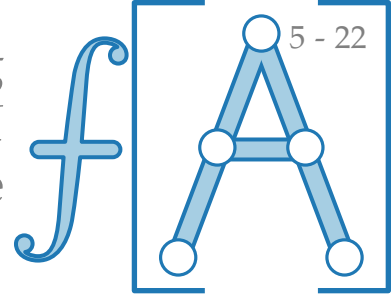
Beweisidee.

- Jede det. Strategie kann so formuliert werden: *kaufe am X -ten guten Tag.*
- Für $X \in \{0, \infty\}$ beliebig schlecht; nehme $X \in \mathbb{N}^+$ an. Beobachte: worst-case ist $T = X$.

$$\frac{c_{\text{det}}}{c_{\text{OPT}}} = \frac{X-1+M}{\min(X, M)} \geq \min \left(\overset{X < M}{\frac{X-1+M}{X}}, \overset{X \geq M}{\frac{M-1+M}{M}} \right) =$$

Strategie III

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Gibt es eine Strategie, die nicht beliebig schlecht werden kann? – Ja!

Strategie III: kaufe am M -ten guten Tag

Beobachtung. Die optimale Lösung bezahlt $\min(M, T)$.

■ Wenn $T < M$, dann ist das kompetitive Verhältnis 1. Sonst ist es $\frac{2M-1}{M} \stackrel{M \rightsquigarrow \infty}{=} 2 - \frac{1}{M}$
 $\Rightarrow$ Strategie III ist deterministisch und **2-kompetitiv**.

Satz. Keine det. Strategie ist besser als 2-kompetitiv (für $M \rightsquigarrow \infty$; im Allgemeinen: $2 - \frac{1}{M}$).

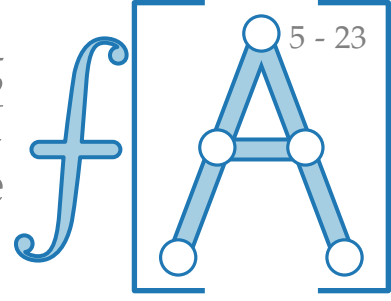
Beweisidee.

- Jede det. Strategie kann so formuliert werden: *kaufe am X -ten guten Tag.*
- Für $X \in \{0, \infty\}$ beliebig schlecht; nehme $X \in \mathbb{N}^+$ an. Beobachte: worst-case ist $T = X$.

$$\frac{c_{\text{det}}}{c_{\text{OPT}}} = \frac{X-1+M}{\min(X, M)} \geq \min \left(\overset{X < M}{\frac{X-1+X+1}{X}}, \overset{X \geq M}{\frac{M-1+M}{M}} \right) =$$

Strategie III

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Gibt es eine Strategie, die nicht beliebig schlecht werden kann? – Ja!

Strategie III: kaufe am M -ten guten Tag

Beobachtung. Die optimale Lösung bezahlt $\min(M, T)$.

■ Wenn $T < M$, dann ist das kompetitive Verhältnis 1. Sonst ist es $\frac{2M-1}{M} \stackrel{M \rightsquigarrow \infty}{=} 2 - \frac{1}{M}$
 $\Rightarrow$ Strategie III ist deterministisch und **2-kompetitiv**.

Satz. Keine det. Strategie ist besser als 2-kompetitiv (für $M \rightsquigarrow \infty$; im Allgemeinen: $2 - \frac{1}{M}$).

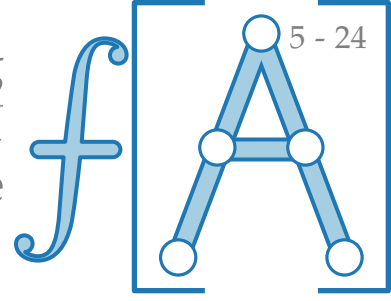
Beweisidee.

- Jede det. Strategie kann so formuliert werden: *kaufe am X -ten guten Tag.*
- Für $X \in \{0, \infty\}$ beliebig schlecht; nehme $X \in \mathbb{N}^+$ an. Beobachte: worst-case ist $T = X$.

$$\frac{c_{\text{det}}}{c_{\text{OPT}}} = \frac{X-1+M}{\min(X, M)} \geq \min \left(\overset{X < M}{\frac{X-1+X+1}{X}}, \overset{X \geq M}{\frac{M-1+M}{M}} \right) = \min \left(2, 2 - \frac{1}{M} \right)$$

Strategie III

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Gibt es eine Strategie, die nicht beliebig schlecht werden kann? – Ja!

Strategie III: kaufe am **M**-ten guten Tag

Beobachtung. Die optimale Lösung bezahlt $\min(M, T)$.

■ Wenn $T < M$, dann ist das kompetitive Verhältnis 1. Sonst ist es $\frac{2M-1}{M} \stackrel{M \rightsquigarrow \infty}{=} 2 - \frac{1}{M}$
 $\Rightarrow$ Strategie III ist deterministisch und **2-kompetitiv**.

Satz. Keine det. Strategie ist besser als 2-kompetitiv (für $M \rightsquigarrow \infty$; im Allgemeinen: $2 - \frac{1}{M}$).

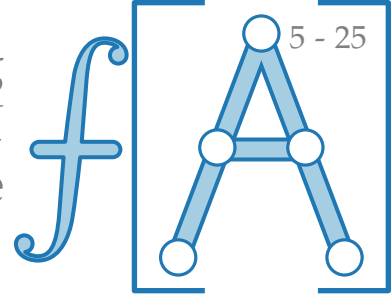
Beweisidee.

- Jede det. Strategie kann so formuliert werden: *kaufe am X -ten guten Tag.*
- Für $X \in \{0, \infty\}$ beliebig schlecht; nehme $X \in \mathbb{N}^+$ an. Beobachte: worst-case ist $T = X$.

$$\frac{c_{\text{det}}}{c_{\text{OPT}}} = \frac{X-1+M}{\min(X, M)} \geq \min \left(\overset{X < M}{\frac{X-1+X+1}{X}}, \overset{X \geq M}{\frac{M-1+M}{M}} \right) = \min \left(2, 2 - \frac{1}{M} \right) = 2 - \frac{1}{M}$$

Strategie III

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Gibt es eine Strategie, die nicht beliebig schlecht werden kann? – Ja!

Strategie III: kaufe am **M**-ten guten Tag

Beobachtung. Die optimale Lösung bezahlt $\min(M, T)$.

■ Wenn $T < M$, dann ist das kompetitive Verhältnis 1. Sonst ist es $\frac{2M-1}{M} \stackrel{M \rightsquigarrow \infty}{=} 2 - \frac{1}{M}$
 $\Rightarrow$ Strategie III ist deterministisch und **2-kompetitiv**.

Satz. Keine det. Strategie ist besser als 2-kompetitiv (für $M \rightsquigarrow \infty$; im Allgemeinen: $2 - \frac{1}{M}$).

Beweisidee.

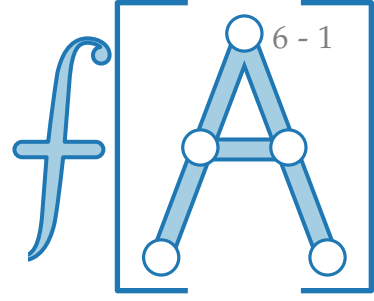
- Jede det. Strategie kann so formuliert werden: *kaufe am X -ten guten Tag.*
- Für $X \in \{0, \infty\}$ beliebig schlecht; nehme $X \in \mathbb{N}^+$ an. Beobachte: worst-case ist $T = X$.

$$\text{■ } \frac{c_{\text{det}}}{c_{\text{OPT}}} = \frac{X-1+M}{\min(X, M)} \geq \min \left(\overset{X < M}{\frac{X-1+X+1}{X}}, \overset{X \geq M}{\frac{M-1+M}{M}} \right) = \min \left(2, 2 - \frac{1}{M} \right) = 2 - \frac{1}{M} \stackrel{M \rightsquigarrow \infty}{=} 2$$

Strategie IV

Können wir die Schranke durch Randomisierung brechen?

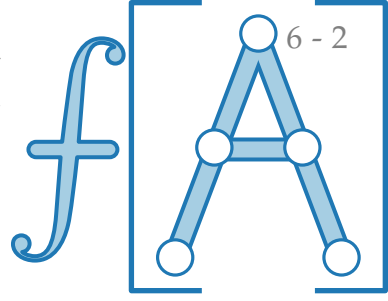
Leihen kostet $1/\text{Tag}$
Kaufen kostet M
 T gute Tage



Strategie IV

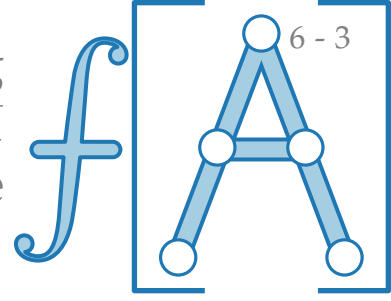
Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage

Können wir die Schranke durch Randomisierung brechen? Versuchen wir's!



Strategie IV

Leihen kostet $1/\text{Tag}$
Kaufen kostet M
 T gute Tage



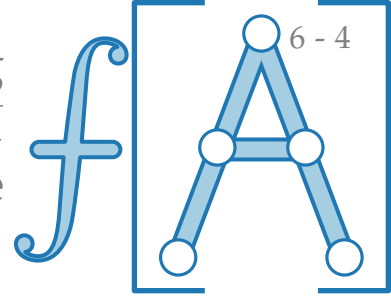
Können wir die Schranke durch Randomisierung brechen? Versuchen wir's!

Strategie IV: Werfe eine Münze;



Strategie IV

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



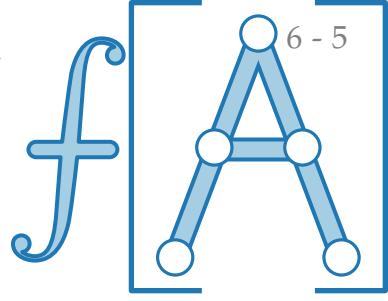
Können wir die Schranke durch Randomisierung brechen? Versuchen wir's!

Strategie IV: Werfe eine Münze; **Kopf:** kaufe am **M**-ten guten Tag



Strategie IV

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



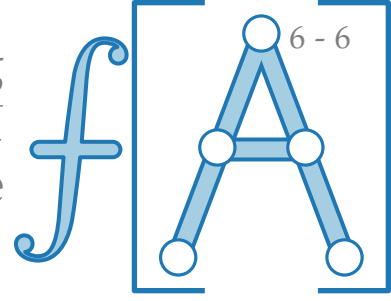
Können wir die Schranke durch Randomisierung brechen? Versuchen wir's!

Strategie IV: Werfe eine Münze; **Kopf:** kaufe am M -ten guten Tag
Zahl: kaufe am αM -ten guten Tag ($\alpha \in (0, 1)$)



Strategie IV

Leihen kostet $1/\text{Tag}$
Kaufen kostet M
 T gute Tage



Können wir die Schranke durch Randomisierung brechen? Versuchen wir's!

Strategie IV: Werfe eine Münze; **Kopf:** kaufe am **M** -ten guten Tag

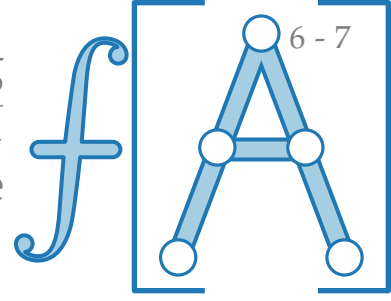
Zahl: kaufe am **αM** -ten guten Tag ($\alpha \in (0, 1)$)



Beobachtung. Schlechtester Fall ist $T = M$ oder $T = \alpha M$

Strategie IV

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Können wir die Schranke durch Randomisierung brechen? Versuchen wir's!

Strategie IV: Werfe eine Münze; **Kopf:** kaufe am M -ten guten Tag
Zahl: kaufe am αM -ten guten Tag ($\alpha \in (0, 1)$)

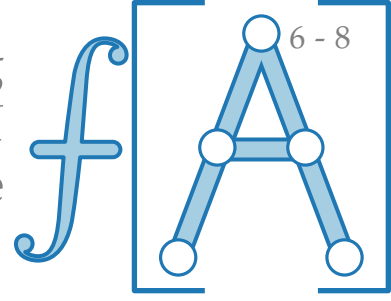


Beobachtung. Schlechtester Fall ist $T = M$ oder $T = \alpha M$

■ Fall $T = M$:

Strategie IV

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Können wir die Schranke durch Randomisierung brechen? Versuchen wir's!

Strategie IV: Werfe eine Münze; **Kopf:** kaufe am M -ten guten Tag
Zahl: kaufe am αM -ten guten Tag ($\alpha \in (0, 1)$)

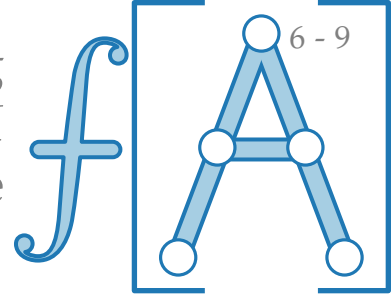


Beobachtung. Schlechtester Fall ist $T = M$ oder $T = \alpha M$

■ Fall $T = M$: $\frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}}$

Strategie IV

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Können wir die Schranke durch Randomisierung brechen? Versuchen wir's!

Strategie IV: Werfe eine Münze; **Kopf:** kaufe am **M** -ten guten Tag
Zahl: kaufe am **αM** -ten guten Tag ($\alpha \in (0, 1)$)

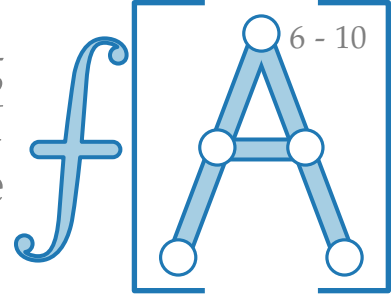


Beobachtung. Schlechtester Fall ist $T = M$ oder $T = \alpha M$

■ Fall $T = M$: $\frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\quad}{M}$

Strategie IV

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Können wir die Schranke durch Randomisierung brechen? Versuchen wir's!

Strategie IV: Werfe eine Münze; **Kopf:** kaufe am **M** -ten guten Tag
Zahl: kaufe am **αM** -ten guten Tag ($\alpha \in (0, 1)$)

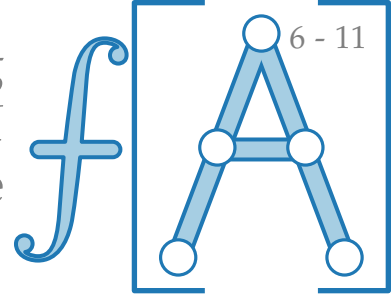


Beobachtung. Schlechtester Fall ist $T = M$ oder $T = \alpha M$

■ Fall $T = M$:
$$\frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot (2M - 1)}{M}$$

Strategie IV

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Können wir die Schranke durch Randomisierung brechen? Versuchen wir's!

Strategie IV: Werfe eine Münze; **Kopf:** kaufe am **M** -ten guten Tag
Zahl: kaufe am **αM** -ten guten Tag ($\alpha \in (0, 1)$)

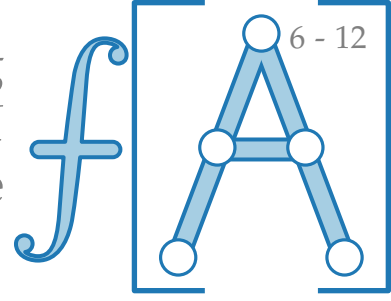


Beobachtung. Schlechtester Fall ist $T = M$ oder $T = \alpha M$

■ Fall $T = M$:
$$\frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot (2M-1) + \frac{1}{2} \cdot ((1+\alpha)M-1)}{M}$$

Strategie IV

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Können wir die Schranke durch Randomisierung brechen? Versuchen wir's!

Strategie IV: Werfe eine Münze; **Kopf:** kaufe am M -ten guten Tag
Zahl: kaufe am αM -ten guten Tag ($\alpha \in (0, 1)$)

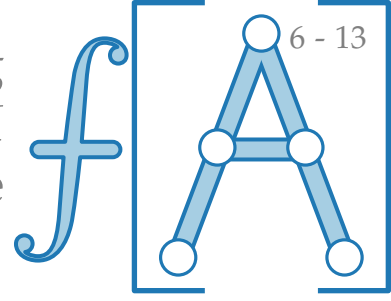


Beobachtung. Schlechtester Fall ist $T = M$ oder $T = \alpha M$

■ Fall $T = M$:
$$\frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot (2M - 1) + \frac{1}{2} \cdot ((1 + \alpha)M - 1)}{M}$$

Strategie IV

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Können wir die Schranke durch Randomisierung brechen? Versuchen wir's!

Strategie IV: Werfe eine Münze; **Kopf:** kaufe am M -ten guten Tag
Zahl: kaufe am αM -ten guten Tag ($\alpha \in (0, 1)$)

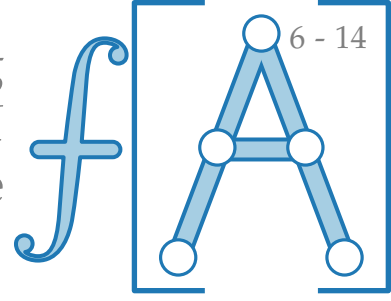


Beobachtung. Schlechtester Fall ist $T = M$ oder $T = \alpha M$

■ Fall $T = M$:
$$\frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot (2M - 1) + \frac{1}{2} \cdot ((1 + \alpha)M - 1)}{M} = \frac{3 + \alpha}{2}$$

Strategie IV

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Können wir die Schranke durch Randomisierung brechen? Versuchen wir's!

Strategie IV: Werfe eine Münze; **Kopf:** kaufe am M -ten guten Tag
Zahl: kaufe am αM -ten guten Tag ($\alpha \in (0, 1)$)

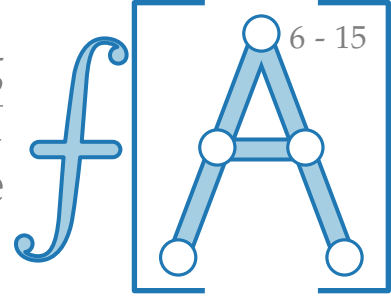


Beobachtung. Schlechtester Fall ist $T = M$ oder $T = \alpha M$

■ Fall $T = M$:
$$\frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot (2M - 1) + \frac{1}{2} \cdot ((1 + \alpha)M - 1)}{M} = \frac{3 + \alpha}{2} - \frac{1}{M}$$

Strategie IV

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Können wir die Schranke durch Randomisierung brechen? Versuchen wir's!

Strategie IV: Werfe eine Münze; **Kopf:** kaufe am M -ten guten Tag
Zahl: kaufe am αM -ten guten Tag ($\alpha \in (0, 1)$)

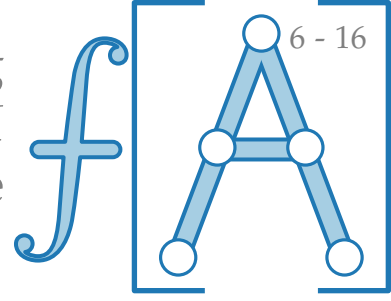


Beobachtung. Schlechtester Fall ist $T = M$ oder $T = \alpha M$

■ Fall $T = M$:
$$\frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot (2M - 1) + \frac{1}{2} \cdot ((1 + \alpha)M - 1)}{M} = \frac{3 + \alpha}{2} - \frac{1}{M} \stackrel{M \rightarrow \infty}{=} \frac{3 + \alpha}{2}$$

Strategie IV

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Können wir die Schranke durch Randomisierung brechen? Versuchen wir's!

Strategie IV: Werfe eine Münze; **Kopf:** kaufe am M -ten guten Tag
Zahl: kaufe am αM -ten guten Tag ($\alpha \in (0, 1)$)



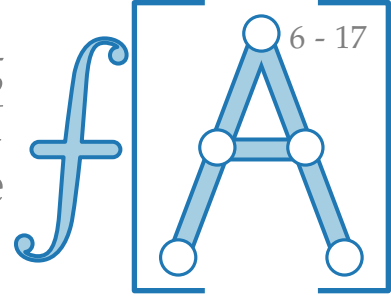
Beobachtung. Schlechtester Fall ist $T = M$ oder $T = \alpha M$

■ Fall $T = M$:
$$\frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot (2M - 1) + \frac{1}{2} \cdot ((1 + \alpha)M - 1)}{M} = \frac{3 + \alpha}{2} - \frac{1}{M} \stackrel{M \rightarrow \infty}{=} \frac{3 + \alpha}{2}$$

■ Fall $T = \alpha M$:
$$\frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}}$$

Strategie IV

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Können wir die Schranke durch Randomisierung brechen? Versuchen wir's!

Strategie IV: Werfe eine Münze; **Kopf:** kaufe am M -ten guten Tag
Zahl: kaufe am αM -ten guten Tag ($\alpha \in (0, 1)$)



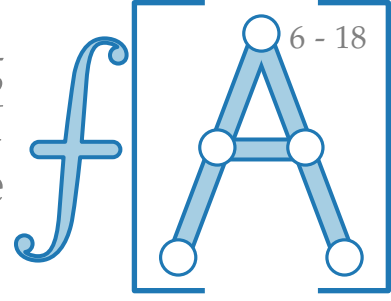
Beobachtung. Schlechtester Fall ist $T = M$ oder $T = \alpha M$

■ Fall $T = M$: $\frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot (2M - 1) + \frac{1}{2} \cdot ((1 + \alpha)M - 1)}{M} = \frac{3 + \alpha}{2} - \frac{1}{M} \stackrel{M \rightarrow \infty}{=} \frac{3 + \alpha}{2}$

■ Fall $T = \alpha M$: $\frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\quad}{\alpha M}$

Strategie IV

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Können wir die Schranke durch Randomisierung brechen? Versuchen wir's!

Strategie IV: Werfe eine Münze; **Kopf:** kaufe am M -ten guten Tag

Zahl: kaufe am αM -ten guten Tag ($\alpha \in (0, 1)$)



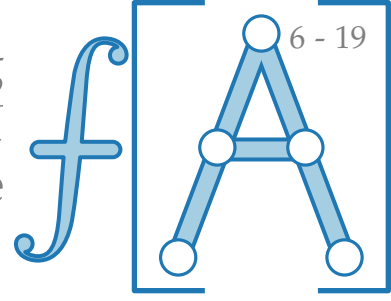
Beobachtung. Schlechtester Fall ist $T = M$ oder $T = \alpha M$

■ Fall $T = M$:
$$\frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot (2M - 1) + \frac{1}{2} \cdot ((1 + \alpha)M - 1)}{M} = \frac{3 + \alpha}{2} - \frac{1}{M} \stackrel{M \rightarrow \infty}{=} \frac{3 + \alpha}{2}$$

■ Fall $T = \alpha M$:
$$\frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot \alpha M}{\alpha M}$$

Strategie IV

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Können wir die Schranke durch Randomisierung brechen? Versuchen wir's!

Strategie IV: Werfe eine Münze; **Kopf:** kaufe am M -ten guten Tag
Zahl: kaufe am αM -ten guten Tag ($\alpha \in (0, 1)$)



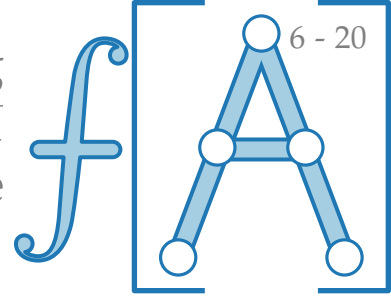
Beobachtung. Schlechtester Fall ist $T = M$ oder $T = \alpha M$

■ Fall $T = M$:
$$\frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot (2M - 1) + \frac{1}{2} \cdot ((1 + \alpha)M - 1)}{M} = \frac{3 + \alpha}{2} - \frac{1}{M} \stackrel{M \rightarrow \infty}{=} \frac{3 + \alpha}{2}$$

■ Fall $T = \alpha M$:
$$\frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot \alpha M + \frac{1}{2} \cdot ((1 + \alpha)M - 1)}{\alpha M}$$

Strategie IV

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Können wir die Schranke durch Randomisierung brechen? Versuchen wir's!

Strategie IV: Werfe eine Münze; **Kopf:** kaufe am M -ten guten Tag
Zahl: kaufe am αM -ten guten Tag ($\alpha \in (0, 1)$)



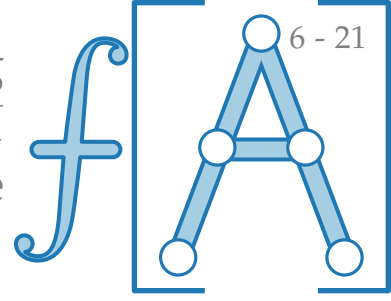
Beobachtung. Schlechtester Fall ist $T = M$ oder $T = \alpha M$

■ Fall $T = M$: $\frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot (2M - 1) + \frac{1}{2} \cdot ((1 + \alpha)M - 1)}{M} = \frac{3 + \alpha}{2} - \frac{1}{M} \stackrel{M \rightarrow \infty}{=} \frac{3 + \alpha}{2}$

■ Fall $T = \alpha M$: $\frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot \alpha M + \frac{1}{2} \cdot ((1 + \alpha)M - 1)}{\alpha M}$

Strategie IV

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Können wir die Schranke durch Randomisierung brechen? Versuchen wir's!

Strategie IV: Werfe eine Münze; **Kopf:** kaufe am M -ten guten Tag

Zahl: kaufe am αM -ten guten Tag ($\alpha \in (0, 1)$)



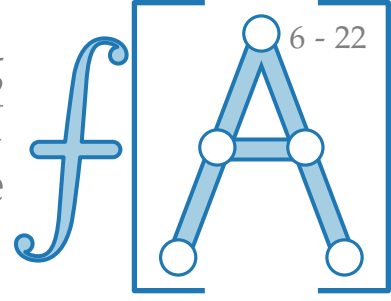
Beobachtung. Schlechtester Fall ist $T = M$ oder $T = \alpha M$

■ Fall $T = M$: $\frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot (2M - 1) + \frac{1}{2} \cdot ((1 + \alpha)M - 1)}{M} = \frac{3 + \alpha}{2} - \frac{1}{M} \stackrel{M \rightarrow \infty}{=} \frac{3 + \alpha}{2}$

■ Fall $T = \alpha M$: $\frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot \alpha M + \frac{1}{2} \cdot ((1 + \alpha)M - 1)}{\alpha M} = 1 + \frac{1}{2\alpha}$

Strategie IV

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Können wir die Schranke durch Randomisierung brechen? Versuchen wir's!

Strategie IV: Werfe eine Münze; **Kopf:** kaufe am M -ten guten Tag

Zahl: kaufe am αM -ten guten Tag ($\alpha \in (0, 1)$)



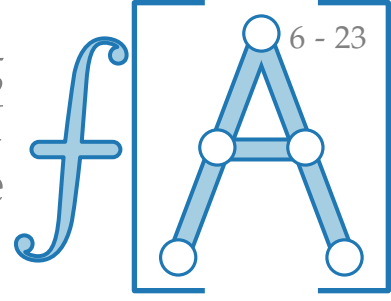
Beobachtung. Schlechtester Fall ist $T = M$ oder $T = \alpha M$

■ Fall $T = M$: $\frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot (2M - 1) + \frac{1}{2} \cdot ((1 + \alpha)M - 1)}{M} = \frac{3 + \alpha}{2} - \frac{1}{M} \stackrel{M \rightarrow \infty}{=} \frac{3 + \alpha}{2}$

■ Fall $T = \alpha M$: $\frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot \alpha M + \frac{1}{2} \cdot ((1 + \alpha)M - 1)}{\alpha M} = 1 + \frac{1}{2\alpha} - \frac{1}{2\alpha M}$

Strategie IV

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Können wir die Schranke durch Randomisierung brechen? Versuchen wir's!

Strategie IV: Werfe eine Münze; **Kopf:** kaufe am M -ten guten Tag

Zahl: kaufe am αM -ten guten Tag ($\alpha \in (0, 1)$)



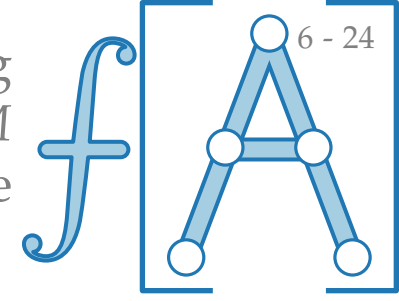
Beobachtung. Schlechtester Fall ist $T = M$ oder $T = \alpha M$

■ Fall $T = M$:
$$\frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot (2M - 1) + \frac{1}{2} \cdot ((1 + \alpha)M - 1)}{M} = \frac{3 + \alpha}{2} - \frac{1}{M} \stackrel{M \rightarrow \infty}{=} \frac{3 + \alpha}{2}$$

■ Fall $T = \alpha M$:
$$\frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot \alpha M + \frac{1}{2} \cdot ((1 + \alpha)M - 1)}{\alpha M} = 1 + \frac{1}{2\alpha} - \frac{1}{2\alpha M} \stackrel{M \rightarrow \infty}{=} 1 + \frac{1}{2\alpha}$$

Strategie IV

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Können wir die Schranke durch Randomisierung brechen? Versuchen wir's!

Strategie IV: Werfe eine Münze; **Kopf:** kaufe am M -ten guten Tag

Zahl: kaufe am αM -ten guten Tag ($\alpha \in (0, 1)$)



Beobachtung. Schlechtester Fall ist $T = M$ oder $T = \alpha M$

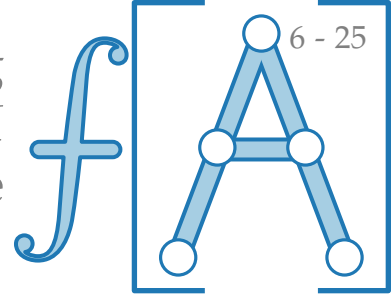
Probiere $\alpha = \frac{1}{2}$

■ Fall $T = M$:
$$\frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot (2M - 1) + \frac{1}{2} \cdot ((1 + \alpha)M - 1)}{M} = \frac{3 + \alpha}{2} - \frac{1}{M} \stackrel{M \rightsquigarrow \infty}{=} \frac{3 + \alpha}{2}$$

■ Fall $T = \alpha M$:
$$\frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot \alpha M + \frac{1}{2} \cdot ((1 + \alpha)M - 1)}{\alpha M} = 1 + \frac{1}{2\alpha} - \frac{1}{2\alpha M} \stackrel{M \rightsquigarrow \infty}{=} 1 + \frac{1}{2\alpha}$$

Strategie IV

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Können wir die Schranke durch Randomisierung brechen? Versuchen wir's!

Strategie IV: Werfe eine Münze; **Kopf:** kaufe am M -ten guten Tag

Zahl: kaufe am αM -ten guten Tag ($\alpha \in (0, 1)$)



Beobachtung. Schlechtester Fall ist $T = M$ oder $T = \alpha M$

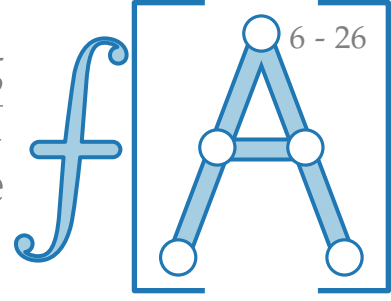
Probiere $\alpha = \frac{1}{2}$

$$\blacksquare \text{ Fall } T = M: \frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot (2M - 1) + \frac{1}{2} \cdot ((1 + \alpha)M - 1)}{M} = \frac{3 + \alpha}{2} - \frac{1}{M} \stackrel{M \rightsquigarrow \infty}{=} \frac{3 + \alpha}{2} = \frac{7}{4} < 2$$

$$\blacksquare \text{ Fall } T = \alpha M: \frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot \alpha M + \frac{1}{2} \cdot ((1 + \alpha)M - 1)}{\alpha M} = 1 + \frac{1}{2\alpha} - \frac{1}{2\alpha M} \stackrel{M \rightsquigarrow \infty}{=} 1 + \frac{1}{2\alpha}$$

Strategie IV

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Können wir die Schranke durch Randomisierung brechen? Versuchen wir's!

Strategie IV: Werfe eine Münze; **Kopf:** kaufe am M -ten guten Tag

Zahl: kaufe am αM -ten guten Tag ($\alpha \in (0, 1)$)



Beobachtung. Schlechtester Fall ist $T = M$ oder $T = \alpha M$

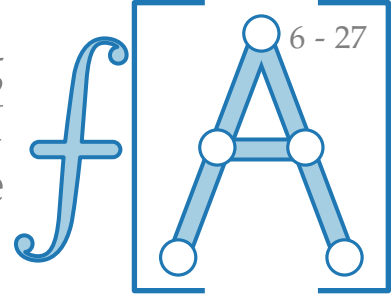
Probiere $\alpha = \frac{1}{2}$

$$\blacksquare \text{ Fall } T = M: \frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot (2M - 1) + \frac{1}{2} \cdot ((1 + \alpha)M - 1)}{M} = \frac{3 + \alpha}{2} - \frac{1}{M} \stackrel{M \rightsquigarrow \infty}{=} \frac{3 + \alpha}{2} = \frac{7}{4} < 2$$

$$\blacksquare \text{ Fall } T = \alpha M: \frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot \alpha M + \frac{1}{2} \cdot ((1 + \alpha)M - 1)}{\alpha M} = 1 + \frac{1}{2\alpha} - \frac{1}{2\alpha M} \stackrel{M \rightsquigarrow \infty}{=} 1 + \frac{1}{2\alpha} = 2$$

Strategie IV

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Können wir die Schranke durch Randomisierung brechen? Versuchen wir's!

Strategie IV: Werfe eine Münze; **Kopf:** kaufe am M -ten guten Tag

Zahl: kaufe am αM -ten guten Tag ($\alpha \in (0, 1)$)



Beobachtung. Schlechtester Fall ist $T = M$ oder $T = \alpha M$

Probiere $\alpha = \frac{1}{2}$

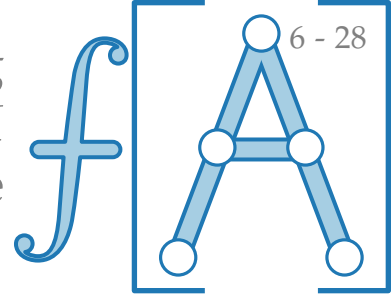
$$\blacksquare \text{ Fall } T = M: \frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot (2M - 1) + \frac{1}{2} \cdot ((1 + \alpha)M - 1)}{M} = \frac{3 + \alpha}{2} - \frac{1}{M} \stackrel{M \rightsquigarrow \infty}{=} \frac{3 + \alpha}{2} = \frac{7}{4} < 2$$

$$\blacksquare \text{ Fall } T = \alpha M: \frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot \alpha M + \frac{1}{2} \cdot ((1 + \alpha)M - 1)}{\alpha M} = 1 + \frac{1}{2\alpha} - \frac{1}{2\alpha M} \stackrel{M \rightsquigarrow \infty}{=} 1 + \frac{1}{2\alpha} = 2$$

nicht besser als deterministische Strategie III

Strategie IV

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Können wir die Schranke durch Randomisierung brechen? Versuchen wir's!

Strategie IV: Werfe eine Münze; **Kopf:** kaufe am M -ten guten Tag

Zahl: kaufe am αM -ten guten Tag ($\alpha \in (0, 1)$)



Beobachtung. Schlechtester Fall ist $T = M$ oder $T = \alpha M$

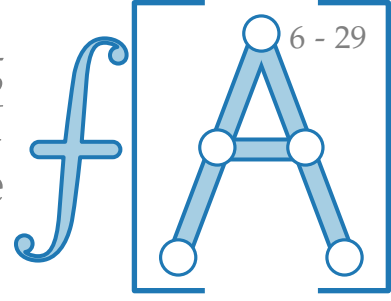
■ Fall $T = M$: $\frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot (2M - 1) + \frac{1}{2} \cdot ((1 + \alpha)M - 1)}{M} = \frac{3 + \alpha}{2} - \frac{1}{M} \stackrel{M \rightsquigarrow \infty}{=} \frac{3 + \alpha}{2}$

■ Fall $T = \alpha M$: $\frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot \alpha M + \frac{1}{2} \cdot ((1 + \alpha)M - 1)}{\alpha M} = 1 + \frac{1}{2\alpha} - \frac{1}{2\alpha M} \stackrel{M \rightsquigarrow \infty}{=} 1 + \frac{1}{2\alpha}$

■ Das Verhältnis ist minimal wenn $\frac{3 + \alpha}{2} = 1 + \frac{1}{2\alpha}$

Strategie IV

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Können wir die Schranke durch Randomisierung brechen? Versuchen wir's!

Strategie IV: Werfe eine Münze; **Kopf:** kaufe am M -ten guten Tag
Zahl: kaufe am αM -ten guten Tag ($\alpha \in (0, 1)$)



Beobachtung. Schlechtester Fall ist $T = M$ oder $T = \alpha M$

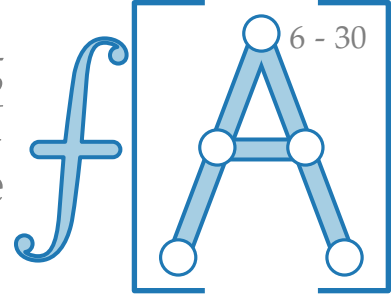
■ Fall $T = M$: $\frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot (2M - 1) + \frac{1}{2} \cdot ((1 + \alpha)M - 1)}{M} = \frac{3 + \alpha}{2} - \frac{1}{M} \stackrel{M \rightarrow \infty}{=} \frac{3 + \alpha}{2}$

■ Fall $T = \alpha M$: $\frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot \alpha M + \frac{1}{2} \cdot ((1 + \alpha)M - 1)}{\alpha M} = 1 + \frac{1}{2\alpha} - \frac{1}{2\alpha M} \stackrel{M \rightarrow \infty}{=} 1 + \frac{1}{2\alpha}$

■ Das Verhältnis ist minimal wenn $\frac{3 + \alpha}{2} = 1 + \frac{1}{2\alpha} \Rightarrow \alpha = \frac{\sqrt{5} - 1}{2}$

Strategie IV

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Können wir die Schranke durch Randomisierung brechen? Versuchen wir's!

Strategie IV: Werfe eine Münze; **Kopf:** kaufe am M -ten guten Tag
Zahl: kaufe am αM -ten guten Tag ($\alpha \in (0, 1)$)



Beobachtung. Schlechtester Fall ist $T = M$ oder $T = \alpha M$

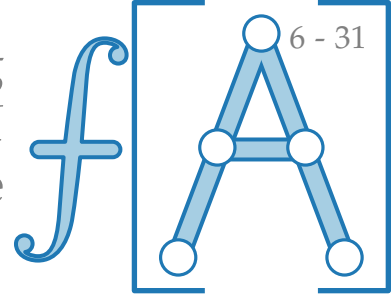
■ Fall $T = M$: $\frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot (2M - 1) + \frac{1}{2} \cdot ((1 + \alpha)M - 1)}{M} = \frac{3 + \alpha}{2} - \frac{1}{M} \stackrel{M \rightarrow \infty}{=} \frac{3 + \alpha}{2}$

■ Fall $T = \alpha M$: $\frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot \alpha M + \frac{1}{2} \cdot ((1 + \alpha)M - 1)}{\alpha M} = 1 + \frac{1}{2\alpha} - \frac{1}{2\alpha M} \stackrel{M \rightarrow \infty}{=} 1 + \frac{1}{2\alpha}$

■ Das Verhältnis ist minimal wenn $\frac{3 + \alpha}{2} = 1 + \frac{1}{2\alpha} \Rightarrow \alpha = \frac{\sqrt{5} - 1}{2} \approx 0.62$

Strategie IV

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Können wir die Schranke durch Randomisierung brechen? Versuchen wir's!

Strategie IV: Werfe eine Münze; **Kopf:** kaufe am M -ten guten Tag

Zahl: kaufe am αM -ten guten Tag ($\alpha \in (0, 1)$)



Beobachtung. Schlechtester Fall ist $T = M$ oder $T = \alpha M$

■ Fall $T = M$: $\frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot (2M - 1) + \frac{1}{2} \cdot ((1 + \alpha)M - 1)}{M} = \frac{3 + \alpha}{2} - \frac{1}{M} \stackrel{M \rightarrow \infty}{=} \frac{3 + \alpha}{2}$

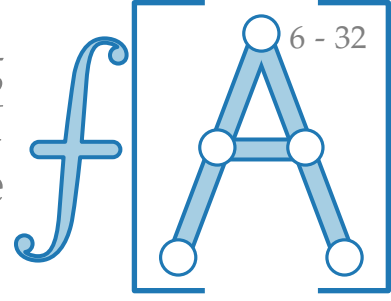
■ Fall $T = \alpha M$: $\frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot \alpha M + \frac{1}{2} \cdot ((1 + \alpha)M - 1)}{\alpha M} = 1 + \frac{1}{2\alpha} - \frac{1}{2\alpha M} \stackrel{M \rightarrow \infty}{=} 1 + \frac{1}{2\alpha}$

■ Das Verhältnis ist minimal wenn $\frac{3 + \alpha}{2} = 1 + \frac{1}{2\alpha} \Rightarrow \alpha = \frac{\sqrt{5} - 1}{2} \approx 0.62$

→ Strategie IV (mit $\alpha = \frac{\sqrt{5} - 1}{2} \approx 0.62$) ist **1.81-kompetitiv**, randomisiert, und besser als jede deterministische Strategie.

Strategie IV

Leihen kostet 1/Tag
Kaufen kostet M
 T gute Tage



Können wir die Schranke durch Randomisierung brechen? Versuchen wir's!

Strategie IV: Werfe eine Münze; **Kopf:** kaufe am M -ten guten Tag
Zahl: kaufe am αM -ten guten Tag ($\alpha \in (0, 1)$)



Beobachtung. Schlechtester Fall ist $T = M$ oder $T = \alpha M$

■ Fall $T = M$: $\frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot (2M - 1) + \frac{1}{2} \cdot ((1 + \alpha)M - 1)}{M} = \frac{3 + \alpha}{2} - \frac{1}{M} \stackrel{M \rightarrow \infty}{=} \frac{3 + \alpha}{2}$

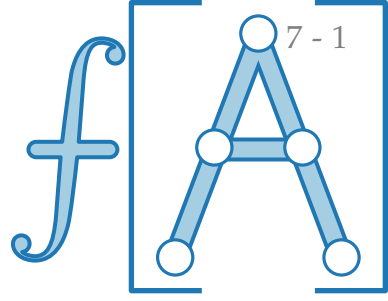
■ Fall $T = \alpha M$: $\frac{E[c_{\text{Strategie IV}}]}{c_{\text{OPT}}} = \frac{\frac{1}{2} \cdot \alpha M + \frac{1}{2} \cdot ((1 + \alpha)M - 1)}{\alpha M} = 1 + \frac{1}{2\alpha} - \frac{1}{2\alpha M} \stackrel{M \rightarrow \infty}{=} 1 + \frac{1}{2\alpha}$

■ Das Verhältnis ist minimal wenn $\frac{3 + \alpha}{2} = 1 + \frac{1}{2\alpha} \Rightarrow \alpha = \frac{\sqrt{5} - 1}{2} \approx 0.62$

➔ Strategie IV (mit $\alpha = \frac{\sqrt{5} - 1}{2} \approx 0.62$) ist **1.81-kompetitiv**, randomisiert, und besser als jede deterministische Strategie.

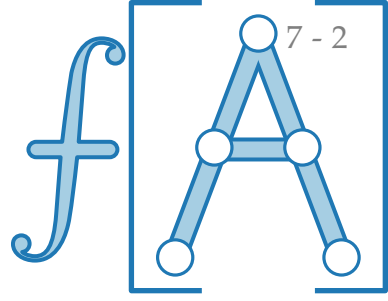
■ Mit einer ausgeklügelteren Wahrscheinlichkeitsverteilung können wir ein kompetitives Verhältnis von $\frac{e}{e-1} \approx 1.58$ erreichen.

Online vs. Offline Algorithmen



Online vs. Offline Algorithmen

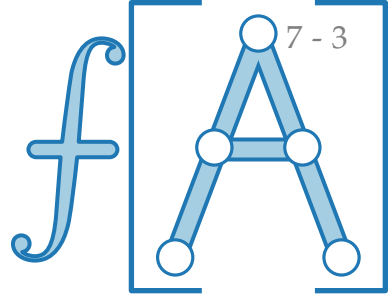
Online Algorithmus



Online vs. Offline Algorithmen

Online Algorithmus

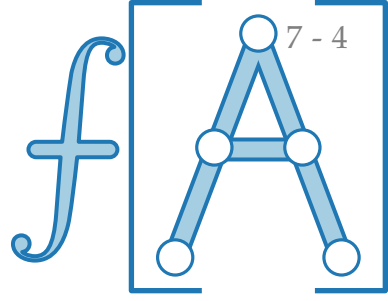
- Anfangs nicht alle Informationen verfügbar (*Online Problem*)



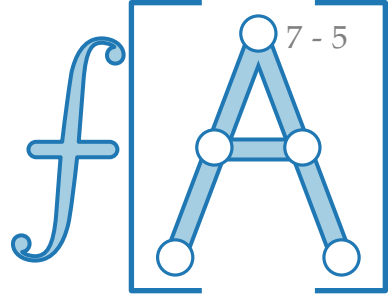
Online vs. Offline Algorithmen

Online Algorithmus

- Anfangs nicht alle Informationen verfügbar (*Online Problem*)
- Entscheidungen werden mit unvollständigen Informationen gefällt.



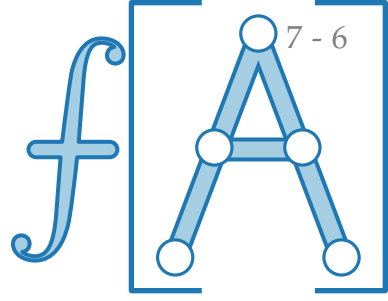
Online vs. Offline Algorithmen



Online Algorithmus

- Anfangs nicht alle Informationen verfügbar (*Online Problem*)
- Entscheidungen werden mit unvollständigen Informationen gefällt.
- Der Algorithmus kann über Zeit mehr Informationen gewinnen.

Online vs. Offline Algorithmen

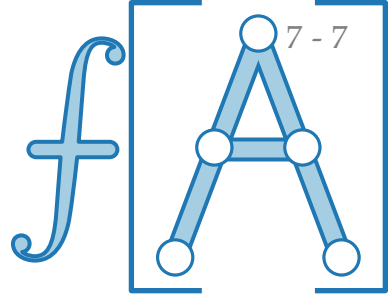


Online Algorithmus

- Anfangs nicht alle Informationen verfügbar (*Online Problem*)
- Entscheidungen werden mit unvollständigen Informationen gefällt.
- Der Algorithmus kann über Zeit mehr Informationen gewinnen.

Offline Algorithmus

Online vs. Offline Algorithmen



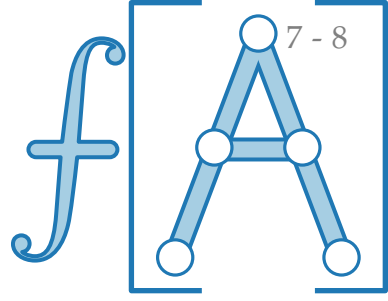
Online Algorithmus

- Anfangs nicht alle Informationen verfügbar (*Online Problem*)
- Entscheidungen werden mit unvollständigen Informationen gefällt.
- Der Algorithmus kann über Zeit mehr Informationen gewinnen.

Offline Algorithmus

- Anfangs alle Informationen verfügbar (*Offline Problem*)

Online vs. Offline Algorithmen



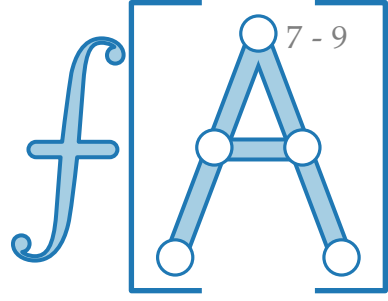
Online Algorithmus

- Anfangs nicht alle Informationen verfügbar (*Online Problem*)
- Entscheidungen werden mit unvollständigen Informationen gefällt.
- Der Algorithmus kann über Zeit mehr Informationen gewinnen.

Offline Algorithmus

- Anfangs alle Informationen verfügbar (*Offline Problem*)
- Entscheidungen werden mit vollständigen Informationen gefällt.

Online vs. Offline Algorithmen



Online Algorithmus

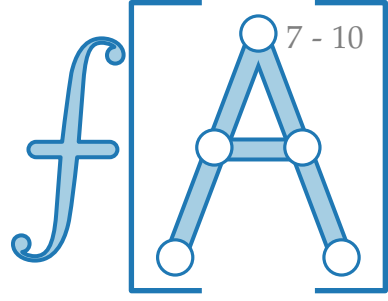
- Anfangs nicht alle Informationen verfügbar (*Online Problem*)
- Entscheidungen werden mit unvollständigen Informationen gefällt.
- Der Algorithmus kann über Zeit mehr Informationen gewinnen.

Offline Algorithmus

- Anfangs alle Informationen verfügbar (*Offline Problem*)
- Entscheidungen werden mit vollständigen Informationen gefällt.

- Der Wert der zurückgegebenen Lösung geteilt durch den Wert einer optimalen (off-line) Lösung ist das **kompetitive Verhältnis**.

Online vs. Offline Algorithmen



Online Algorithmus

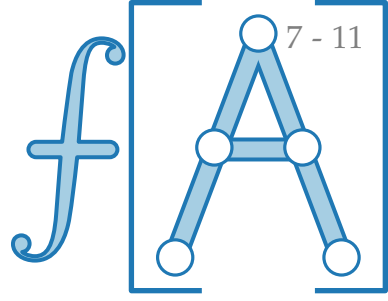
- Anfangs nicht alle Informationen verfügbar (*Online Problem*)
- Entscheidungen werden mit unvollständigen Informationen gefällt.
- Der Algorithmus kann über Zeit mehr Informationen gewinnen.

Offline Algorithmus

- Anfangs alle Informationen verfügbar (*Offline Problem*)
- Entscheidungen werden mit vollständigen Informationen gefällt.

- Der Wert der zurückgegebenen Lösung geteilt durch den Wert einer optimalen (off-line) Lösung ist das **kompetitive Verhältnis**.

Online vs. Offline Algorithmen



Online Algorithmus

- Anfangs nicht alle Informationen verfügbar (*Online Problem*)
- Entscheidungen werden mit unvollständigen Informationen gefällt.
- Der Algorithmus kann über Zeit mehr Informationen gewinnen.

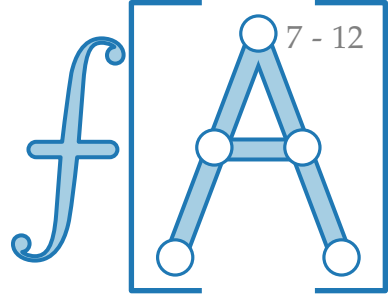
Offline Algorithmus

- Anfangs alle Informationen verfügbar (*Offline Problem*)
- Entscheidungen werden mit vollständigen Informationen gefällt.

worst-case (determ. Algo.)

- Der Wert der zurückgegebenen Lösung geteilt durch den Wert einer optimalen (offline) Lösung ist das **kompetitive Verhältnis**.

Online vs. Offline Algorithmen



Online Algorithmus

- Anfangs nicht alle Informationen verfügbar (*Online Problem*)
- Entscheidungen werden mit unvollständigen Informationen gefällt.
- Der Algorithmus kann über Zeit mehr Informationen gewinnen.

Offline Algorithmus

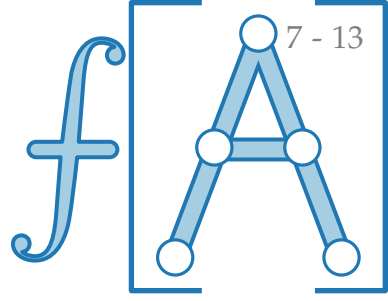
- Anfangs alle Informationen verfügbar (*Offline Problem*)
- Entscheidungen werden mit vollständigen Informationen gefällt.

worst-case (determ. Algo.)

durchschn. worst-case (random. Algo.)

- Der Wert der zurückgegebenen Lösung geteilt durch den Wert einer optimalen (off-line) Lösung ist das **kompetitive Verhältnis**.

Online vs. Offline Algorithmen



Online Algorithmus

- Anfangs nicht alle Informationen verfügbar (*Online Problem*)
- Entscheidungen werden mit unvollständigen Informationen gefällt.
- Der Algorithmus kann über Zeit mehr Informationen gewinnen.

Offline Algorithmus

- Anfangs alle Informationen verfügbar (*Offline Problem*)
- Entscheidungen werden mit vollständigen Informationen gefällt.

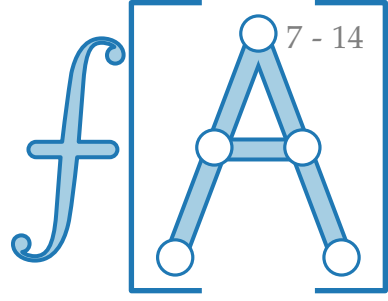
worst-case (determ. Algo.)

durchschn. worst-case (random. Algo.)

- Der Wert der zurückgegebenen Lösung geteilt durch den Wert einer optimalen (off-line) Lösung ist das **kompetitive Verhältnis**.

Beispiele. (Probleme & Algorithmen)

Online vs. Offline Algorithmen



Online Algorithmus

- Anfangs nicht alle Informationen verfügbar (*Online Problem*)
- Entscheidungen werden mit unvollständigen Informationen gefällt.
- Der Algorithmus kann über Zeit mehr Informationen gewinnen.

Offline Algorithmus

- Anfangs alle Informationen verfügbar (*Offline Problem*)
- Entscheidungen werden mit vollständigen Informationen gefällt.

worst-case (determ. Algo.)

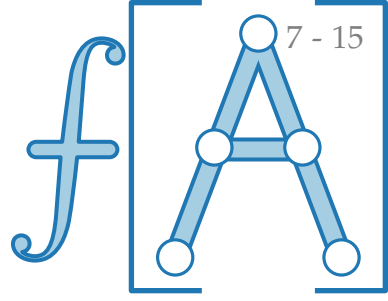
durchschn. worst-case (random. Algo.)

- Der Wert der zurückgegebenen Lösung geteilt durch den Wert einer optimalen (off-line) Lösung ist das **kompetitive Verhältnis**.

Beispiele. (Probleme & Algorithmen)

Ski-Verleih Problem

Online vs. Offline Algorithmen



Online Algorithmus

- Anfangs nicht alle Informationen verfügbar (*Online Problem*)
- Entscheidungen werden mit unvollständigen Informationen gefällt.
- Der Algorithmus kann über Zeit mehr Informationen gewinnen.

Offline Algorithmus

- Anfangs alle Informationen verfügbar (*Offline Problem*)
- Entscheidungen werden mit vollständigen Informationen gefällt.

worst-case (determ. Algo.)

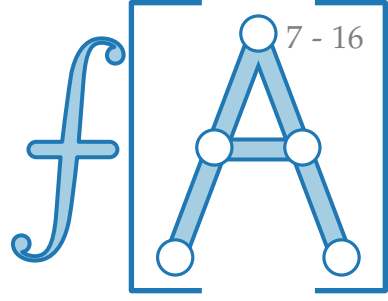
durchschn. worst-case (random. Algo.)

- Der Wert der zurückgegebenen Lösung geteilt durch den Wert einer optimalen (off-line) Lösung ist das **kompetitive Verhältnis**.

Beispiele. (Probleme & Algorithmen)

Ski-Verleih Problem, Suchen in unbekannten Umgebungen,

Online vs. Offline Algorithmen



Online Algorithmus

- Anfangs nicht alle Informationen verfügbar (*Online Problem*)
- Entscheidungen werden mit unvollständigen Informationen gefällt.
- Der Algorithmus kann über Zeit mehr Informationen gewinnen.

Offline Algorithmus

- Anfangs alle Informationen verfügbar (*Offline Problem*)
- Entscheidungen werden mit vollständigen Informationen gefällt.

worst-case (determ. Algo.)

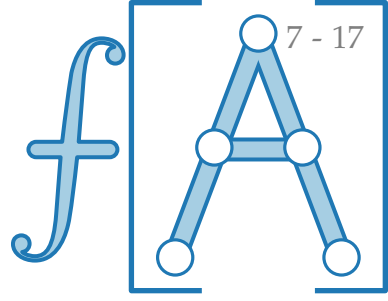
durchschn. worst-case (random. Algo.)

- Der Wert der zurückgegebenen Lösung geteilt durch den Wert einer optimalen (off-line) Lösung ist das **kompetitive Verhältnis**.

Beispiele. (Probleme & Algorithmen)

Ski-Verleih Problem, Suchen in unbekannten Umgebungen, Kuh-Pfad Problem,

Online vs. Offline Algorithmen



Online Algorithmus

- Anfangs nicht alle Informationen verfügbar (*Online Problem*)
- Entscheidungen werden mit unvollständigen Informationen gefällt.
- Der Algorithmus kann über Zeit mehr Informationen gewinnen.

Offline Algorithmus

- Anfangs alle Informationen verfügbar (*Offline Problem*)
- Entscheidungen werden mit vollständigen Informationen gefällt.

worst-case (determ. Algo.)

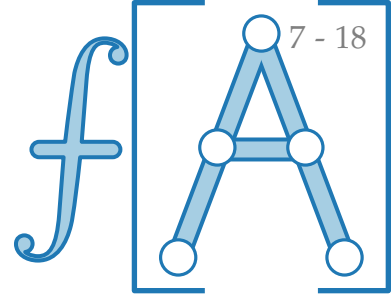
durchschn. worst-case (random. Algo.)

- Der Wert der zurückgegebenen Lösung geteilt durch den Wert einer optimalen (off-line) Lösung ist das **kompetitive Verhältnis**.

Beispiele. (Probleme & Algorithmen)

Ski-Verleih Problem, Suchen in unbekannten Umgebungen, Kuh-Pfad Problem, Werkstattfertigung,

Online vs. Offline Algorithmen



Online Algorithmus

- Anfangs nicht alle Informationen verfügbar (*Online Problem*)
- Entscheidungen werden mit unvollständigen Informationen gefällt.
- Der Algorithmus kann über Zeit mehr Informationen gewinnen.

Offline Algorithmus

- Anfangs alle Informationen verfügbar (*Offline Problem*)
- Entscheidungen werden mit vollständigen Informationen gefällt.

worst-case (determ. Algo.)

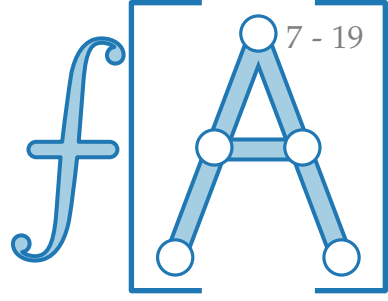
durchschn. worst-case (random. Algo.)

- Der Wert der zurückgegebenen Lösung geteilt durch den Wert einer optimalen (off-line) Lösung ist das **kompetitive Verhältnis**.

Beispiele. (Probleme & Algorithmen)

Ski-Verleih Problem, Suchen in unbekannten Umgebungen, Kuh-Pfad Problem, Werkstattfertigung, PAGING (Einträge im Speicher ersetzen),

Online vs. Offline Algorithmen



Online Algorithmus

- Anfangs nicht alle Informationen verfügbar (*Online Problem*)
- Entscheidungen werden mit unvollständigen Informationen gefällt.
- Der Algorithmus kann über Zeit mehr Informationen gewinnen.

Offline Algorithmus

- Anfangs alle Informationen verfügbar (*Offline Problem*)
- Entscheidungen werden mit vollständigen Informationen gefällt.

worst-case (determ. Algo.)

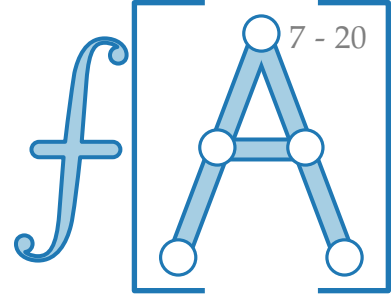
durchschn. worst-case (random. Algo.)

- Der Wert der zurückgegebenen Lösung geteilt durch den Wert einer optimalen (off-line) Lösung ist das **kompetitive Verhältnis**.

Beispiele. (Probleme & Algorithmen)

Ski-Verleih Problem, Suchen in unbekannten Umgebungen, Kuh-Pfad Problem, Werkstattfertigung, PAGING (Einträge im Speicher ersetzen), INSERTIONSORT

Online vs. Offline Algorithmen



Online Algorithmus

- Anfangs nicht alle Informationen verfügbar (*Online Problem*)
- Entscheidungen werden mit unvollständigen Informationen gefällt.
- Der Algorithmus kann über Zeit mehr Informationen gewinnen.

Offline Algorithmus

- Anfangs alle Informationen verfügbar (*Offline Problem*)
- Entscheidungen werden mit vollständigen Informationen gefällt.

worst-case (determ. Algo.)

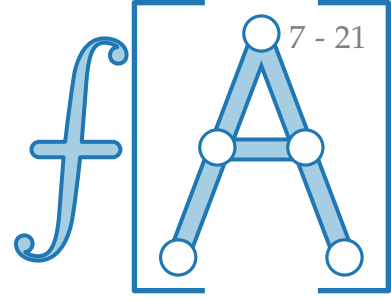
durchschn. worst-case (random. Algo.)

- Der Wert der zurückgegebenen Lösung geteilt durch den Wert einer optimalen (off-line) Lösung ist das **kompetitive Verhältnis**.

Beispiele. (Probleme & Algorithmen)

Ski-Verleih Problem, Suchen in unbekannten Umgebungen, Kuh-Pfad Problem, Werkstattfertigung, PAGING (Einträge im Speicher ersetzen), INSERTIONSORT

Online vs. Offline Algorithmen



Online Algorithmus

- Anfangs nicht alle Informationen verfügbar (*Online Problem*)
- Entscheidungen werden mit unvollständigen Informationen gefällt.
- Der Algorithmus kann über Zeit mehr Informationen gewinnen.

Offline Algorithmus

- Anfangs alle Informationen verfügbar (*Offline Problem*)
- Entscheidungen werden mit vollständigen Informationen gefällt.

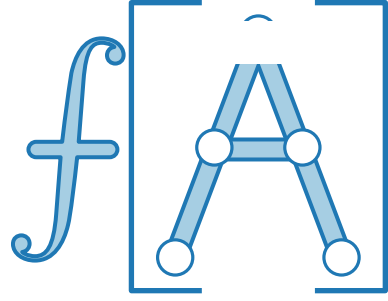
worst-case (determ. Algo.)

durchschn. worst-case (random. Algo.)

- Der Wert der zurückgegebenen Lösung geteilt durch den Wert einer optimalen (off-line) Lösung ist das **kompetitive Verhältnis**.

Beispiele. (Probleme & Algorithmen)

Ski-Verleih Problem, Suchen in unbekannten Umgebungen, Kuh-Pfad Problem, Werkstattfertigung, PAGING (Einträge im Speicher ersetzen), INSERTIONSORT



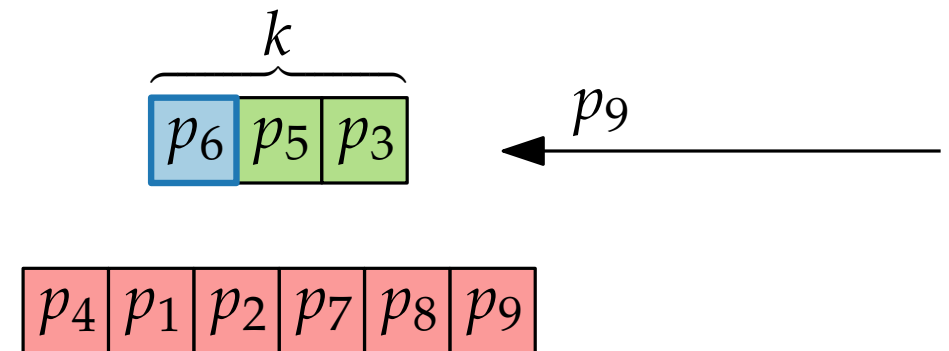
Fortgeschrittene Algorithmen

6. Vorlesung Online Algorithmen

Teil II: PAGING



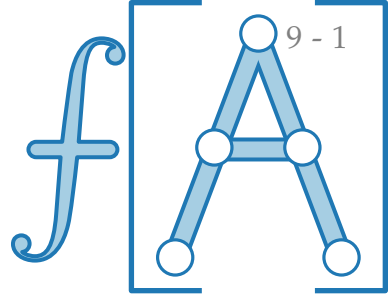
Philipp Kindermann



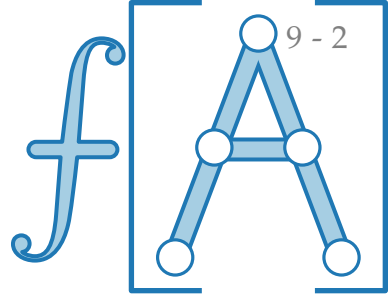
Wintersemester 2025/26

Problemstellung

Gegeben ([offline](#)/[online](#)):



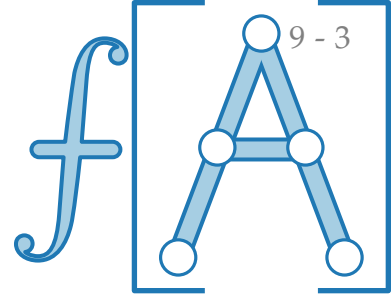
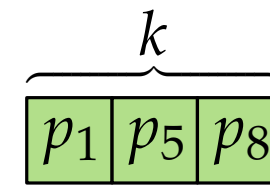
Problemstellung



Gegeben (**offline**/**online**):

- Speicher mit schnellem Zugriff (**Cache**) mit k Seiten Kapazität

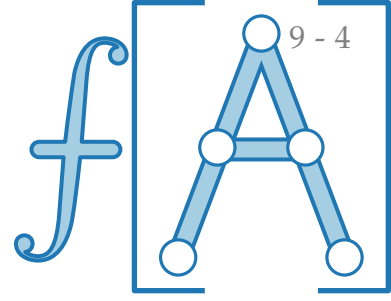
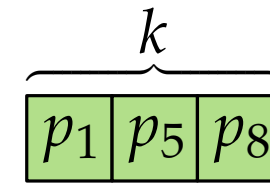
Problemstellung



Gegeben (**offline**/**online**):

- Speicher mit schnellem Zugriff (**Cache**) mit k Seiten Kapazität

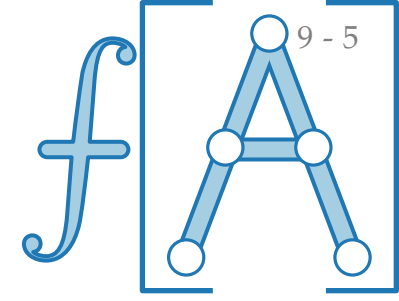
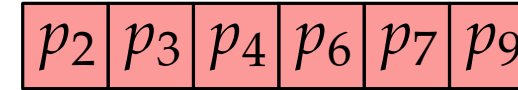
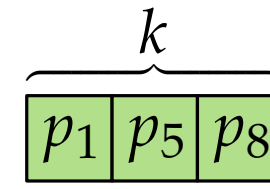
Problemstellung



Gegeben (**offline**/**online**):

- Speicher mit schnellem Zugriff (**Cache**) mit k Seiten Kapazität
- Speicher mit langsamem Zugriff (**Festplatte**) mit unbegrenzter Kapazität

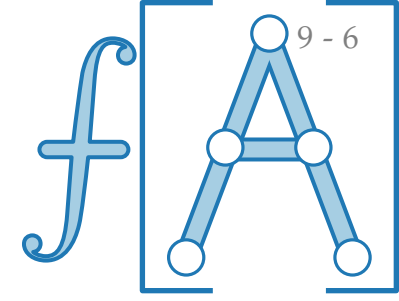
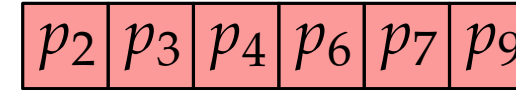
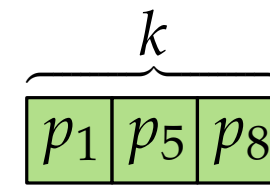
Problemstellung



Gegeben (**offline**/**online**):

- Speicher mit schnellem Zugriff (**Cache**) mit k Seiten Kapazität
- Speicher mit langsamem Zugriff (**Festplatte**) mit unbegrenzter Kapazität

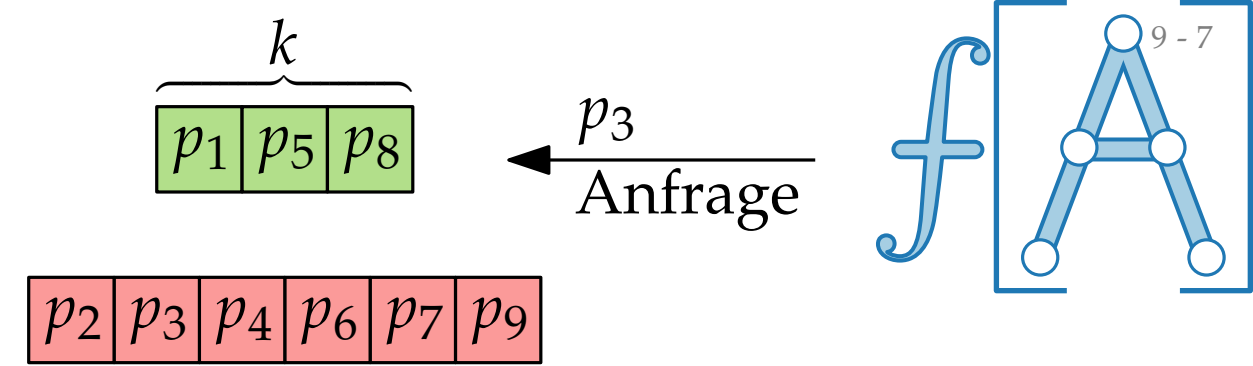
Problemstellung



Gegeben (**offline**/**online**):

- Speicher mit schnellem Zugriff (**Cache**) mit k Seiten Kapazität
- Speicher mit langsamem Zugriff (**Festplatte**) mit unbegrenzter Kapazität
- Wenn eine Seite angefragt wird, aber nicht im Cache liegt (**Seitenfehler**), muss sie mit einer Seite im Cache getauscht werden (**Seitentausch**).

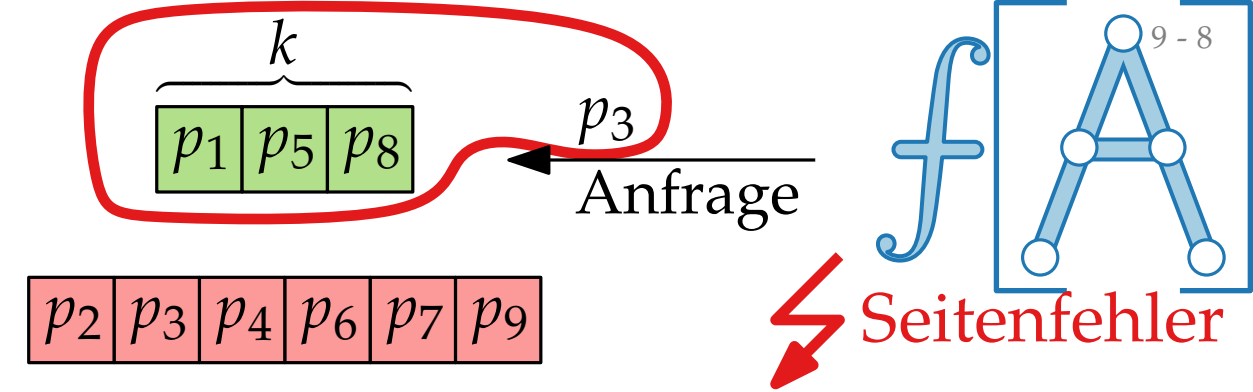
Problemstellung



Gegeben (**offline**/**online**):

- Speicher mit schnellem Zugriff (**Cache**) mit k Seiten Kapazität
- Speicher mit langsamem Zugriff (**Festplatte**) mit unbegrenzter Kapazität
- Wenn eine Seite angefragt wird, aber nicht im Cache liegt (**Seitenfehler**), muss sie mit einer Seite im Cache getauscht werden (**Seitentausch**).

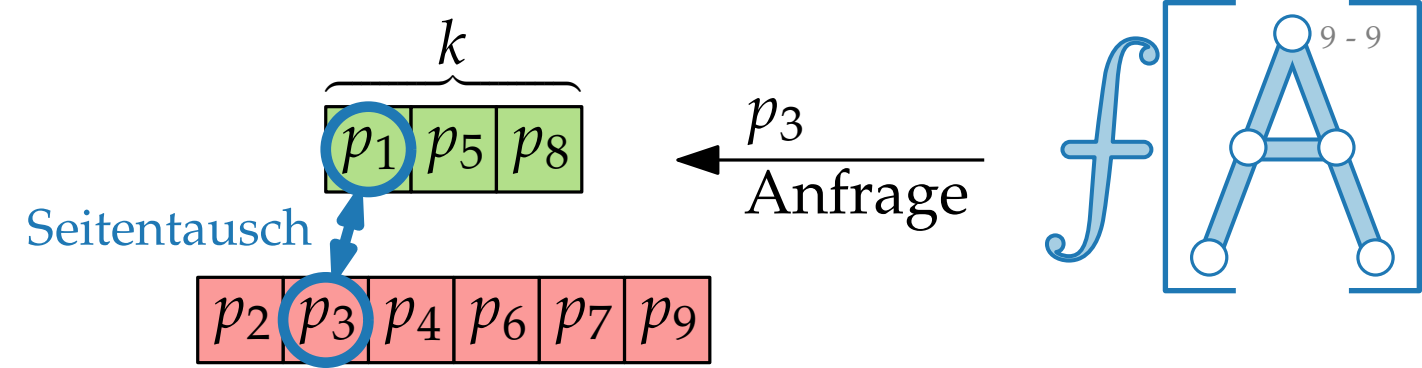
Problemstellung



Gegeben (**offline**/**online**):

- Speicher mit schnellem Zugriff (**Cache**) mit k Seiten Kapazität
- Speicher mit langsamem Zugriff (**Festplatte**) mit unbegrenzter Kapazität
- Wenn eine Seite angefragt wird, aber nicht im Cache liegt (**Seitenfehler**), muss sie mit einer Seite im Cache getauscht werden (**Seitentausch**).

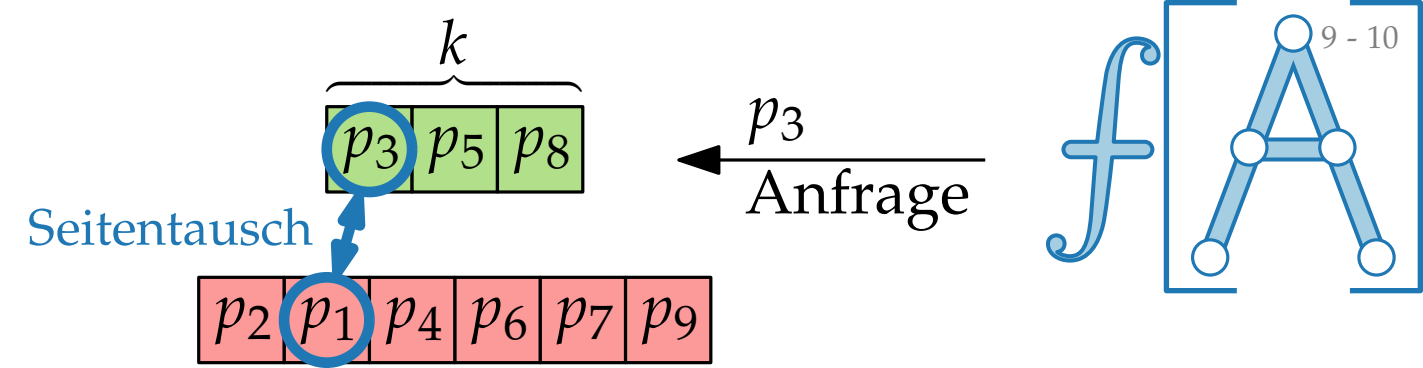
Problemstellung



Gegeben (**offline**/**online**):

- Speicher mit schnellem Zugriff (**Cache**) mit k Seiten Kapazität
- Speicher mit langsamem Zugriff (**Festplatte**) mit unbegrenzter Kapazität
- Wenn eine Seite angefragt wird, aber nicht im Cache liegt (**Seitenfehler**), muss sie mit einer Seite im Cache getauscht werden (**Seitentausch**).

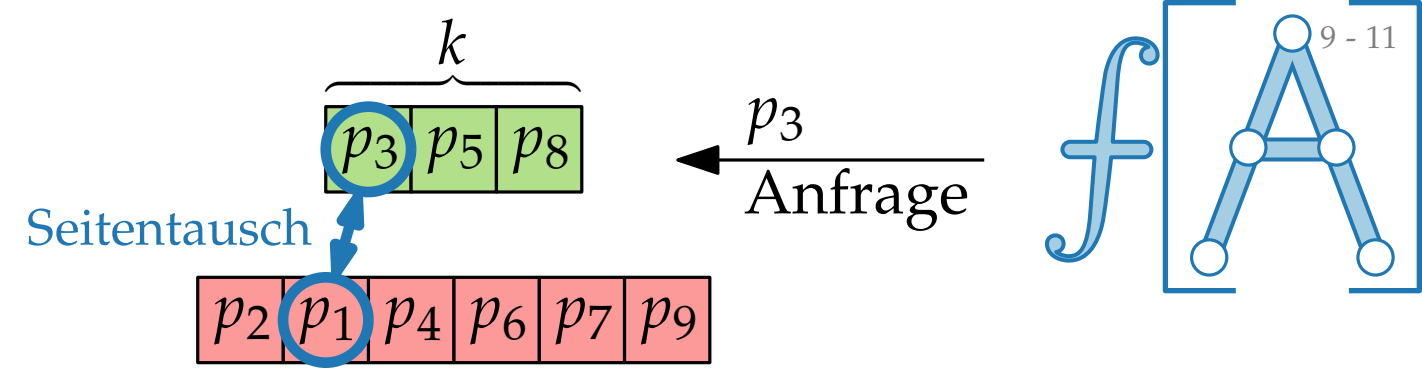
Problemstellung



Gegeben (**offline**/**online**):

- Speicher mit schnellem Zugriff (**Cache**) mit k Seiten Kapazität
- Speicher mit langsamem Zugriff (**Festplatte**) mit unbegrenzter Kapazität
- Wenn eine Seite angefragt wird, aber nicht im Cache liegt (**Seitenfehler**), muss sie mit einer Seite im Cache getauscht werden (**Seitentausch**).

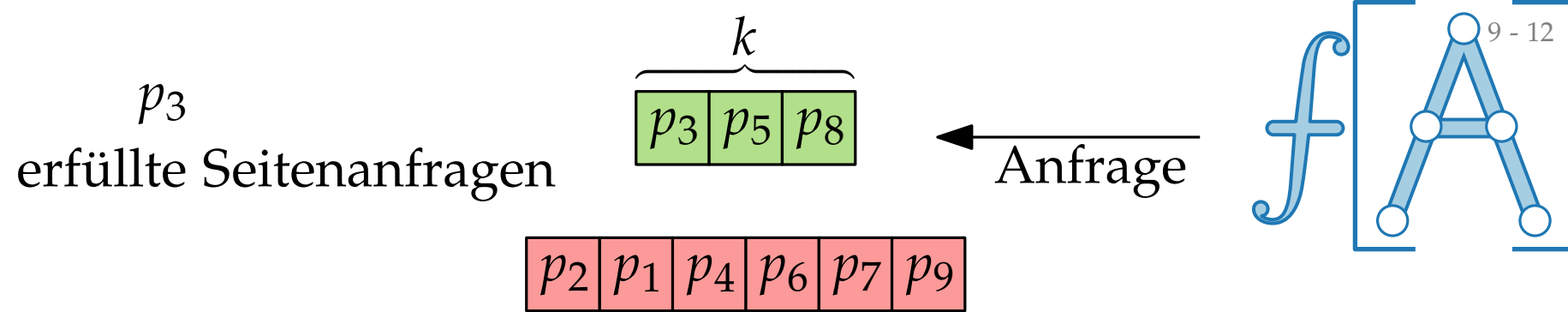
Problemstellung



Gegeben (**offline**/**online**):

- Speicher mit schnellem Zugriff (**Cache**) mit k Seiten Kapazität
- Speicher mit langsamem Zugriff (**Festplatte**) mit unbegrenzter Kapazität
- Wenn eine Seite angefragt wird, aber nicht im Cache liegt (**Seitenfehler**), muss sie mit einer Seite im Cache getauscht werden (**Seitentausch**).
Eine Seitenanfrage ist **erfüllt**, wenn die Seite im Cache liegt.

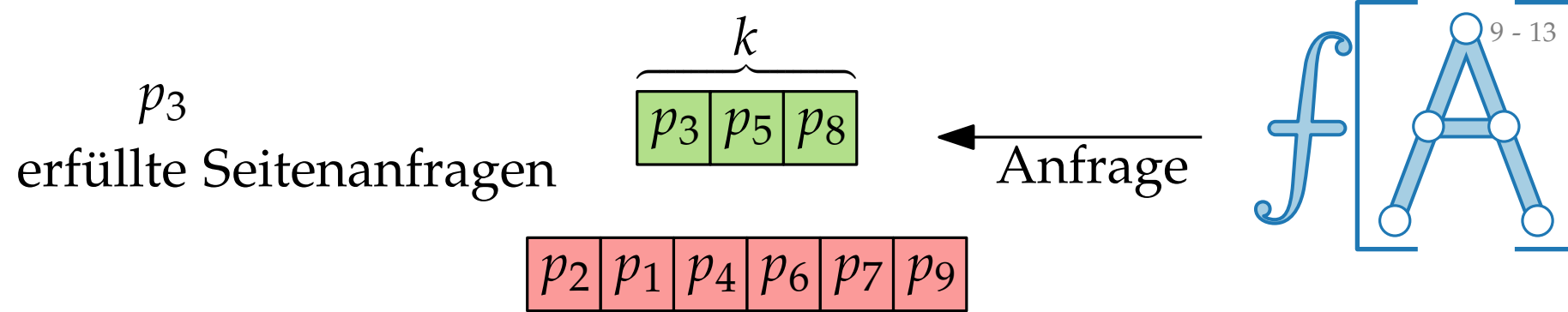
Problemstellung



Gegeben (**offline**/**online**):

- Speicher mit schnellem Zugriff (**Cache**) mit k Seiten Kapazität
- Speicher mit langsamem Zugriff (**Festplatte**) mit unbegrenzter Kapazität
- Wenn eine Seite angefragt wird, aber nicht im Cache liegt (**Seitenfehler**), muss sie mit einer Seite im Cache getauscht werden (**Seitentausch**).
Eine Seitenanfrage ist **erfüllt**, wenn die Seite im Cache liegt.

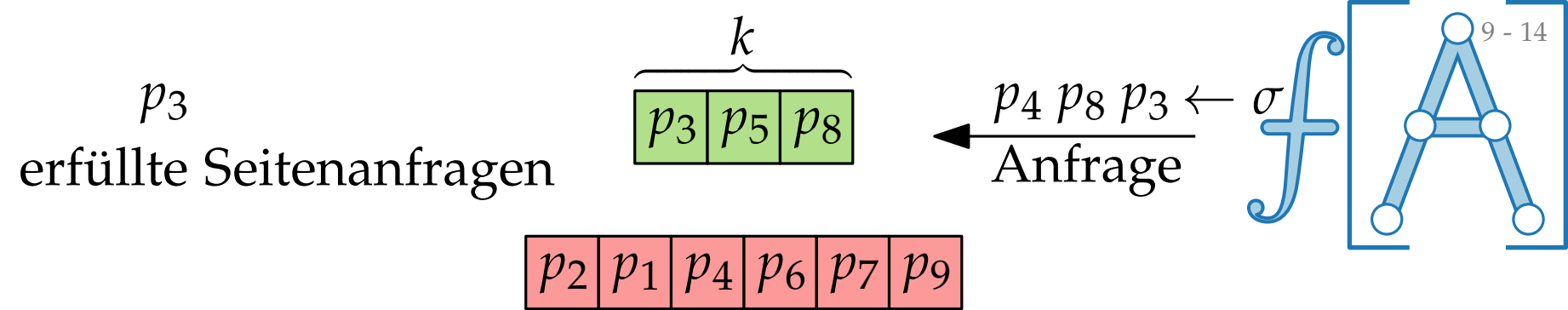
Problemstellung



Gegeben (offline/online):

- Speicher mit schnellem Zugriff (**Cache**) mit k Seiten Kapazität
- Speicher mit langsamem Zugriff (**Festplatte**) mit unbegrenzter Kapazität
- Wenn eine Seite angefragt wird, aber nicht im Cache liegt (**Seitenfehler**), muss sie mit einer Seite im Cache getauscht werden (**Seitentausch**).
Eine Seitenanfrage ist **erfüllt**, wenn die Seite im Cache liegt.
- Eine Sequenz σ von Seitenanfragen muss **in dieser Reihenfolge erfüllt werden**.
/ **Wir müssen eine Anfrage erfüllen, bevor wir die nächste sehen.**

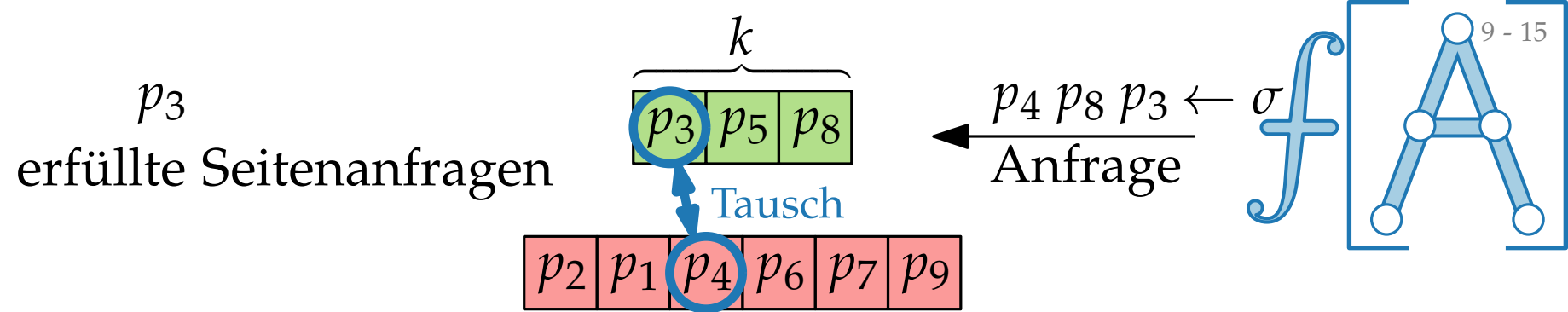
Problemstellung



Gegeben (**offline**/**online**):

- Speicher mit schnellem Zugriff (**Cache**) mit k Seiten Kapazität
- Speicher mit langsamem Zugriff (**Festplatte**) mit unbegrenzter Kapazität
- Wenn eine Seite angefragt wird, aber nicht im Cache liegt (**Seitenfehler**), muss sie mit einer Seite im Cache getauscht werden (**Seitentausch**).
Eine Seitenanfrage ist **erfüllt**, wenn die Seite im Cache liegt.
- Eine Sequenz σ von Seitenanfragen muss **in dieser Reihenfolge erfüllt werden**.
/ **Wir müssen eine Anfrage erfüllen, bevor wir die nächste sehen.**

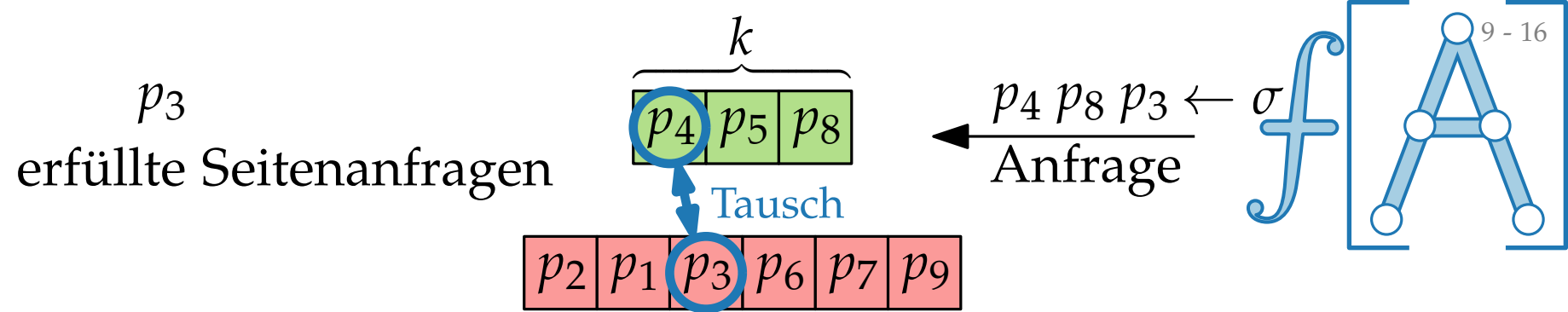
Problemstellung



Gegeben (**offline**/**online**):

- Speicher mit schnellem Zugriff (**Cache**) mit k Seiten Kapazität
- Speicher mit langsamem Zugriff (**Festplatte**) mit unbegrenzter Kapazität
- Wenn eine Seite angefragt wird, aber nicht im Cache liegt (**Seitenfehler**), muss sie mit einer Seite im Cache getauscht werden (**Seitentausch**).
Eine Seitenanfrage ist **erfüllt**, wenn die Seite im Cache liegt.
- Eine Sequenz σ von Seitenanfragen muss **in dieser Reihenfolge erfüllt werden**.
/ **Wir müssen eine Anfrage erfüllen, bevor wir die nächste sehen.**

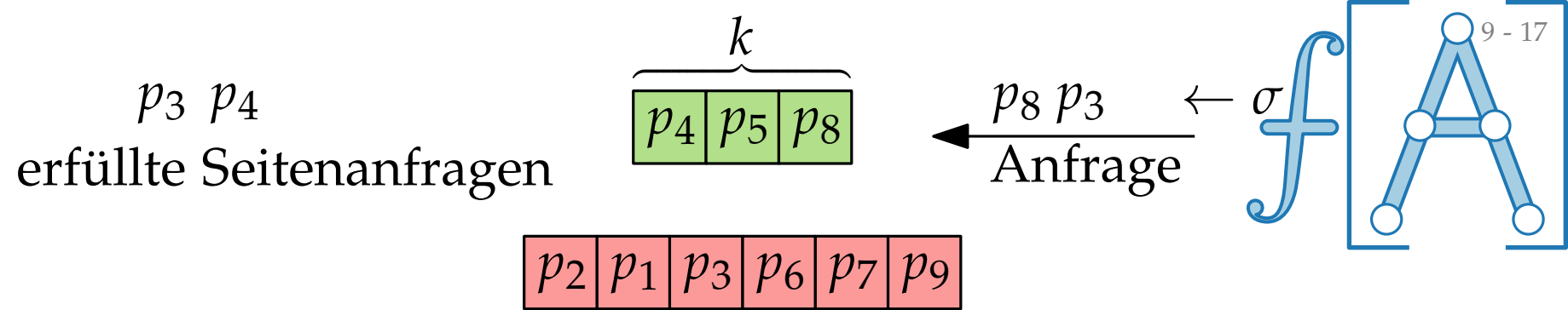
Problemstellung



Gegeben (**offline**/**online**):

- Speicher mit schnellem Zugriff (**Cache**) mit k Seiten Kapazität
- Speicher mit langsamem Zugriff (**Festplatte**) mit unbegrenzter Kapazität
- Wenn eine Seite angefragt wird, aber nicht im Cache liegt (**Seitenfehler**), muss sie mit einer Seite im Cache getauscht werden (**Seitentausch**).
Eine Seitenanfrage ist **erfüllt**, wenn die Seite im Cache liegt.
- Eine Sequenz σ von Seitenanfragen muss **in dieser Reihenfolge erfüllt werden**.
/ **Wir müssen eine Anfrage erfüllen, bevor wir die nächste sehen.**

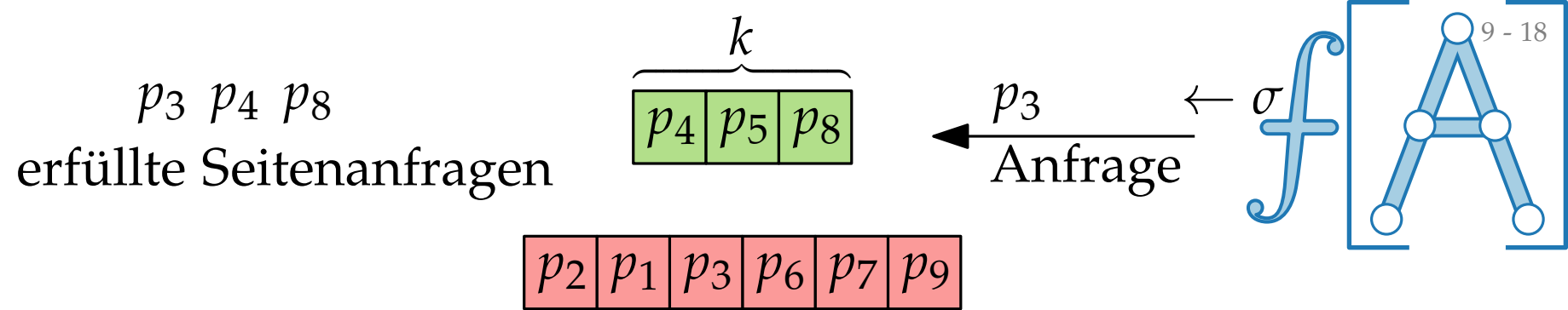
Problemstellung



Gegeben (**offline**/**online**):

- Speicher mit schnellem Zugriff (**Cache**) mit k Seiten Kapazität
- Speicher mit langsamem Zugriff (**Festplatte**) mit unbegrenzter Kapazität
- Wenn eine Seite angefragt wird, aber nicht im Cache liegt (**Seitenfehler**), muss sie mit einer Seite im Cache getauscht werden (**Seitentausch**).
Eine Seitenanfrage ist **erfüllt**, wenn die Seite im Cache liegt.
- Eine Sequenz σ von Seitenanfragen muss **in dieser Reihenfolge erfüllt werden**.
/ **Wir müssen eine Anfrage erfüllen, bevor wir die nächste sehen.**

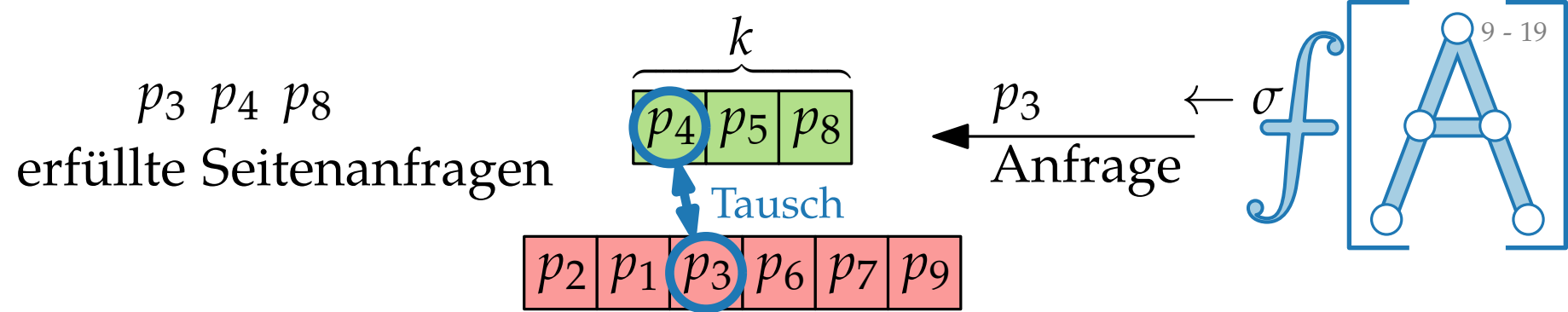
Problemstellung



Gegeben (**offline**/**online**):

- Speicher mit schnellem Zugriff (**Cache**) mit k Seiten Kapazität
- Speicher mit langsamem Zugriff (**Festplatte**) mit unbegrenzter Kapazität
- Wenn eine Seite angefragt wird, aber nicht im Cache liegt (**Seitenfehler**), muss sie mit einer Seite im Cache getauscht werden (**Seitentausch**).
Eine Seitenanfrage ist **erfüllt**, wenn die Seite im Cache liegt.
- Eine Sequenz σ von Seitenanfragen muss **in dieser Reihenfolge erfüllt werden**.
/ **Wir müssen eine Anfrage erfüllen, bevor wir die nächste sehen.**

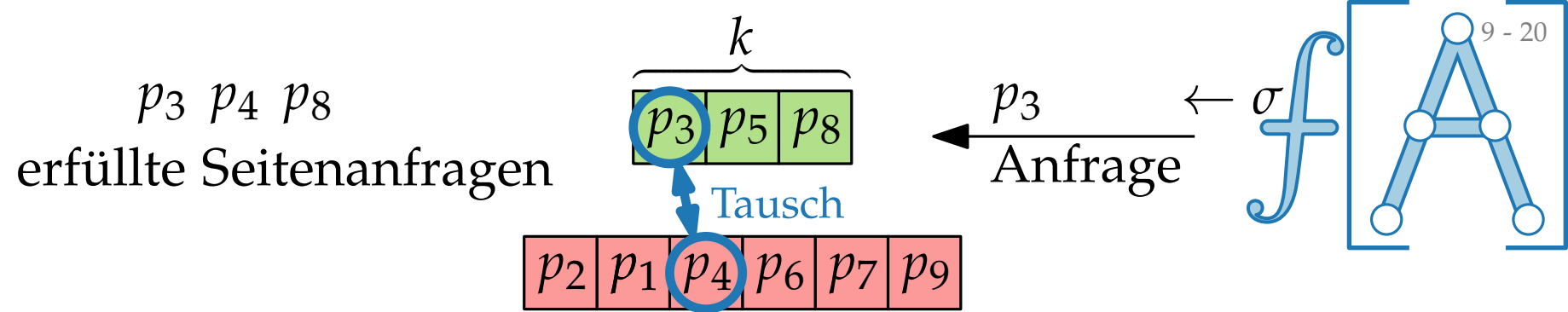
Problemstellung



Gegeben (**offline**/**online**):

- Speicher mit schnellem Zugriff (**Cache**) mit k Seiten Kapazität
- Speicher mit langsamem Zugriff (**Festplatte**) mit unbegrenzter Kapazität
- Wenn eine Seite angefragt wird, aber nicht im Cache liegt (**Seitenfehler**), muss sie mit einer Seite im Cache getauscht werden (**Seitentausch**).
Eine Seitenanfrage ist **erfüllt**, wenn die Seite im Cache liegt.
- Eine Sequenz σ von Seitenanfragen muss **in dieser Reihenfolge erfüllt werden**.
/ **Wir müssen eine Anfrage erfüllen, bevor wir die nächste sehen.**

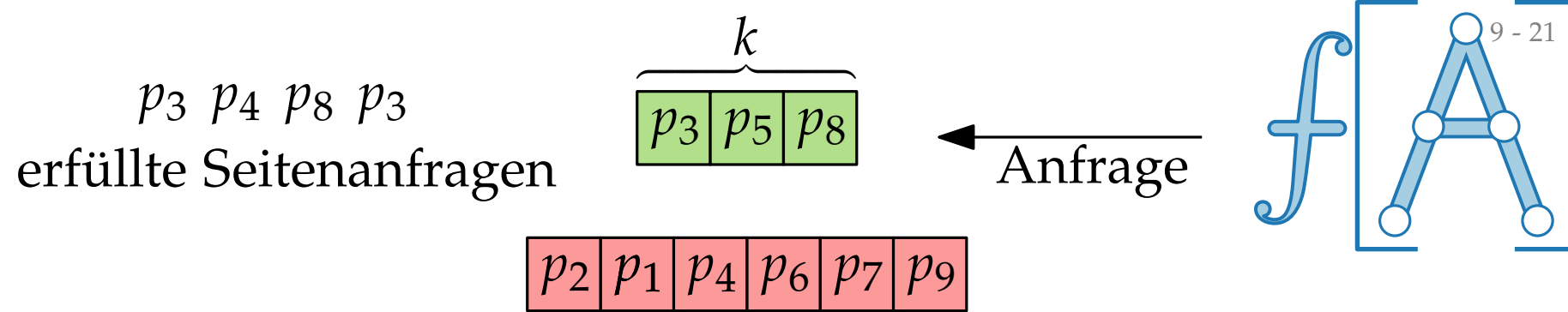
Problemstellung



Gegeben (**offline**/**online**):

- Speicher mit schnellem Zugriff (**Cache**) mit k Seiten Kapazität
- Speicher mit langsamem Zugriff (**Festplatte**) mit unbegrenzter Kapazität
- Wenn eine Seite angefragt wird, aber nicht im Cache liegt (**Seitenfehler**), muss sie mit einer Seite im Cache getauscht werden (**Seitentausch**).
Eine Seitenanfrage ist **erfüllt**, wenn die Seite im Cache liegt.
- Eine Sequenz σ von Seitenanfragen muss **in dieser Reihenfolge erfüllt werden**.
/ **Wir müssen eine Anfrage erfüllen, bevor wir die nächste sehen.**

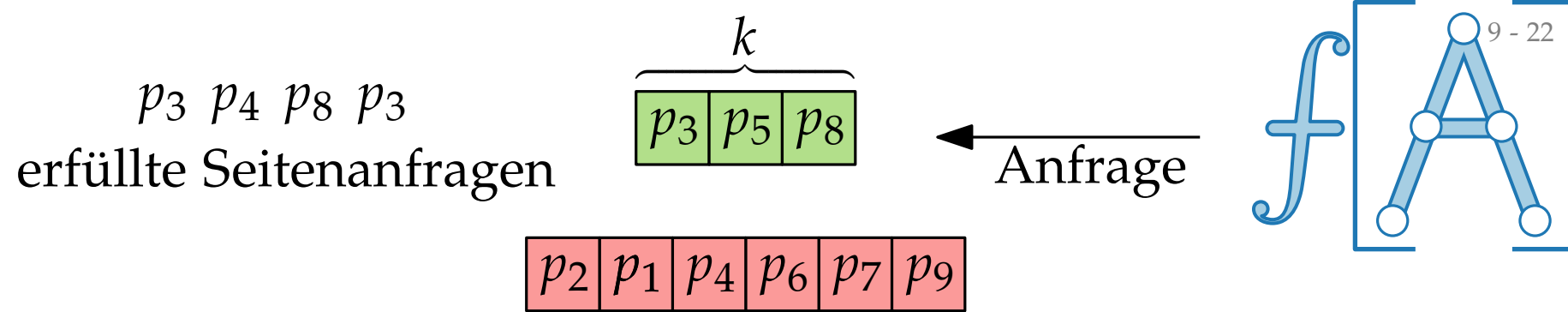
Problemstellung



Gegeben (**offline**/**online**):

- Speicher mit schnellem Zugriff (**Cache**) mit k Seiten Kapazität
- Speicher mit langsamem Zugriff (**Festplatte**) mit unbegrenzter Kapazität
- Wenn eine Seite angefragt wird, aber nicht im Cache liegt (**Seitenfehler**), muss sie mit einer Seite im Cache getauscht werden (**Seitentausch**).
Eine Seitenanfrage ist **erfüllt**, wenn die Seite im Cache liegt.
- Eine Sequenz σ von Seitenanfragen muss **in dieser Reihenfolge erfüllt werden**.
/ **Wir müssen eine Anfrage erfüllen, bevor wir die nächste sehen.**

Problemstellung

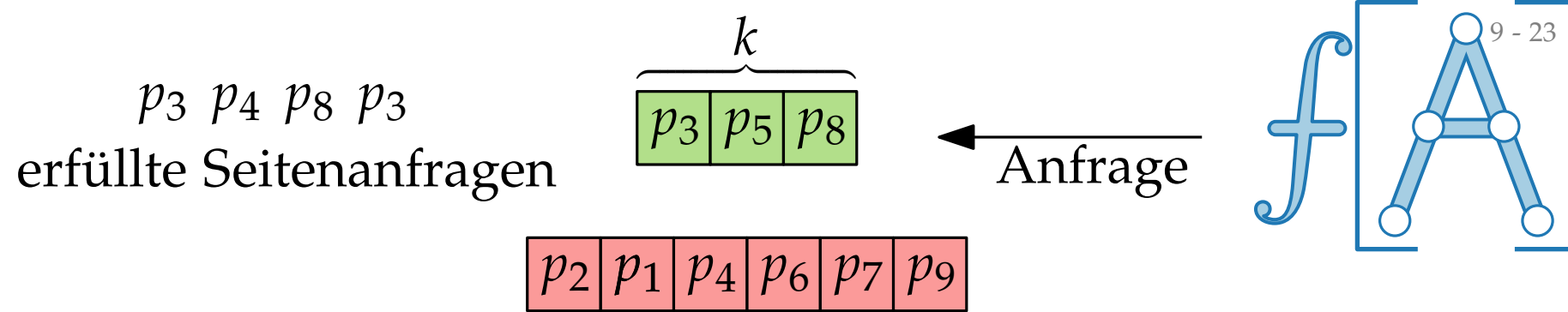


Gegeben (**offline**/**online**):

- Speicher mit schnellem Zugriff (**Cache**) mit k Seiten Kapazität
- Speicher mit langsamem Zugriff (**Festplatte**) mit unbegrenzter Kapazität
- Wenn eine Seite angefragt wird, aber nicht im Cache liegt (**Seitenfehler**), muss sie mit einer Seite im Cache getauscht werden (**Seitentausch**).
Eine Seitenanfrage ist **erfüllt**, wenn die Seite im Cache liegt.
- Eine Sequenz σ von Seitenanfragen muss **in dieser Reihenfolge erfüllt werden**.
/ **Wir müssen eine Anfrage erfüllen, bevor wir die nächste sehen.**

Zielfunktion.

Problemstellung

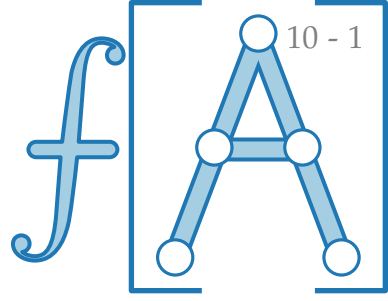


Gegeben (**offline**/**online**):

- Speicher mit schnellem Zugriff (**Cache**) mit k Seiten Kapazität
- Speicher mit langsamem Zugriff (**Festplatte**) mit unbegrenzter Kapazität
- Wenn eine Seite angefragt wird, aber nicht im Cache liegt (**Seitenfehler**), muss sie mit einer Seite im Cache getauscht werden (**Seitentausch**).
Eine Seitenanfrage ist **erfüllt**, wenn die Seite im Cache liegt.
- Eine Sequenz σ von Seitenanfragen muss **in dieser Reihenfolge erfüllt werden**.
/ **Wir müssen eine Anfrage erfüllen, bevor wir die nächste sehen.**

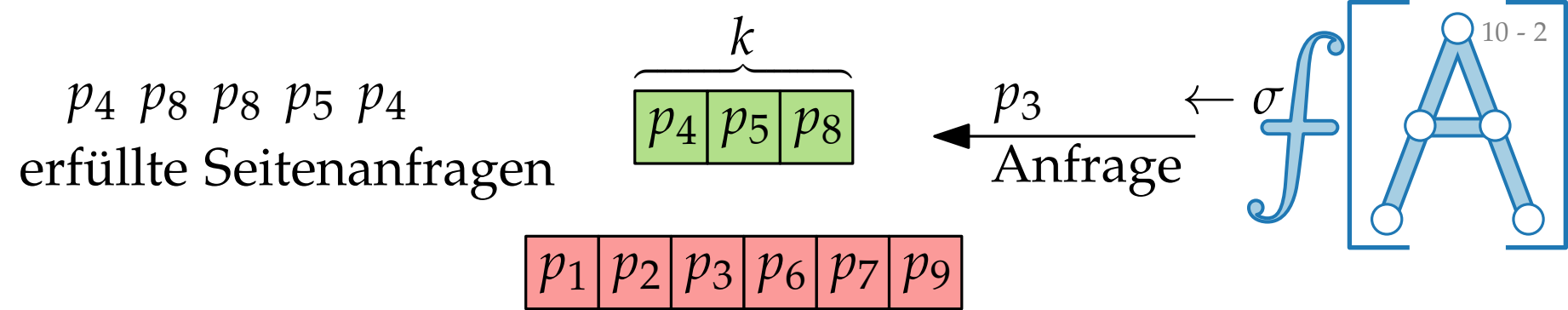
Zielfunktion. Minimiere die Anzahl der Seitenfehler während wir σ erfüllen.

Det. Strategie



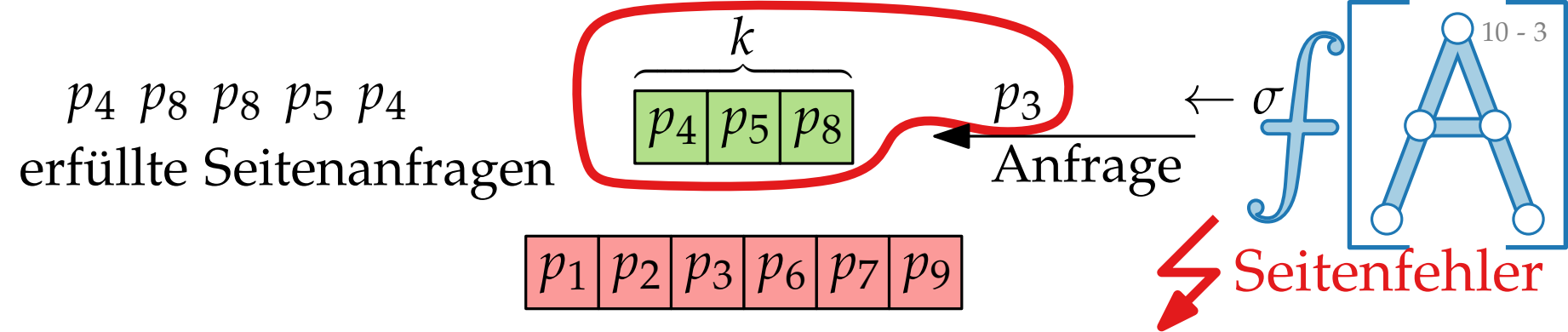
- Bei einem Seitenfehler wählt der PAGING Algorithmus aus, welche Seite aus dem Cache genommen wird.

Det. Strategie



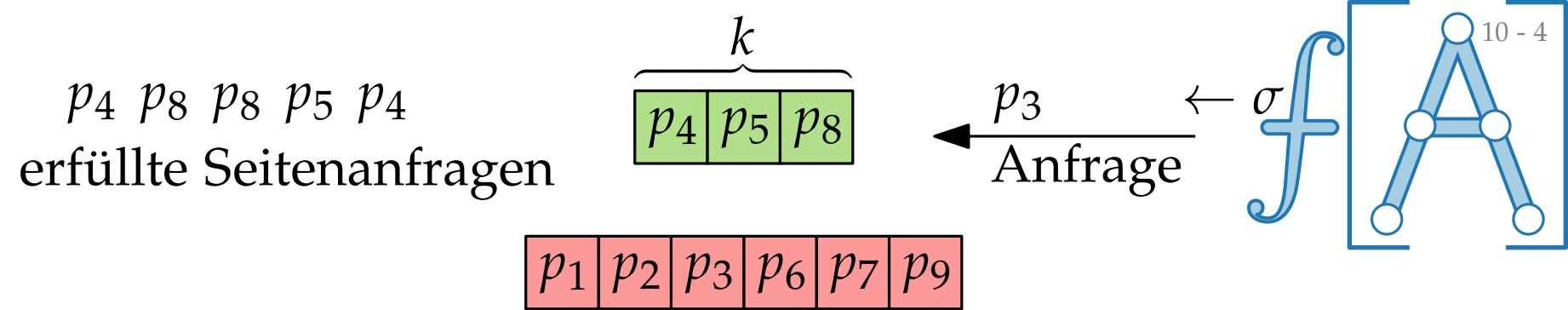
- Bei einem Seitenfehler wählt der PAGING Algorithmus aus, welche Seite aus dem Cache genommen wird.

Det. Strategie



- Bei einem Seitenfehler wählt der PAGING Algorithmus aus, welche Seite aus dem Cache genommen wird.

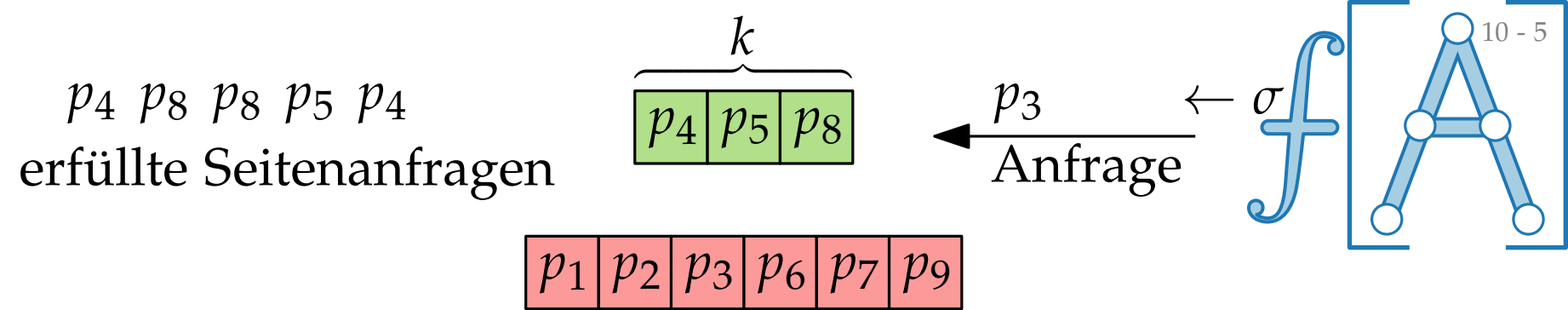
Det. Strategie



- Bei einem Seitenfehler wählt der PAGING Algorithmus aus, welche Seite aus dem Cache genommen wird.

Deterministische Strategien: Entnahme folgende Seite:

Det. Strategie

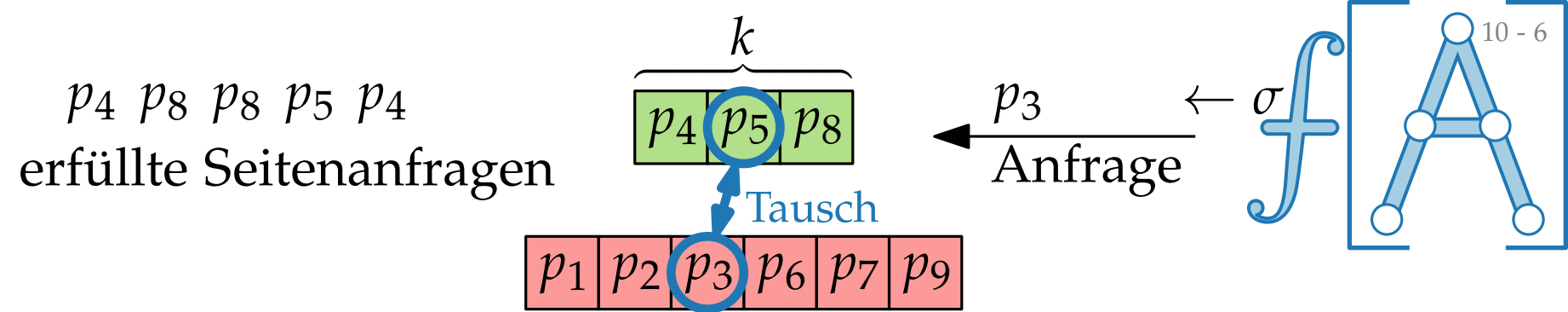


- Bei einem Seitenfehler wählt der PAGING Algorithmus aus, welche Seite aus dem Cache genommen wird.

Deterministische Strategien: Entnahme folgende Seite:

- Least Frequently Used (LFU): ... am wenigsten angefragt

Det. Strategie

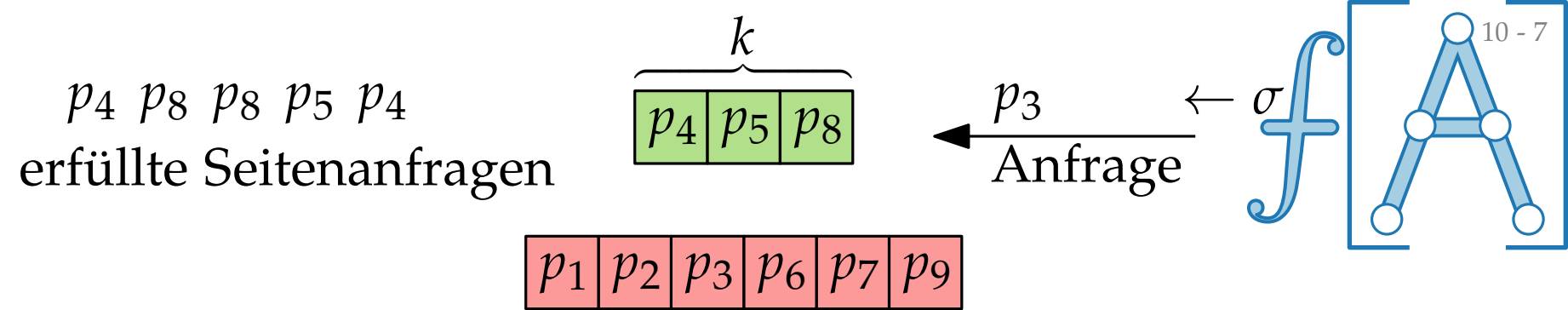


- Bei einem Seitenfehler wählt der PAGING Algorithmus aus, welche Seite aus dem Cache genommen wird.

Deterministische Strategien: Entnahme folgende Seite:

- Least Frequently Used (LFU): ... am wenigsten angefragt

Det. Strategie

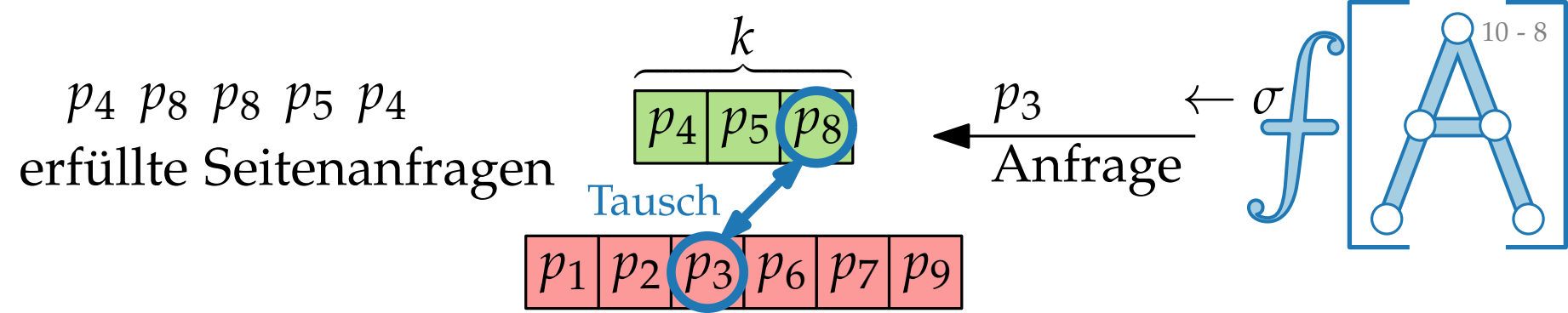


- Bei einem Seitenfehler wählt der PAGING Algorithmus aus, welche Seite aus dem Cache genommen wird.

Deterministische Strategien: Entnahme folgende Seite:

- Least Frequently Used (LFU): ... am wenigsten angefragt
- Least Recently Used (LRU): ... längste Zeit seit letzter Anfrage

Det. Strategie

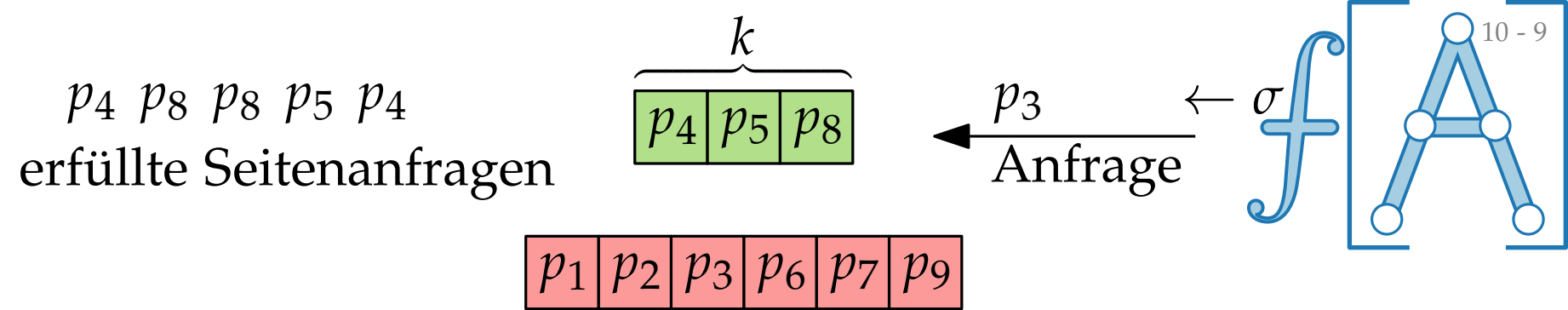


- Bei einem Seitenfehler wählt der PAGING Algorithmus aus, welche Seite aus dem Cache genommen wird.

Deterministische Strategien: Entnahme folgende Seite:

- Least Frequently Used (LFU): ... am wenigsten angefragt
- Least Recently Used (LRU): ... längste Zeit seit letzter Anfrage

Det. Strategie

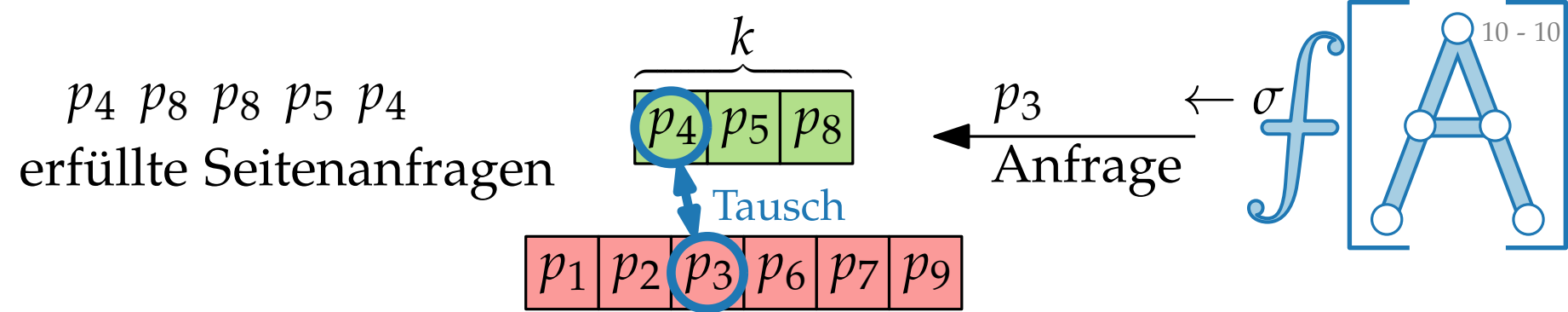


- Bei einem Seitenfehler wählt der PAGING Algorithmus aus, welche Seite aus dem Cache genommen wird.

Deterministische Strategien: Entnahme folgende Seite:

- Least Frequently Used (LFU): ... am wenigsten angefragt
- Least Recently Used (LRU): ... längste Zeit seit letzter Anfrage
- First-in-first-out (FIFO): ... am längsten im Cache.

Det. Strategie

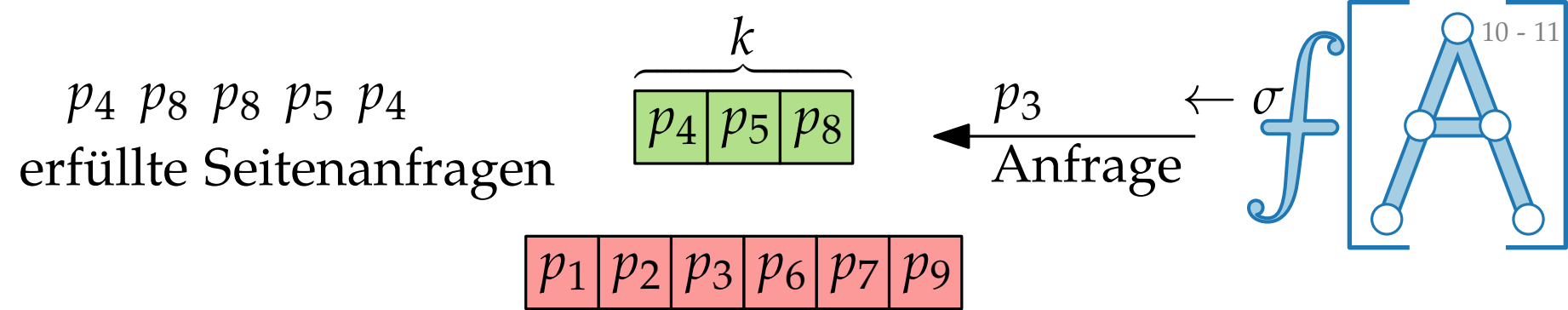


- Bei einem Seitenfehler wählt der PAGING Algorithmus aus, welche Seite aus dem Cache genommen wird.

Deterministische Strategien: Entnahme folgende Seite:

- Least Frequently Used (LFU): ... am wenigsten angefragt
- Least Recently Used (LRU): ... längste Zeit seit letzter Anfrage
- First-in-first-out (FIFO): ... am längsten im Cache.

Det. Strategie



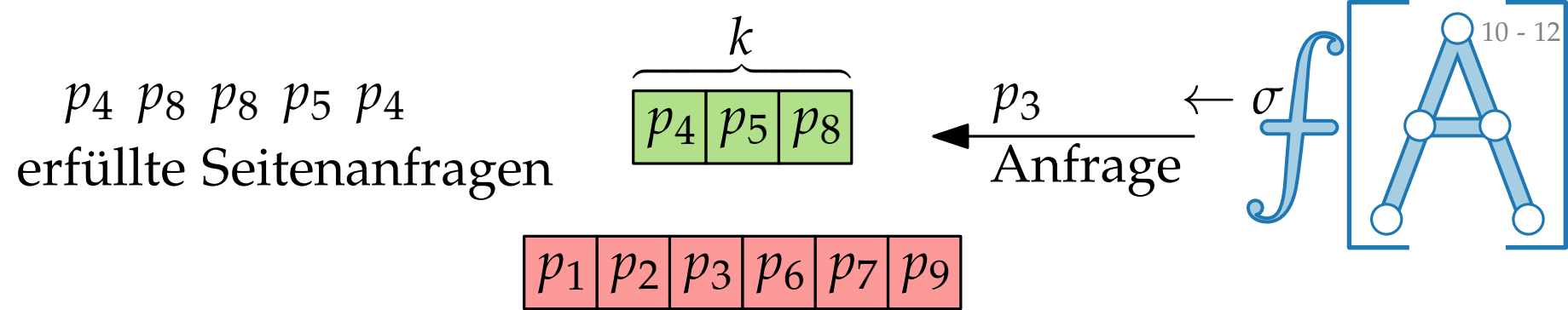
- Bei einem Seitenfehler wählt der PAGING Algorithmus aus, welche Seite aus dem Cache genommen wird.

Deterministische Strategien: Entnahme folgende Seite:

- Least Frequently Used (LFU): ... am wenigsten angefragt
- Least Recently Used (LRU): ... längste Zeit seit letzter Anfrage
- First-in-first-out (FIFO): ... am längsten im Cache.

Welche der Strategien ist die—theoretisch beweisbar—beste Strategie?

Det. Strategie



- Bei einem Seitenfehler wählt der PAGING Algorithmus aus, welche Seite aus dem Cache genommen wird.

Deterministische Strategien: Entnahme folgende Seite:

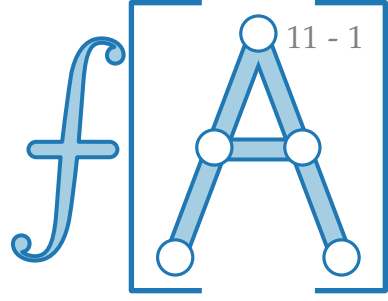
- Least Frequently Used (LFU): ... am wenigsten angefragt
- Least Recently Used (LRU): ... längste Zeit seit letzter Anfrage
- First-in-first-out (FIFO): ... am längsten im Cache.

Welche der Strategien ist die—theoretisch beweisbar—beste Strategie?

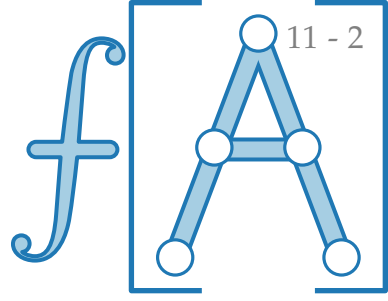
Satz. LRU & FIFO sind k -kompetitiv. Keine det. Strategie ist besser.

Analyse

Satz. LRU & FIFO sind k -kompetitiv. Keine det. Strategie ist besser.



Analyse



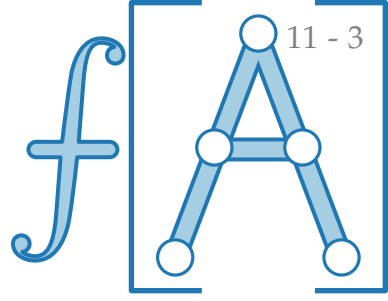
Satz. LRU & FIFO sind k -kompetitiv. Keine det. Strategie ist besser.

Beweis. (für LRU, FIFO ähnlich)

Least Recently Used (LRU): ... längste Zeit seit letzter Anfrage

Analyse

OPT: optimale Strategie
 σ : Anfragesequenz



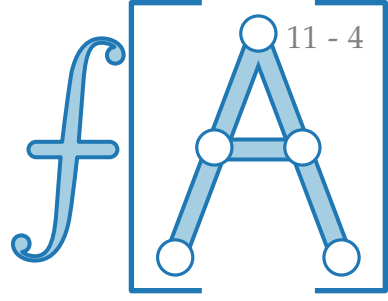
Satz. LRU & FIFO sind k -kompetitiv. Keine det. Strategie ist besser.

Beweis. (für LRU, FIFO ähnlich)

Least Recently Used (LRU): ... längste Zeit seit letzter Anfrage

Analyse

OPT: optimale Strategie
 σ : Anfragesequenz



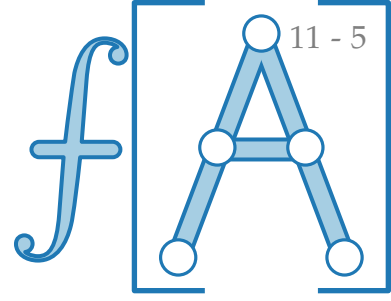
Satz. LRU & FIFO sind k -kompetitiv. Keine det. Strategie ist besser.

Beweis. (für LRU, FIFO ähnlich) Least Recently Used (LRU): ... längste Zeit seit letzter Anfrage

- Anfangs enthält der Cache bei jeder Strategie die gleichen Seiten.

Analyse

OPT: optimale Strategie
 σ : Anfragesequenz



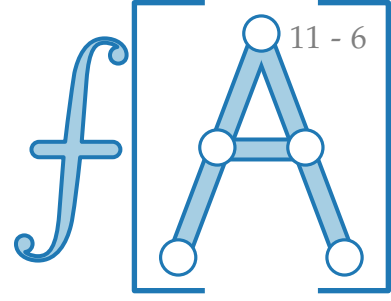
Satz. LRU & FIFO sind k -kompetitiv. Keine det. Strategie ist besser.

Beweis. (für LRU, FIFO ähnlich) Least Recently Used (LRU): ... längste Zeit seit letzter Anfrage

- Anfangs enthält der Cache bei jeder Strategie die gleichen Seiten.
- Wir partitionieren σ in Phasen $P_0, P_1, \dots$, so dass **LRU** höchstens k Seitenfehler in P_0 und **genau k Seitenfehler** in allen anderen Phasen hat.

Analyse

OPT: optimale Strategie
 σ : Anfragesequenz



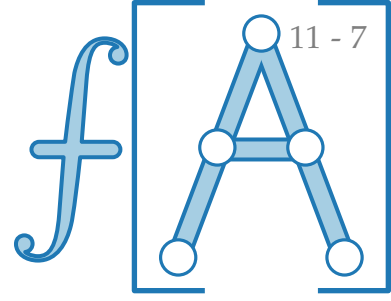
Satz. LRU & FIFO sind k -kompetitiv. Keine det. Strategie ist besser.

Beweis. (für LRU, FIFO ähnlich) Least Recently Used (LRU): ... längste Zeit seit letzter Anfrage

- Anfangs enthält der Cache bei jeder Strategie die gleichen Seiten.
- Wir partitionieren σ in Phasen $P_0, P_1, \dots$, so dass LRU höchstens k Seitenfehler in P_0 und genau k Seitenfehler in allen anderen Phasen hat.
- Wir zeigen: OPT hat mindestens 1 Seitenfehler in jeder Phase.

Analyse

OPT: optimale Strategie
 σ : Anfragesequenz



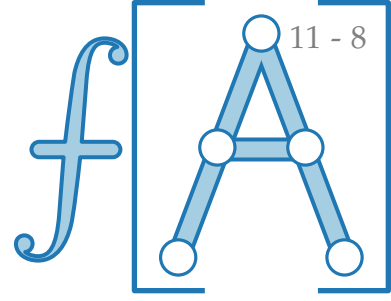
Satz. LRU & FIFO sind k -kompetitiv. Keine det. Strategie ist besser.

Beweis. (für LRU, FIFO ähnlich) Least Recently Used (LRU): ... längste Zeit seit letzter Anfrage

- Anfangs enthält der Cache bei jeder Strategie die gleichen Seiten.
- Wir partitionieren σ in Phasen $P_0, P_1, \dots$, so dass LRU höchstens k Seitenfehler in P_0 und genau k Seitenfehler in allen anderen Phasen hat.
- Wir zeigen: OPT hat mindestens 1 Seitenfehler in jeder Phase.
- Offensichtlich für P_0

Analyse

OPT: optimale Strategie
 σ : Anfragesequenz



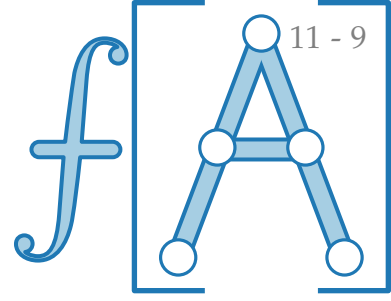
Satz. LRU & FIFO sind k -kompetitiv. Keine det. Strategie ist besser.

Beweis. (für LRU, FIFO ähnlich) Least Recently Used (LRU): ... längste Zeit seit letzter Anfrage

- Anfangs enthält der Cache bei jeder Strategie die gleichen Seiten.
- Wir partitionieren σ in Phasen $P_0, P_1, \dots$, so dass LRU höchstens k Seitenfehler in P_0 und genau k Seitenfehler in allen anderen Phasen hat.
- Wir zeigen: OPT hat mindestens 1 Seitenfehler in jeder Phase.
- Offensichtlich für P_0 ; betrachte $P_{i \geq 1}$ und sei p die letzte angefragte Seite von P_{i-1} .

Analyse

OPT: optimale Strategie
 σ : Anfragesequenz



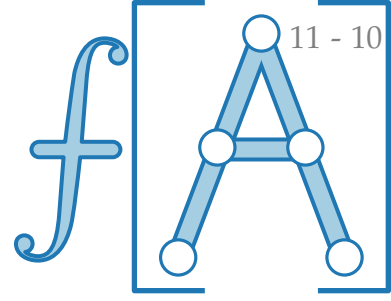
Satz. LRU & FIFO sind k -kompetitiv. Keine det. Strategie ist besser.

Beweis. (für LRU, FIFO ähnlich) Least Recently Used (LRU): ... längste Zeit seit letzter Anfrage

- Anfangs enthält der Cache bei jeder Strategie die gleichen Seiten.
- Wir partitionieren σ in Phasen $P_0, P_1, \dots$, so dass LRU höchstens k Seitenfehler in P_0 und genau k Seitenfehler in allen anderen Phasen hat.
- Wir zeigen: OPT hat mindestens 1 Seitenfehler in jeder Phase. $\Rightarrow p$ auf jeden Fall im Cache
- Offensichtlich für P_0 ; betrachte $P_{i \geq 1}$ und sei p die letzte angefragte Seite von P_{i-1} .

Analyse

OPT: optimale Strategie
 σ : Anfragesequenz



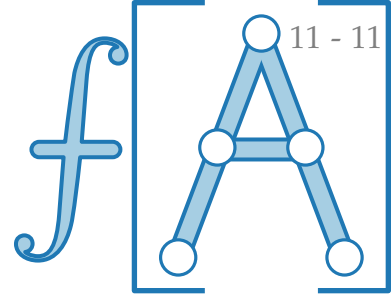
Satz. LRU & FIFO sind k -kompetitiv. Keine det. Strategie ist besser.

Beweis. (für LRU, FIFO ähnlich) Least Recently Used (LRU): ... längste Zeit seit letzter Anfrage

- Anfangs enthält der Cache bei jeder Strategie die gleichen Seiten.
- Wir partitionieren σ in Phasen $P_0, P_1, \dots$, so dass LRU höchstens k Seitenfehler in P_0 und genau k Seitenfehler in allen anderen Phasen hat.
- Wir zeigen: OPT hat mindestens 1 Seitenfehler in jeder Phase. $\Rightarrow p$ auf jeden Fall im Cache
- Offensichtlich für P_0 ; betrachte $P_{i \geq 1}$ und sei p die letzte angefragte Seite von P_{i-1} .
- Zeige: P_i enthält k Anfragen auf untersch. Seiten (außer p)

Analyse

OPT: optimale Strategie
 σ : Anfragesequenz



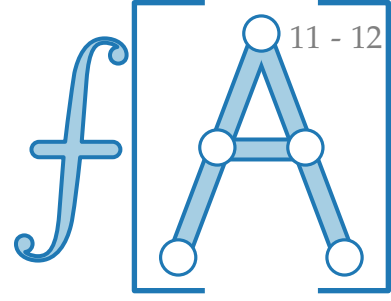
Satz. LRU & FIFO sind k -kompetitiv. Keine det. Strategie ist besser.

Beweis. (für LRU, FIFO ähnlich) Least Recently Used (LRU): ... längste Zeit seit letzter Anfrage

- Anfangs enthält der Cache bei jeder Strategie die gleichen Seiten.
- Wir partitionieren σ in Phasen $P_0, P_1, \dots$, so dass LRU höchstens k Seitenfehler in P_0 und genau k Seitenfehler in allen anderen Phasen hat.
- Wir zeigen: OPT hat mindestens 1 Seitenfehler in jeder Phase. $\Rightarrow p$ auf jeden Fall im Cache
- Offensichtlich für P_0 ; betrachte $P_{i \geq 1}$ und sei p die letzte angefragte Seite von P_{i-1} .
- Zeige: P_i enthält k Anfragen auf untersch. Seiten (außer p) $\Rightarrow$ Seitenfehler für OPT.

Analyse

OPT: optimale Strategie
 σ : Anfragesequenz



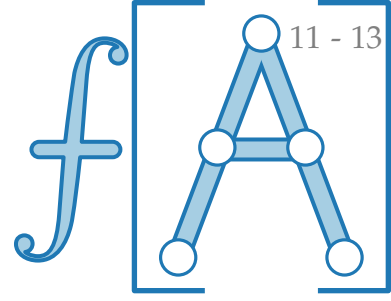
Satz. LRU & FIFO sind k -kompetitiv. Keine det. Strategie ist besser.

Beweis. (für LRU, FIFO ähnlich) Least Recently Used (LRU): ... längste Zeit seit letzter Anfrage

- Anfangs enthält der Cache bei jeder Strategie die gleichen Seiten.
- Wir partitionieren σ in Phasen $P_0, P_1, \dots$, so dass **LRU** höchstens k Seitenfehler in P_0 und **genau k Seitenfehler** in allen anderen Phasen hat.
- Wir zeigen: OPT hat mindestens 1 Seitenfehler in jeder Phase. $\Rightarrow p$ auf jeden Fall im Cache
- Offensichtlich für P_0 ; betrachte $P_{i \geq 1}$ und sei p die letzte angefragte Seite von P_{i-1} .
- Zeige: P_i enthält k Anfragen auf untersch. Seiten (außer p) $\Rightarrow$ Seitenfehler für OPT.
- Wenn die **k Fehler** von LRU in P_i auf untersch. Seiten sind (außer p), sind wir fertig.

Analyse

OPT: optimale Strategie
 σ : Anfragesequenz



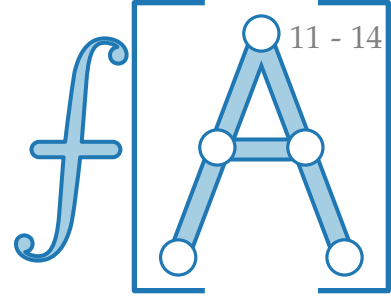
Satz. LRU & FIFO sind k -kompetitiv. Keine det. Strategie ist besser.

Beweis. (für LRU, FIFO ähnlich) Least Recently Used (LRU): ... längste Zeit seit letzter Anfrage

- Anfangs enthält der Cache bei jeder Strategie die gleichen Seiten.
- Wir partitionieren σ in Phasen $P_0, P_1, \dots$, so dass **LRU** höchstens k Seitenfehler in P_0 und **genau k Seitenfehler** in allen anderen Phasen hat.
- Wir zeigen: OPT hat mindestens 1 Seitenfehler in jeder Phase. $\Rightarrow p$ auf jeden Fall im Cache
- Offensichtlich für P_0 ; betrachte $P_{i \geq 1}$ und sei **p die letzte angefragte Seite von P_{i-1} .**
- Zeige: P_i enthält **k Anfragen auf untersch. Seiten (außer p)** $\Rightarrow$ Seitenfehler für OPT.
- Wenn die **k Fehler** von LRU in P_i auf untersch. Seiten sind (außer p), sind wir fertig.
- Angenommen, LRU hat in P_i zwei Fehler auf einer Seite q .

Analyse

OPT: optimale Strategie
 σ : Anfragesequenz



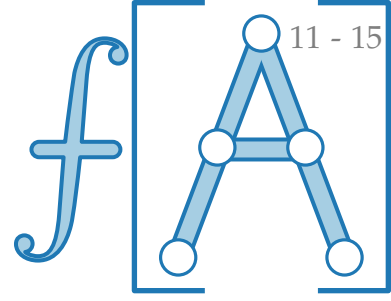
Satz. LRU & FIFO sind k -kompetitiv. Keine det. Strategie ist besser.

Beweis. (für LRU, FIFO ähnlich) Least Recently Used (LRU): ... längste Zeit seit letzter Anfrage

- Anfangs enthält der Cache bei jeder Strategie die gleichen Seiten.
- Wir partitionieren σ in Phasen $P_0, P_1, \dots$, so dass **LRU** höchstens k Seitenfehler in P_0 und **genau k Seitenfehler** in allen anderen Phasen hat.
- Wir zeigen: OPT hat mindestens 1 Seitenfehler in jeder Phase. $\Rightarrow p$ auf jeden Fall im Cache
- Offensichtlich für P_0 ; betrachte $P_{i \geq 1}$ und sei **p die letzte angefragte Seite von P_{i-1} .**
- Zeige: P_i enthält **k Anfragen auf untersch. Seiten (außer p)** $\Rightarrow$ Seitenfehler für OPT.
- Wenn die **k Fehler** von LRU in P_i auf untersch. Seiten sind (außer p), sind wir fertig.
- Angenommen, LRU hat in P_i zwei Fehler auf einer Seite q . Dazwischen muss q entnommen werden.

Analyse

OPT: optimale Strategie
 σ : Anfragesequenz



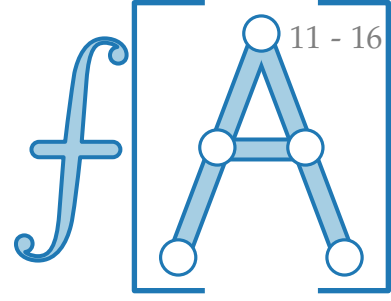
Satz. LRU & FIFO sind k -kompetitiv. Keine det. Strategie ist besser.

Beweis. (für LRU, FIFO ähnlich) Least Recently Used (LRU): ... längste Zeit seit letzter Anfrage

- Anfangs enthält der Cache bei jeder Strategie die gleichen Seiten.
- Wir partitionieren σ in Phasen $P_0, P_1, \dots$, so dass **LRU** höchstens k Seitenfehler in P_0 und **genau k Seitenfehler** in allen anderen Phasen hat.
- Wir zeigen: OPT hat mindestens 1 Seitenfehler in jeder Phase. $\Rightarrow p$ auf jeden Fall im Cache
- Offensichtlich für P_0 ; betrachte $P_{i \geq 1}$ und sei **p die letzte angefragte Seite von P_{i-1} .**
- Zeige: P_i enthält **k Anfragen auf untersch. Seiten (außer p)** $\Rightarrow$ Seitenfehler für OPT.
- Wenn die **k Fehler** von LRU in P_i auf untersch. Seiten sind (außer p), sind wir fertig.
- Angenommen, LRU hat in P_i zwei Fehler auf einer Seite q . Dazwischen muss q entnommen werden. Nach LRU gab es dazwischen k unterschiedliche Anfragen.

Analyse

OPT: optimale Strategie
 σ : Anfragesequenz



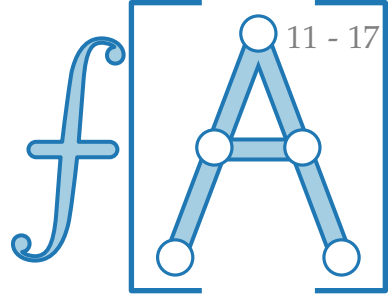
Satz. LRU & FIFO sind k -kompetitiv. Keine det. Strategie ist besser.

Beweis. (für LRU, FIFO ähnlich) Least Recently Used (LRU): ... längste Zeit seit letzter Anfrage

- Anfangs enthält der Cache bei jeder Strategie die gleichen Seiten.
- Wir partitionieren σ in Phasen $P_0, P_1, \dots$, so dass LRU höchstens k Seitenfehler in P_0 und genau k Seitenfehler in allen anderen Phasen hat.
- Wir zeigen: OPT hat mindestens 1 Seitenfehler in jeder Phase. $\Rightarrow p$ auf jeden Fall im Cache
- Offensichtlich für P_0 ; betrachte $P_{i \geq 1}$ und sei p die letzte angefragte Seite von P_{i-1} .
- Zeige: P_i enthält k Anfragen auf untersch. Seiten (außer p) $\Rightarrow$ Seitenfehler für OPT.
- Wenn die k Fehler von LRU in P_i auf untersch. Seiten sind (außer p), sind wir fertig.
- Angenommen, LRU hat in P_i zwei Fehler auf einer Seite q . Dazwischen muss q entnommen werden. Nach LRU gab es dazwischen k unterschiedliche Anfragen.
- Wenn LRU Fehler auf p in P_i hat, gab es dazwischen auch k untersch. Anfragen.

Analyse

OPT: optimale Strategie
 σ : Anfragesequenz



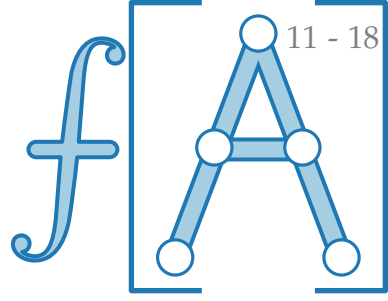
Satz. LRU & FIFO sind k -kompetitiv. Keine det. Strategie ist besser.

Beweis. (für LRU, FIFO ähnlich) Least Recently Used (LRU): ... längste Zeit seit letzter Anfrage

- Noch zu Zeigen: Keine deterministische Strategie ist besser als k -kompetitiv.

Analyse

OPT: optimale Strategie
 σ : Anfragesequenz



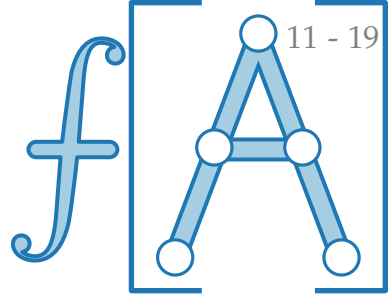
Satz. LRU & FIFO sind k -kompetitiv. Keine det. Strategie ist besser.

Beweis. (für LRU, FIFO ähnlich) Least Recently Used (LRU): ... längste Zeit seit letzter Anfrage

- Noch zu Zeigen: Keine deterministische Strategie ist besser als k -kompetitiv.
- Angenommen, wir haben insgesamt $k + 1$ Seiten im Speichersystem.

Analyse

OPT: optimale Strategie
 σ : Anfragesequenz



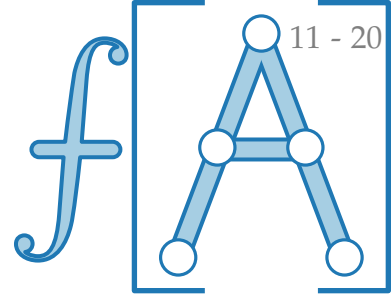
Satz. LRU & FIFO sind k -kompetitiv. Keine det. Strategie ist besser.

Beweis. (für LRU, FIFO ähnlich) Least Recently Used (LRU): ... längste Zeit seit letzter Anfrage

- Noch zu Zeigen: Keine deterministische Strategie ist besser als k -kompetitiv.
- Angenommen, wir haben insgesamt $k + 1$ Seiten im Speichersystem.
- Für jede deterministische Strategie gibt es eine schlechteste Anfragesequenz σ^* , die immer die Seite anfragt, die gerade nicht im Cache ist.

Analyse

OPT: optimale Strategie
 σ : Anfragesequenz



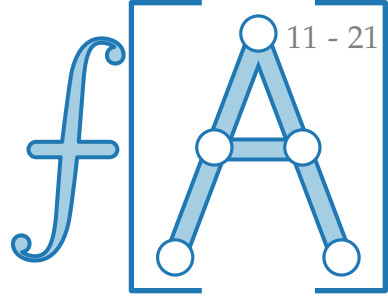
Satz. LRU & FIFO sind k -kompetitiv. Keine det. Strategie ist besser.

Beweis. (für LRU, FIFO ähnlich) Least Recently Used (LRU): ... längste Zeit seit letzter Anfrage

- Noch zu Zeigen: Keine deterministische Strategie ist besser als k -kompetitiv.
- Angenommen, wir haben insgesamt $k + 1$ Seiten im Speichersystem.
- Für jede deterministische Strategie gibt es eine schlechteste Anfragesequenz σ^* , die immer die Seite anfragt, die gerade nicht im Cache ist.
- Angenommen, OPT hat einen Seitenfehler bei der i -ten Anfrage von σ^* .

Analyse

OPT: optimale Strategie
 σ : Anfragesequenz



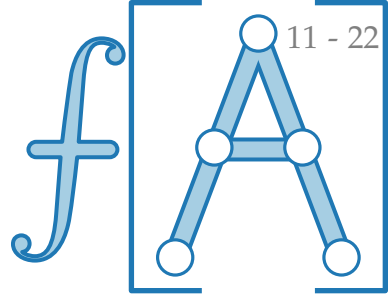
Satz. LRU & FIFO sind k -kompetitiv. Keine det. Strategie ist besser.

Beweis. (für LRU, FIFO ähnlich) Least Recently Used (LRU): ... längste Zeit seit letzter Anfrage

- Noch zu Zeigen: Keine deterministische Strategie ist besser als k -kompetitiv.
- Angenommen, wir haben insgesamt $k + 1$ Seiten im Speichersystem.
- Für jede deterministische Strategie gibt es eine schlechteste Anfragesequenz σ^* , die immer die Seite anfragt, die gerade nicht im Cache ist.
- Angenommen, OPT hat einen Seitenfehler bei der i -ten Anfrage von σ^* .
- Dann sind die nächsten $k - 1$ angefragten Seiten schon im Cache

Analyse

OPT: optimale Strategie
 σ : Anfragesequenz



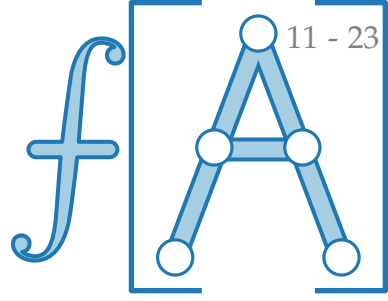
Satz. LRU & FIFO sind k -kompetitiv. Keine det. Strategie ist besser.

Beweis. (für LRU, FIFO ähnlich) Least Recently Used (LRU): ... längste Zeit seit letzter Anfrage

- Noch zu Zeigen: Keine deterministische Strategie ist besser als k -kompetitiv.
- Angenommen, wir haben insgesamt $k + 1$ Seiten im Speichersystem.
- Für jede deterministische Strategie gibt es eine schlechteste Anfragesequenz σ^* , die immer die Seite anfragt, die gerade nicht im Cache ist.
- Angenommen, OPT hat einen Seitenfehler bei der i -ten Anfrage von σ^* .
- Dann sind die nächsten $k - 1$ angefragten Seiten schon im Cache
 $\Rightarrow$ der nächste Fehler passiert frühestens bei der $(i + k)$ -ten Seite von σ^* .

Analyse

OPT: optimale Strategie
 σ : Anfragesequenz



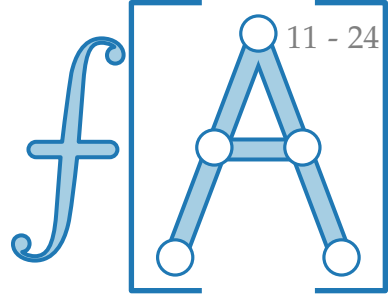
Satz. LRU & FIFO sind k -kompetitiv. Keine det. Strategie ist besser.

Beweis. (für LRU, FIFO ähnlich) Least Recently Used (LRU): ... längste Zeit seit letzter Anfrage

- Noch zu Zeigen: Keine deterministische Strategie ist besser als k -kompetitiv.
- Angenommen, wir haben insgesamt $k + 1$ Seiten im Speichersystem.
- Für jede deterministische Strategie gibt es eine schlechteste Anfragesequenz σ^* , die immer die Seite anfragt, die gerade nicht im Cache ist.
- Angenommen, OPT hat einen Seitenfehler bei der i -ten Anfrage von σ^* .
- Dann sind die nächsten $k - 1$ angefragten Seiten schon im Cache
 $\Rightarrow$ der nächste Fehler passiert frühestens bei der $(i + k)$ -ten Seite von σ^* .
Bis dahin hat die deterministische Strategie k Fehler.

Analyse

OPT: optimale Strategie
 σ : Anfragesequenz



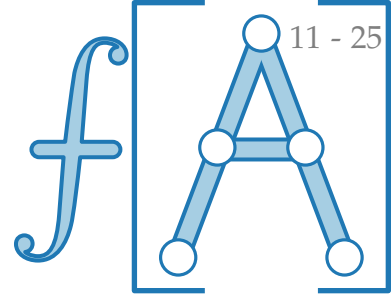
Satz. LRU & FIFO sind k -kompetitiv. Keine det. Strategie ist besser.

Beweis. (für LRU, FIFO ähnlich) Least Recently Used (LRU): ... längste Zeit seit letzter Anfrage

- Noch zu Zeigen: Keine deterministische Strategie ist besser als k -kompetitiv.
 - Angenommen, wir haben insgesamt $k + 1$ Seiten im Speichersystem.
 - Für jede deterministische Strategie gibt es eine schlechteste Anfragesequenz σ^* , die immer die Seite anfragt, die gerade nicht im Cache ist.
 - Angenommen, OPT hat einen Seitenfehler bei der i -ten Anfrage von σ^* .
 - Dann sind die nächsten $k - 1$ angefragten Seiten schon im Cache
 $\Rightarrow$ der nächste Fehler passiert frühestens bei der $(i + k)$ -ten Seite von σ^* .
Bis dahin hat die deterministische Strategie k Fehler.
- $\Rightarrow$ Das kompetitive Verhältnis kann nicht besser sein als

Analyse

OPT: optimale Strategie
 σ : Anfragesequenz



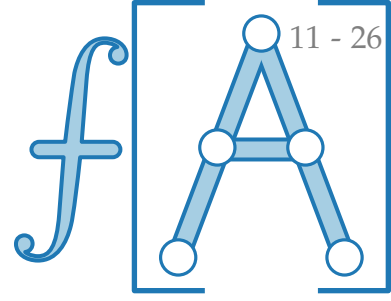
Satz. LRU & FIFO sind k -kompetitiv. Keine det. Strategie ist besser.

Beweis. (für LRU, FIFO ähnlich) Least Recently Used (LRU): ... längste Zeit seit letzter Anfrage

- Noch zu Zeigen: Keine deterministische Strategie ist besser als k -kompetitiv.
 - Angenommen, wir haben insgesamt $k + 1$ Seiten im Speichersystem.
 - Für jede deterministische Strategie gibt es eine schlechteste Anfragesequenz σ^* , die immer die Seite anfragt, die gerade nicht im Cache ist.
 - Angenommen, OPT hat einen Seitenfehler bei der i -ten Anfrage von σ^* .
 - Dann sind die nächsten $k - 1$ angefragten Seiten schon im Cache
 $\Rightarrow$ der nächste Fehler passiert frühestens bei der $(i + k)$ -ten Seite von σ^* .
Bis dahin hat die deterministische Strategie k Fehler.
- $\Rightarrow$ Das kompetitive Verhältnis kann nicht besser sein als $\frac{|\sigma^*|}{k}$

Analyse

OPT: optimale Strategie
 σ : Anfragesequenz



Satz. LRU & FIFO sind k -kompetitiv. Keine det. Strategie ist besser.

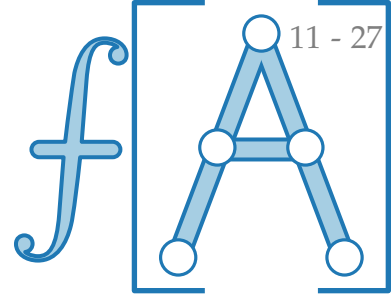
Beweis. (für LRU, FIFO ähnlich) Least Recently Used (LRU): ... längste Zeit seit letzter Anfrage

- Noch zu Zeigen: Keine deterministische Strategie ist besser als k -kompetitiv.
- Angenommen, wir haben insgesamt $k + 1$ Seiten im Speichersystem.
- Für jede deterministische Strategie gibt es eine schlechteste Anfragesequenz σ^* , die immer die Seite anfragt, die gerade nicht im Cache ist.
- Angenommen, OPT hat einen Seitenfehler bei der i -ten Anfrage von σ^* .
- Dann sind die nächsten $k - 1$ angefragten Seiten schon im Cache
 $\Rightarrow$ der nächste Fehler passiert frühestens bei der $(i + k)$ -ten Seite von σ^* .
Bis dahin hat die deterministische Strategie k Fehler.

$\Rightarrow$ Das kompetitive Verhältnis kann nicht besser sein als $\left\lceil \frac{|\sigma^*|}{k} \right\rceil$

Analyse

OPT: optimale Strategie
 σ : Anfragesequenz



Satz. LRU & FIFO sind k -kompetitiv. Keine det. Strategie ist besser.

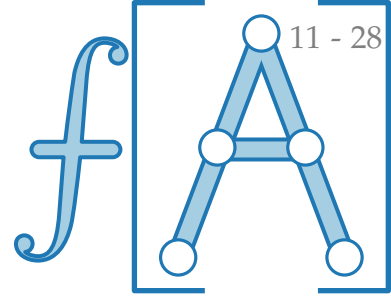
Beweis. (für LRU, FIFO ähnlich) Least Recently Used (LRU): ... längste Zeit seit letzter Anfrage

- Noch zu Zeigen: Keine deterministische Strategie ist besser als k -kompetitiv.
- Angenommen, wir haben insgesamt $k + 1$ Seiten im Speichersystem.
- Für jede deterministische Strategie gibt es eine schlechteste Anfragesequenz σ^* , die immer die Seite anfragt, die gerade nicht im Cache ist.
- Angenommen, OPT hat einen Seitenfehler bei der i -ten Anfrage von σ^* .
- Dann sind die nächsten $k - 1$ angefragten Seiten schon im Cache
 $\Rightarrow$ der nächste Fehler passiert frühestens bei der $(i + k)$ -ten Seite von σ^* .
Bis dahin hat die deterministische Strategie k Fehler.

$\Rightarrow$ Das kompetitive Verhältnis kann nicht besser sein als $\frac{|\sigma^*|}{\left\lceil \frac{|\sigma^*|}{k} \right\rceil} \stackrel{|\sigma^*| \rightsquigarrow \infty}{=} k$.

Analyse

OPT: optimale Strategie
 σ : Anfragesequenz



Satz. LRU & FIFO sind k -kompetitiv. Keine det. Strategie ist besser.

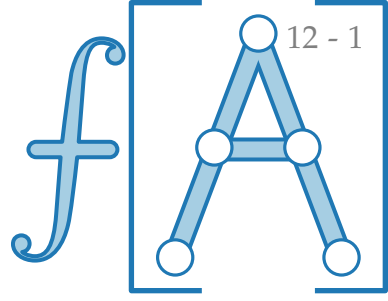
Beweis. (für LRU, FIFO ähnlich) Least Recently Used (LRU): ... längste Zeit seit letzter Anfrage

- Noch zu Zeigen: Keine deterministische Strategie ist besser als k -kompetitiv.
- Angenommen, wir haben insgesamt $k + 1$ Seiten im Speichersystem.
- Für jede deterministische Strategie gibt es eine schlechteste Anfragesequenz σ^* , die immer die Seite anfragt, die gerade nicht im Cache ist.
- Angenommen, OPT hat einen Seitenfehler bei der i -ten Anfrage von σ^* .
- Dann sind die nächsten $k - 1$ angefragten Seiten schon im Cache
 $\Rightarrow$ der nächste Fehler passiert frühestens bei der $(i + k)$ -ten Seite von σ^* .
Bis dahin hat die deterministische Strategie k Fehler.

$\Rightarrow$ Das kompetitive Verhältnis kann nicht besser sein als $\frac{|\sigma^*|}{\left\lceil \frac{|\sigma^*|}{k} \right\rceil} \stackrel{|\sigma^*| \rightarrow \infty}{=} k.$

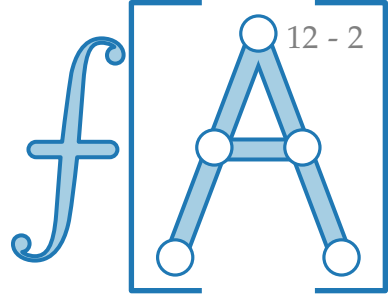


Randomisierte Strategie



Randomisierte Strategie: MARKING

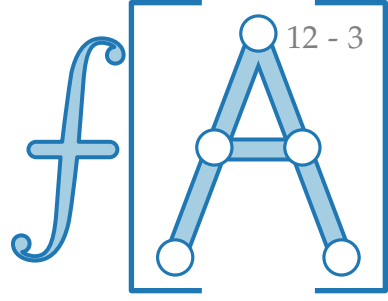
Randomisierte Strategie



Randomisierte Strategie: MARKING

- Geht in Phasen vor

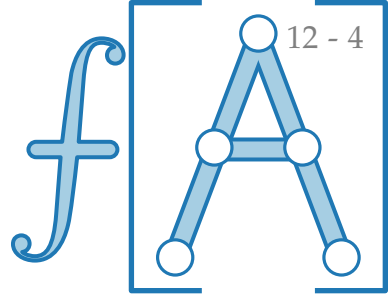
Randomisierte Strategie



Randomisierte Strategie: MARKING

- Geht in Phasen vor
- Am Anfang jeder Phase sind alle Seiten unmarkiert.

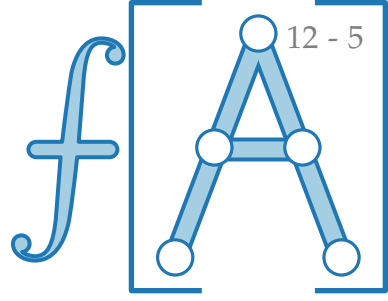
Randomisierte Strategie



Randomisierte Strategie: MARKING

- Geht in Phasen vor
- Am Anfang jeder Phase sind alle Seiten unmarkiert.
- Wenn eine Seite angefragt wird, wird sie **markiert**.

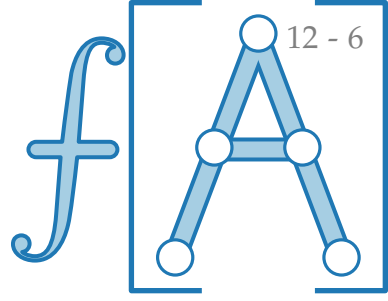
Randomisierte Strategie



Randomisierte Strategie: MARKING

- Geht in Phasen vor
- Am Anfang jeder Phase sind alle Seiten unmarkiert.
- Wenn eine Seite angefragt wird, wird sie **markiert**.
- Unter den unmarkierten Seiten wird eine **zufällig gleichverteilt** entnommen.

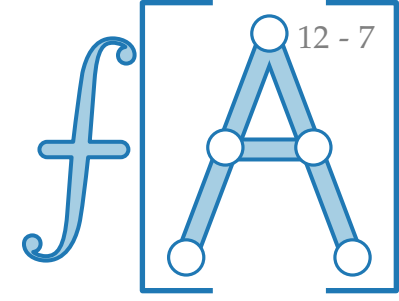
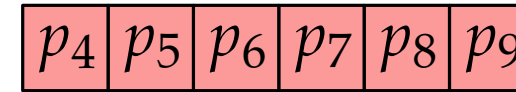
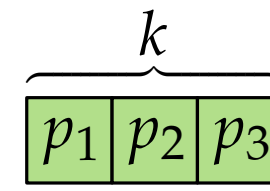
Randomisierte Strategie



Randomisierte Strategie: MARKING

- Geht in Phasen vor
- Am Anfang jeder Phase sind alle Seiten unmarkiert.
- Wenn eine Seite angefragt wird, wird sie **markiert**.
- Unter den unmarkierten Seiten wird eine **zufällig gleichverteilt** entnommen.
- Wenn alle Seiten markiert sind und ein Seitenfehler auftritt, entferne Markierungen und starte neue Phase.

Randomisierte Strategie

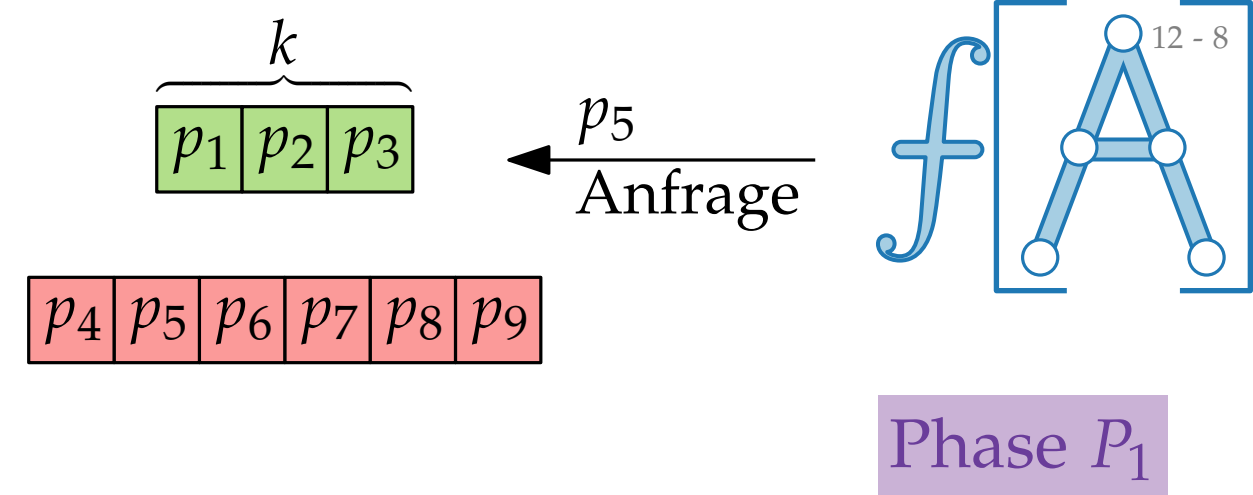


Phase P_1

Randomisierte Strategie: MARKING

- Geht in Phasen vor
- Am Anfang jeder Phase sind alle Seiten unmarkiert.
- Wenn eine Seite angefragt wird, wird sie **markiert**.
- Unter den unmarkierten Seiten wird eine **zufällig gleichverteilt** entnommen.
- Wenn alle Seiten markiert sind und ein Seitenfehler auftritt, entferne Markierungen und starte neue Phase.

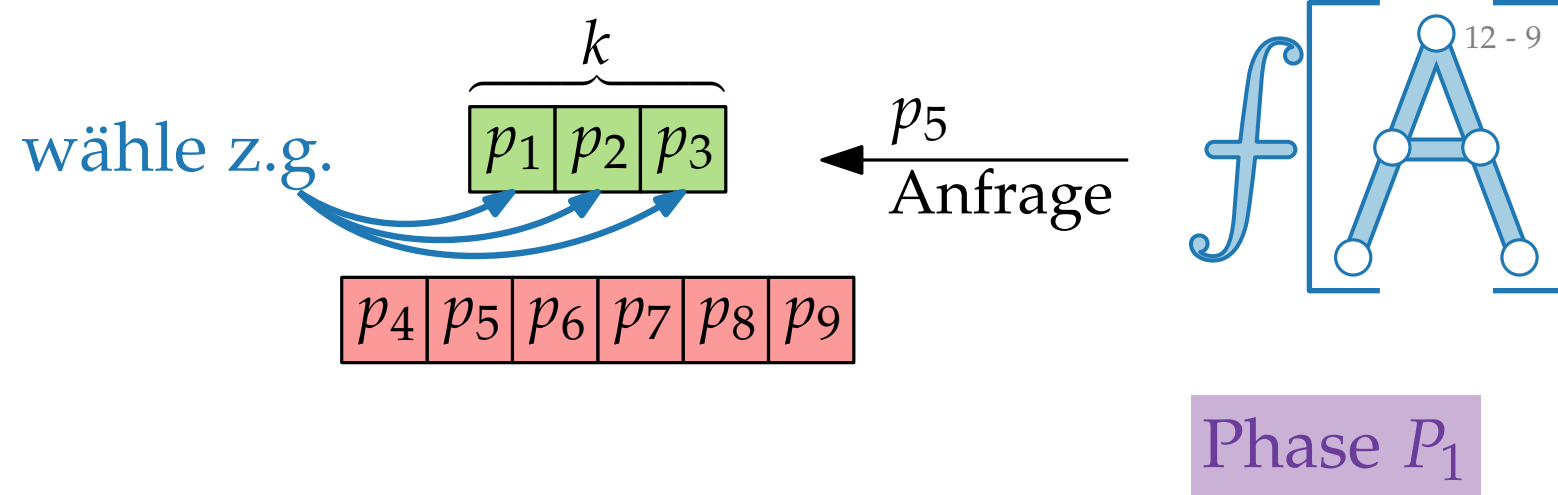
Randomisierte Strategie



Randomisierte Strategie: MARKING

- Geht in Phasen vor
- Am Anfang jeder Phase sind alle Seiten unmarkiert.
- Wenn eine Seite angefragt wird, wird sie **markiert**.
- Unter den unmarkierten Seiten wird eine **zufällig gleichverteilt** entnommen.
- Wenn alle Seiten markiert sind und ein Seitenfehler auftritt, entferne Markierungen und starte neue Phase.

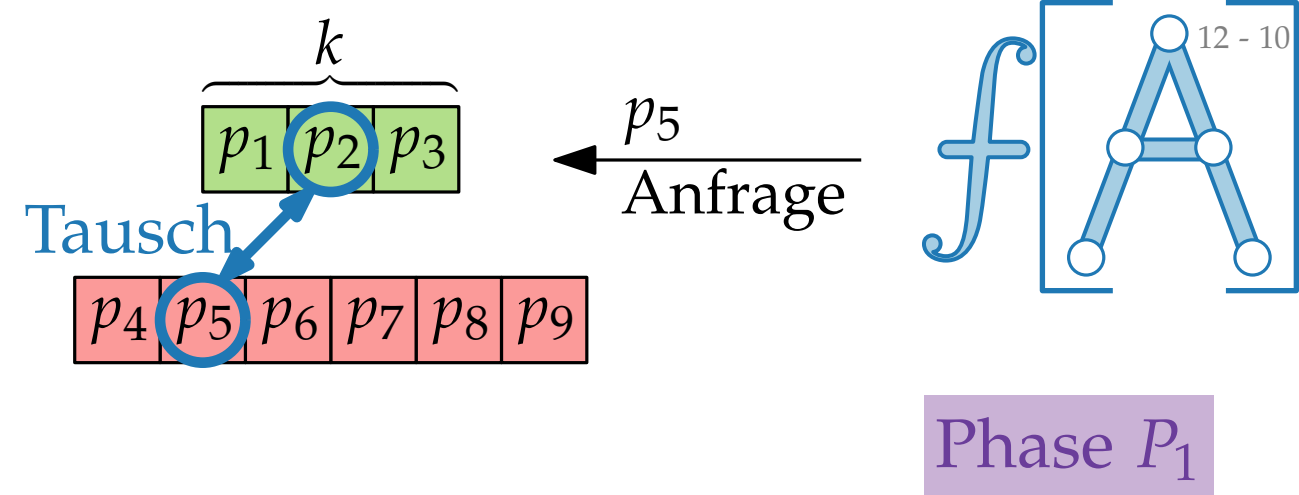
Randomisierte Strategie



Randomisierte Strategie: MARKING

- Geht in Phasen vor
- Am Anfang jeder Phase sind alle Seiten unmarkiert.
- Wenn eine Seite angefragt wird, wird sie **markiert**.
- Unter den unmarkierten Seiten wird eine **zufällig gleichverteilt** entnommen.
- Wenn alle Seiten markiert sind und ein Seitenfehler auftritt, entferne Markierungen und starte neue Phase.

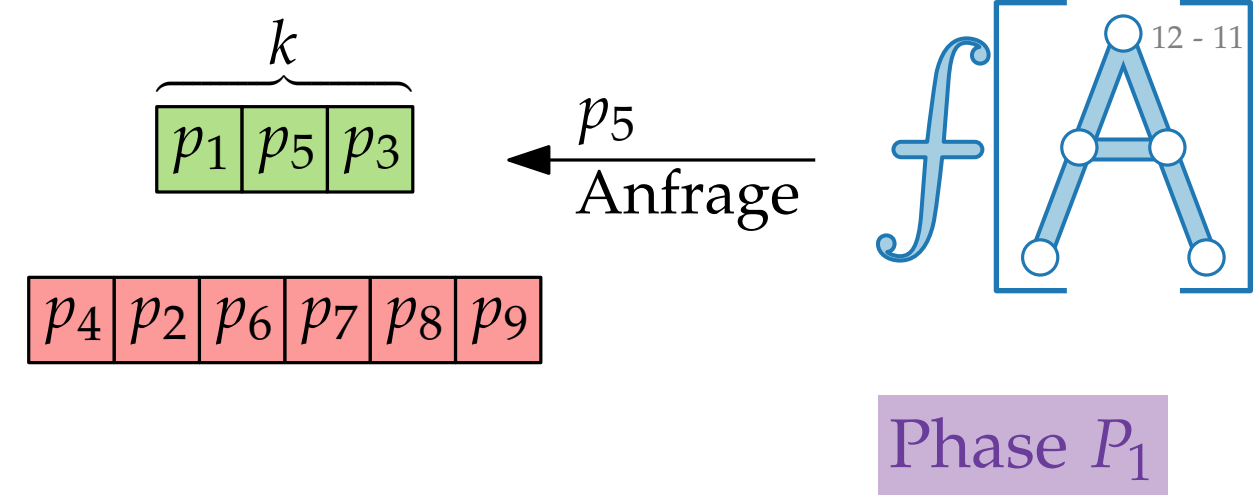
Randomisierte Strategie



Randomisierte Strategie: MARKING

- Geht in Phasen vor
- Am Anfang jeder Phase sind alle Seiten unmarkiert.
- Wenn eine Seite angefragt wird, wird sie **markiert**.
- Unter den unmarkierten Seiten wird eine **zufällig gleichverteilt** entnommen.
- Wenn alle Seiten markiert sind und ein Seitenfehler auftritt, entferne Markierungen und starte neue Phase.

Randomisierte Strategie

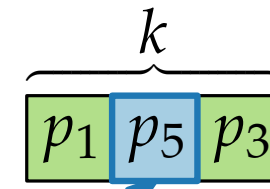


Randomisierte Strategie: MARKING

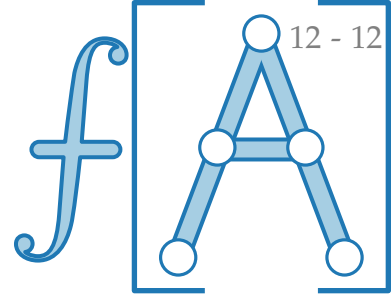
- Geht in Phasen vor
- Am Anfang jeder Phase sind alle Seiten unmarkiert.
- Wenn eine Seite angefragt wird, wird sie **markiert**.
- Unter den unmarkierten Seiten wird eine **zufällig gleichverteilt** entnommen.
- Wenn alle Seiten markiert sind und ein Seitenfehler auftritt, entferne Markierungen und starte neue Phase.

Randomisierte Strategie

markiere Seite



p_5
Anfrage

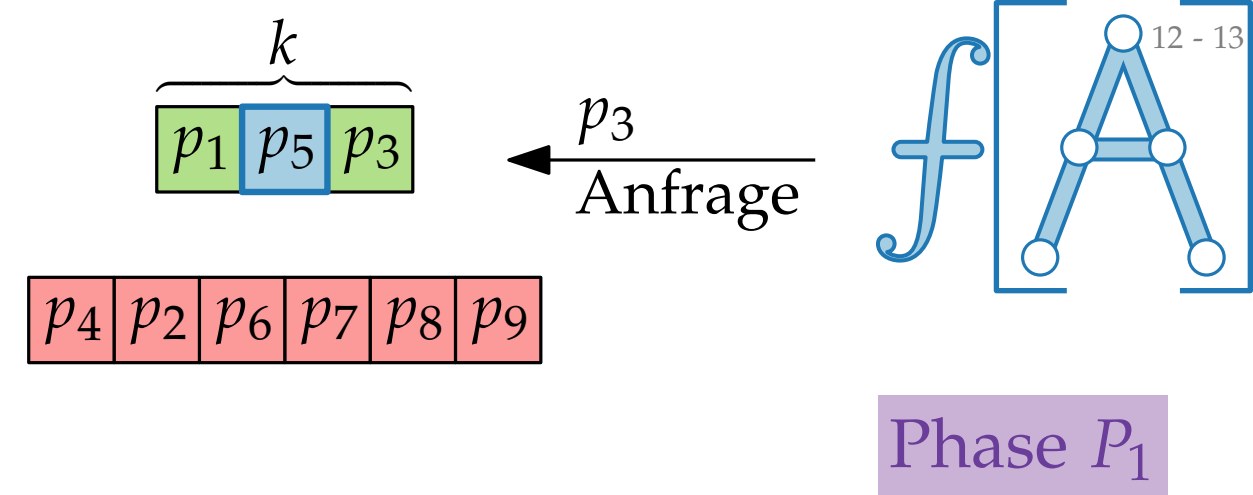


Phase P_1

Randomisierte Strategie: MARKING

- Geht in Phasen vor
- Am Anfang jeder Phase sind alle Seiten unmarkiert.
- Wenn eine Seite angefragt wird, wird sie **markiert**.
- Unter den unmarkierten Seiten wird eine **zufällig gleichverteilt** entnommen.
- Wenn alle Seiten markiert sind und ein Seitenfehler auftritt, entferne Markierungen und starte neue Phase.

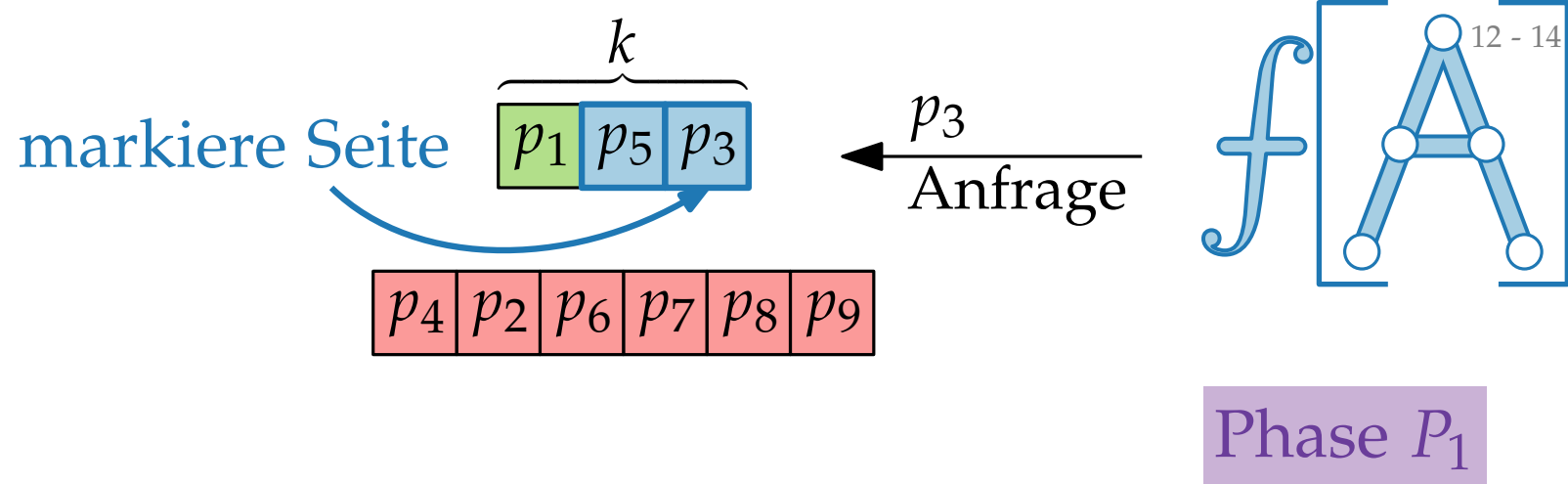
Randomisierte Strategie



Randomisierte Strategie: MARKING

- Geht in Phasen vor
- Am Anfang jeder Phase sind alle Seiten unmarkiert.
- Wenn eine Seite angefragt wird, wird sie **markiert**.
- Unter den unmarkierten Seiten wird eine **zufällig gleichverteilt** entnommen.
- Wenn alle Seiten markiert sind und ein Seitenfehler auftritt, entferne Markierungen und starte neue Phase.

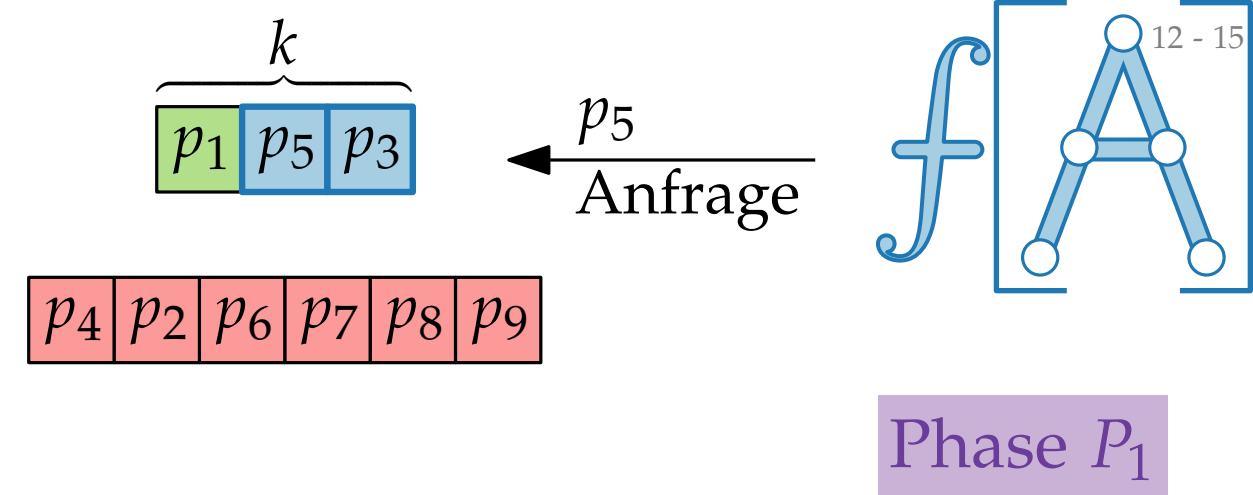
Randomisierte Strategie



Randomisierte Strategie: MARKING

- Geht in Phasen vor
- Am Anfang jeder Phase sind alle Seiten unmarkiert.
- Wenn eine Seite angefragt wird, wird sie **markiert**.
- Unter den unmarkierten Seiten wird eine **zufällig gleichverteilt** entnommen.
- Wenn alle Seiten markiert sind und ein Seitenfehler auftritt, entferne Markierungen und starte neue Phase.

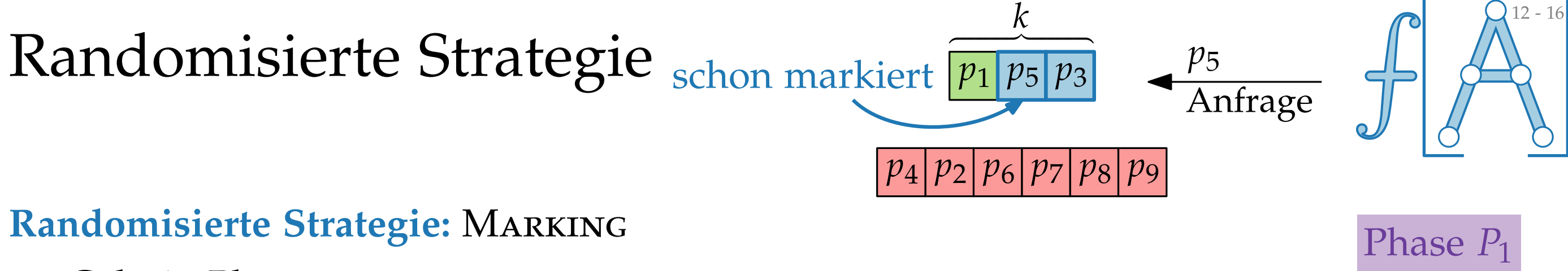
Randomisierte Strategie



Randomisierte Strategie: MARKING

- Geht in Phasen vor
- Am Anfang jeder Phase sind alle Seiten unmarkiert.
- Wenn eine Seite angefragt wird, wird sie **markiert**.
- Unter den unmarkierten Seiten wird eine **zufällig gleichverteilt** entnommen.
- Wenn alle Seiten markiert sind und ein Seitenfehler auftritt, entferne Markierungen und starte neue Phase.

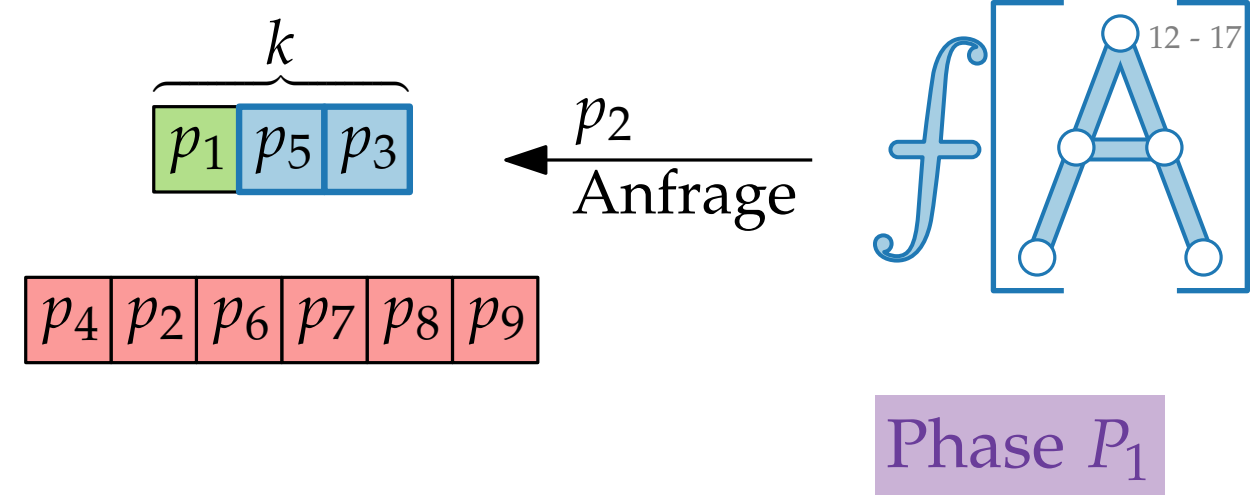
Randomisierte Strategie



Randomisierte Strategie: MARKING

- Geht in Phasen vor
- Am Anfang jeder Phase sind alle Seiten unmarkiert.
- Wenn eine Seite angefragt wird, wird sie **markiert**.
- Unter den unmarkierten Seiten wird eine **zufällig gleichverteilt** entnommen.
- Wenn alle Seiten markiert sind und ein Seitenfehler auftritt, entferne Markierungen und starte neue Phase.

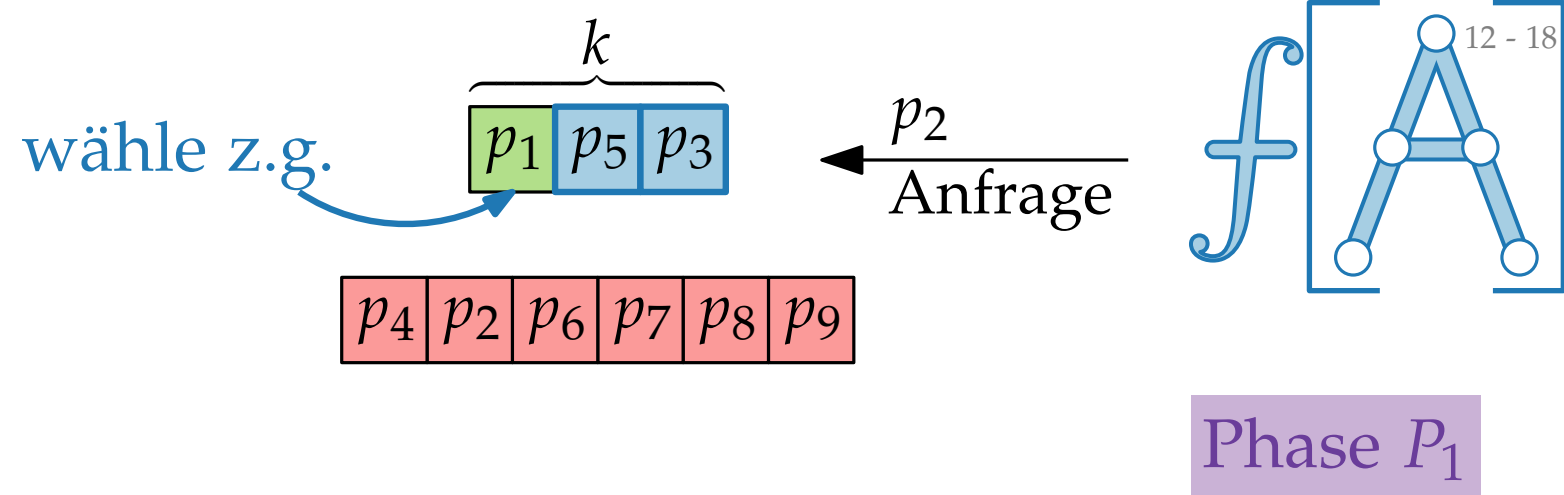
Randomisierte Strategie



Randomisierte Strategie: MARKING

- Geht in Phasen vor
- Am Anfang jeder Phase sind alle Seiten unmarkiert.
- Wenn eine Seite angefragt wird, wird sie **markiert**.
- Unter den unmarkierten Seiten wird eine **zufällig gleichverteilt** entnommen.
- Wenn alle Seiten markiert sind und ein Seitenfehler auftritt, entferne Markierungen und starte neue Phase.

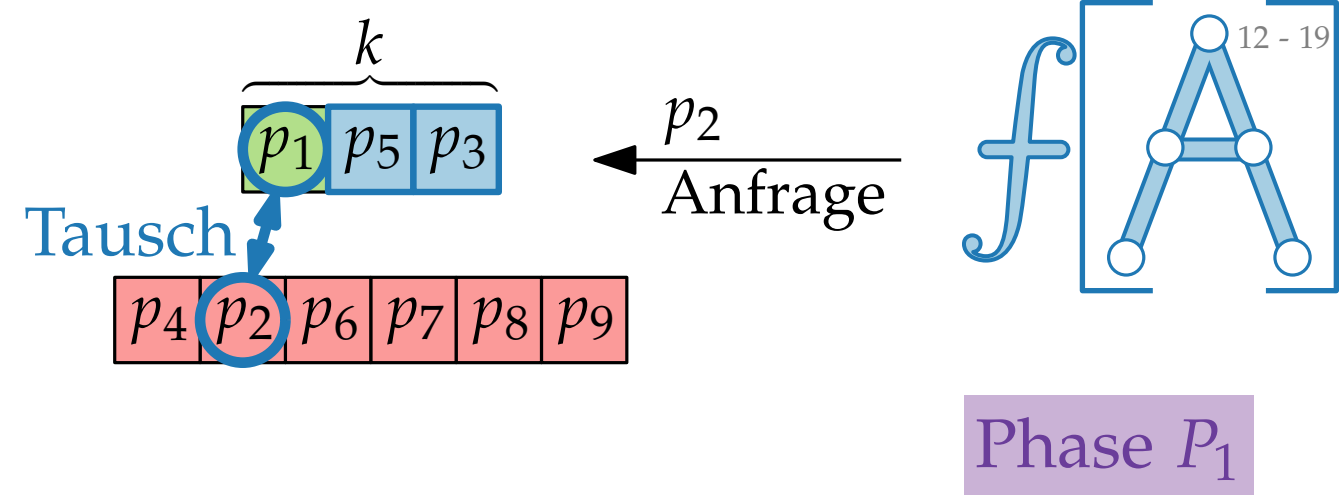
Randomisierte Strategie



Randomisierte Strategie: MARKING

- Geht in Phasen vor
- Am Anfang jeder Phase sind alle Seiten unmarkiert.
- Wenn eine Seite angefragt wird, wird sie **markiert**.
- Unter den unmarkierten Seiten wird eine **zufällig gleichverteilt** entnommen.
- Wenn alle Seiten markiert sind und ein Seitenfehler auftritt, entferne Markierungen und starte neue Phase.

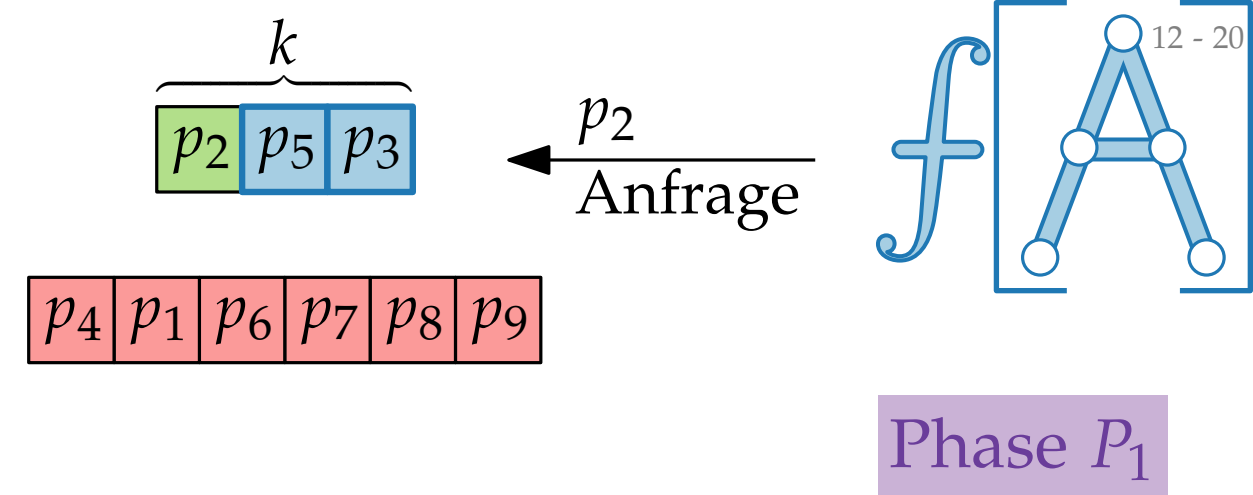
Randomisierte Strategie



Randomisierte Strategie: MARKING

- Geht in Phasen vor
- Am Anfang jeder Phase sind alle Seiten unmarkiert.
- Wenn eine Seite angefragt wird, wird sie **markiert**.
- Unter den unmarkierten Seiten wird eine **zufällig gleichverteilt** entnommen.
- Wenn alle Seiten markiert sind und ein Seitenfehler auftritt, entferne Markierungen und starte neue Phase.

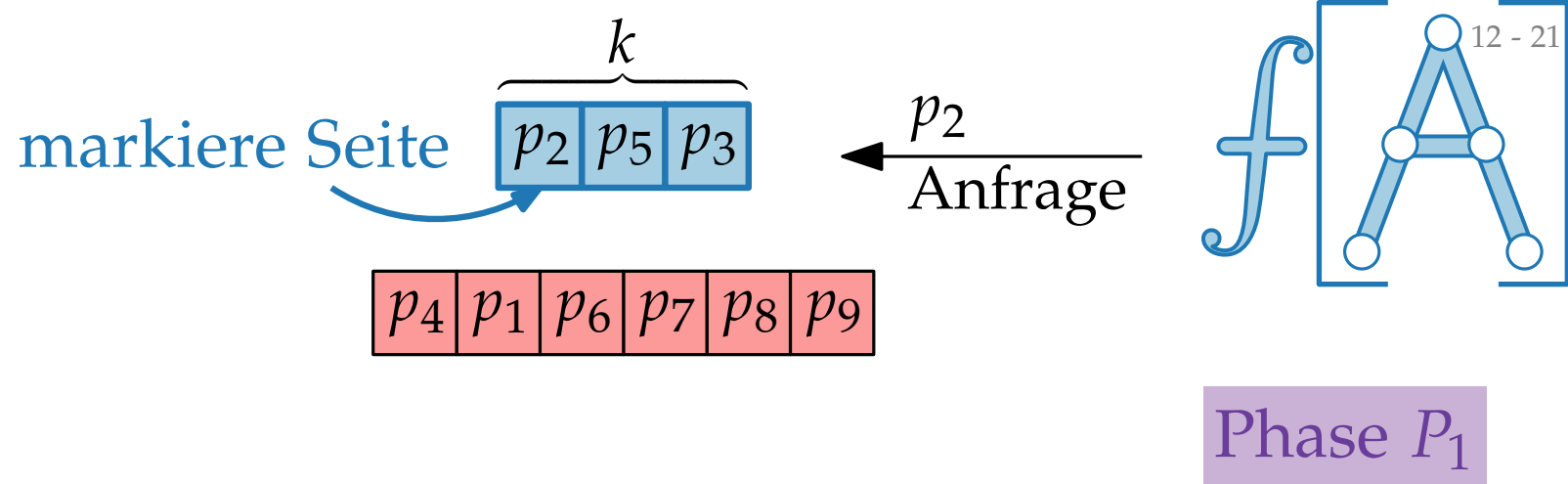
Randomisierte Strategie



Randomisierte Strategie: MARKING

- Geht in Phasen vor
- Am Anfang jeder Phase sind alle Seiten unmarkiert.
- Wenn eine Seite angefragt wird, wird sie **markiert**.
- Unter den unmarkierten Seiten wird eine **zufällig gleichverteilt** entnommen.
- Wenn alle Seiten markiert sind und ein Seitenfehler auftritt, entferne Markierungen und starte neue Phase.

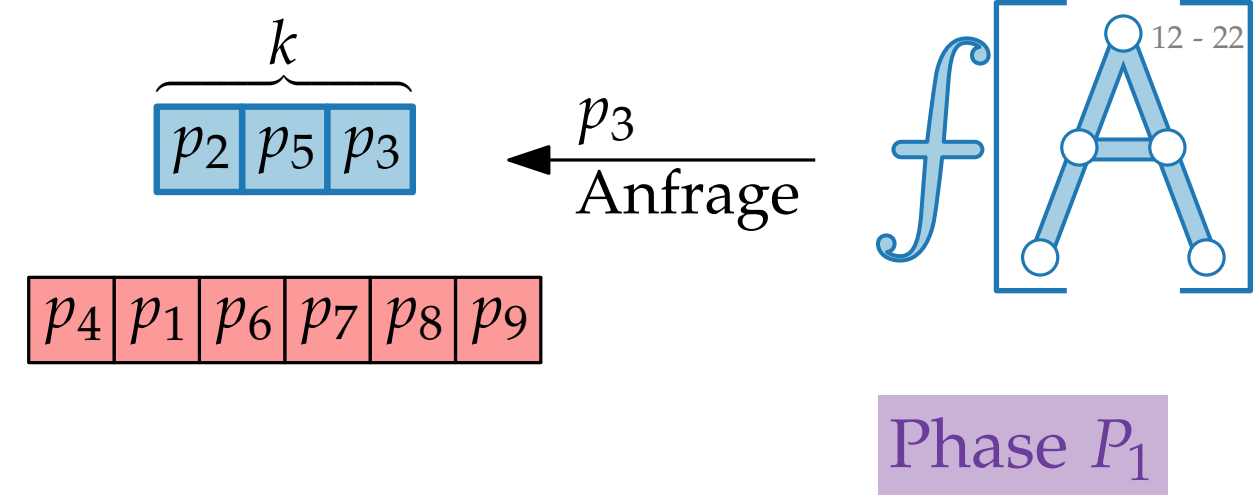
Randomisierte Strategie



Randomisierte Strategie: MARKING

- Geht in Phasen vor
- Am Anfang jeder Phase sind alle Seiten unmarkiert.
- Wenn eine Seite angefragt wird, wird sie **markiert**.
- Unter den unmarkierten Seiten wird eine **zufällig gleichverteilt** entnommen.
- Wenn alle Seiten markiert sind und ein Seitenfehler auftritt, entferne Markierungen und starte neue Phase.

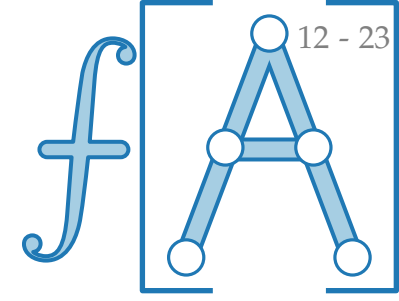
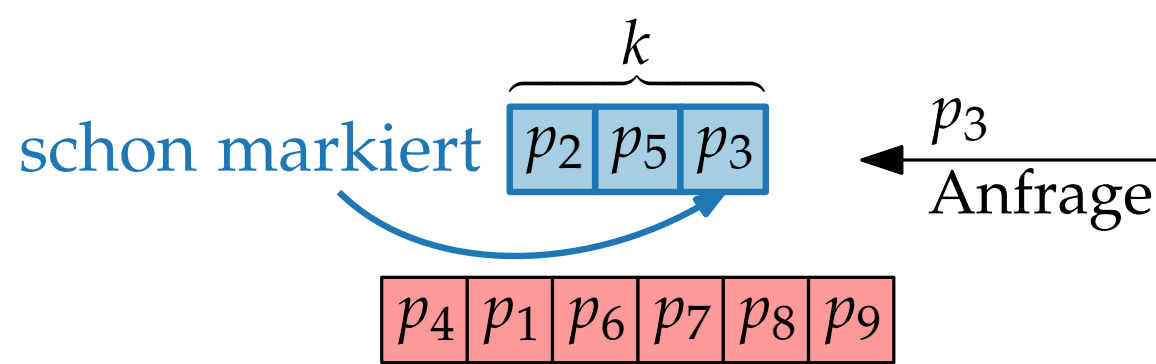
Randomisierte Strategie



Randomisierte Strategie: MARKING

- Geht in Phasen vor
- Am Anfang jeder Phase sind alle Seiten unmarkiert.
- Wenn eine Seite angefragt wird, wird sie **markiert**.
- Unter den unmarkierten Seiten wird eine **zufällig gleichverteilt** entnommen.
- Wenn alle Seiten markiert sind und ein Seitenfehler auftritt, entferne Markierungen und starte neue Phase.

Randomisierte Strategie

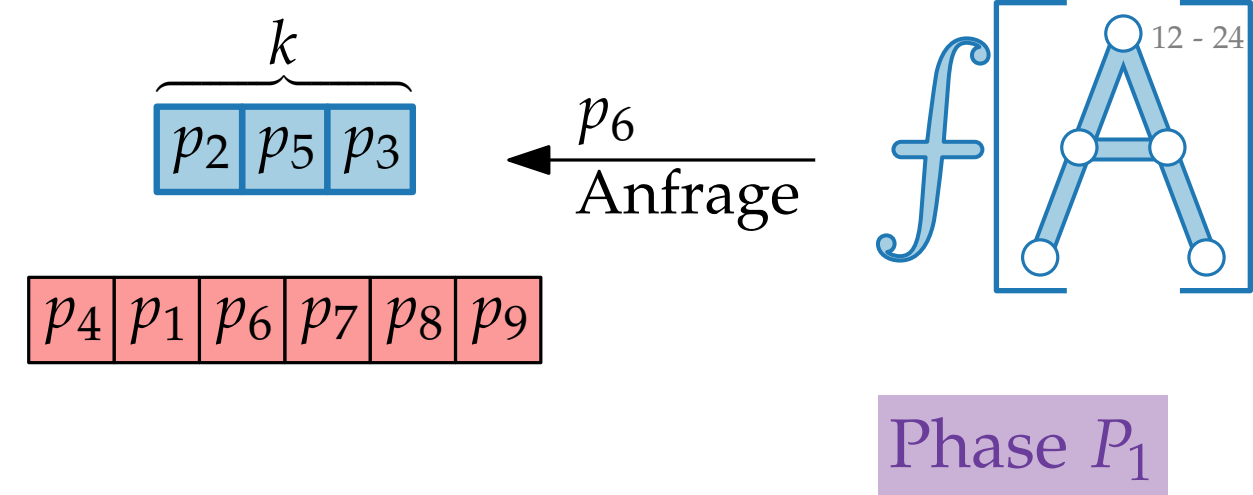


Phase P_1

Randomisierte Strategie: MARKING

- Geht in Phasen vor
- Am Anfang jeder Phase sind alle Seiten unmarkiert.
- Wenn eine Seite angefragt wird, wird sie **markiert**.
- Unter den unmarkierten Seiten wird eine **zufällig gleichverteilt** entnommen.
- Wenn alle Seiten markiert sind und ein Seitenfehler auftritt, entferne Markierungen und starte neue Phase.

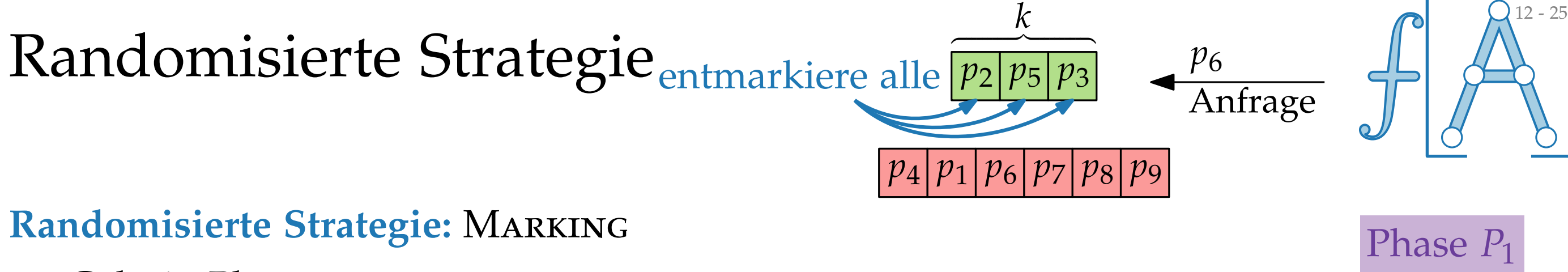
Randomisierte Strategie



Randomisierte Strategie: MARKING

- Geht in Phasen vor
- Am Anfang jeder Phase sind alle Seiten unmarkiert.
- Wenn eine Seite angefragt wird, wird sie **markiert**.
- Unter den unmarkierten Seiten wird eine **zufällig gleichverteilt** entnommen.
- Wenn alle Seiten markiert sind und ein Seitenfehler auftritt, entferne Markierungen und starte neue Phase.

Randomisierte Strategie



Randomisierte Strategie: MARKING

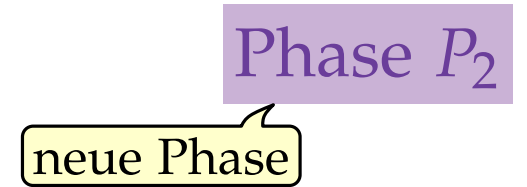
- Geht in Phasen vor
- Am Anfang jeder Phase sind alle Seiten unmarkiert.
- Wenn eine Seite angefragt wird, wird sie **markiert**.
- Unter den unmarkierten Seiten wird eine **zufällig gleichverteilt** entnommen.
- Wenn alle Seiten markiert sind und ein Seitenfehler auftritt, entferne Markierungen und starte neue Phase.

Randomisierte Strategie

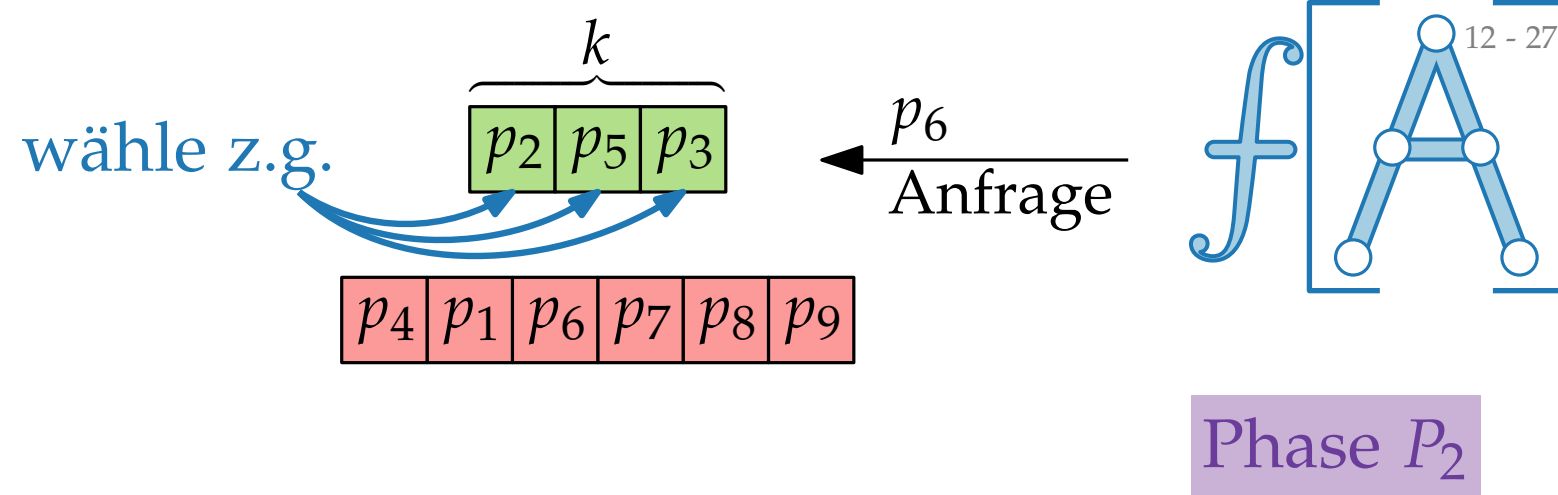


Randomisierte Strategie: MARKING

- Geht in Phasen vor
- Am Anfang jeder Phase sind alle Seiten unmarkiert.
- Wenn eine Seite angefragt wird, wird sie **markiert**.
- Unter den unmarkierten Seiten wird eine **zufällig gleichverteilt** entnommen.
- Wenn alle Seiten markiert sind und ein Seitenfehler auftritt, entferne Markierungen und starte neue Phase.



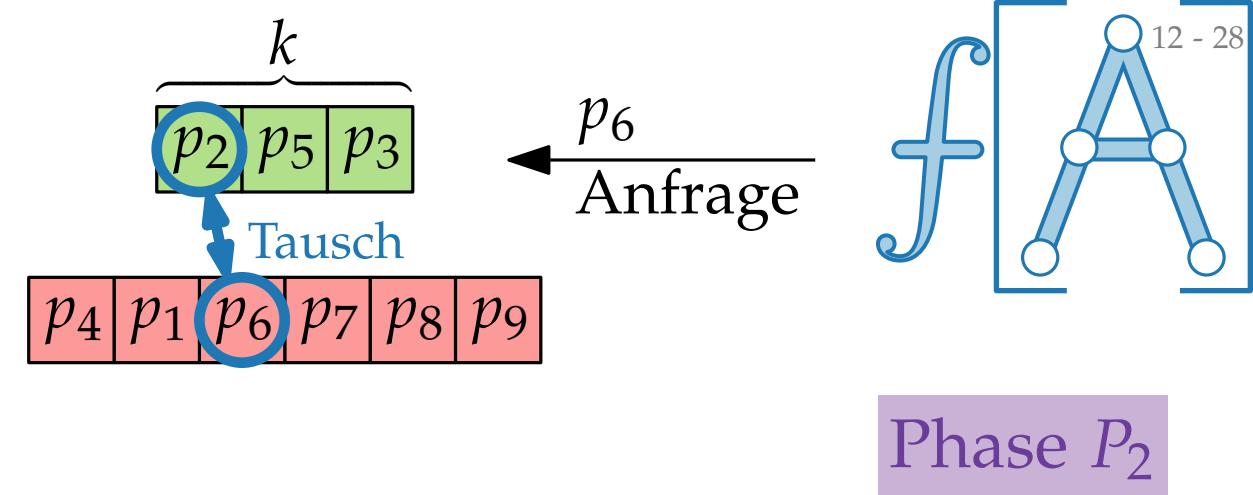
Randomisierte Strategie



Randomisierte Strategie: MARKING

- Geht in Phasen vor
- Am Anfang jeder Phase sind alle Seiten unmarkiert.
- Wenn eine Seite angefragt wird, wird sie **markiert**.
- Unter den unmarkierten Seiten wird eine **zufällig gleichverteilt** entnommen.
- Wenn alle Seiten markiert sind und ein Seitenfehler auftritt, entferne Markierungen und starte neue Phase.

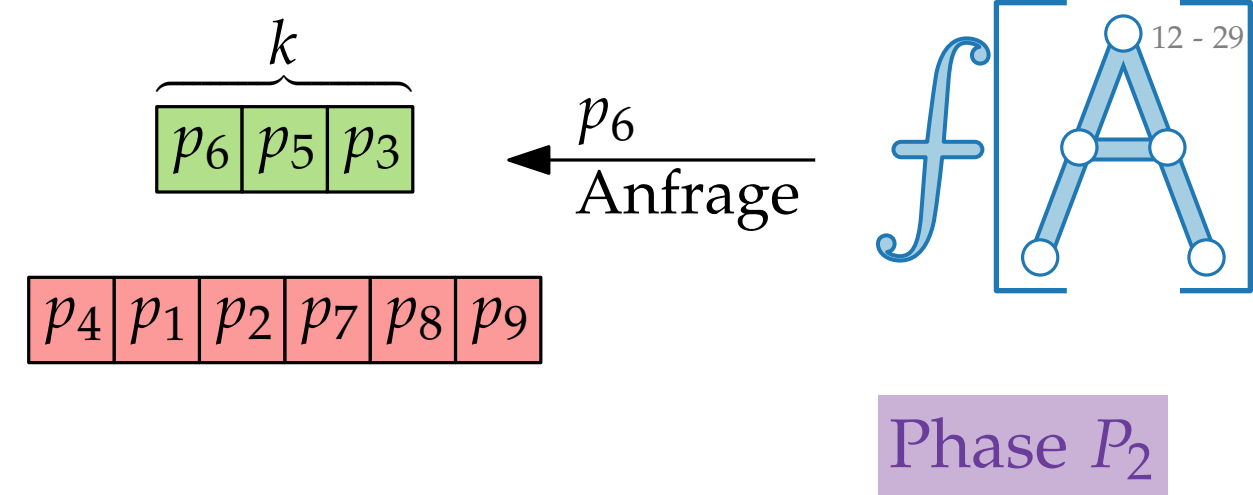
Randomisierte Strategie



Randomisierte Strategie: MARKING

- Geht in Phasen vor
- Am Anfang jeder Phase sind alle Seiten unmarkiert.
- Wenn eine Seite angefragt wird, wird sie **markiert**.
- Unter den unmarkierten Seiten wird eine **zufällig gleichverteilt** entnommen.
- Wenn alle Seiten markiert sind und ein Seitenfehler auftritt, entferne Markierungen und starte neue Phase.

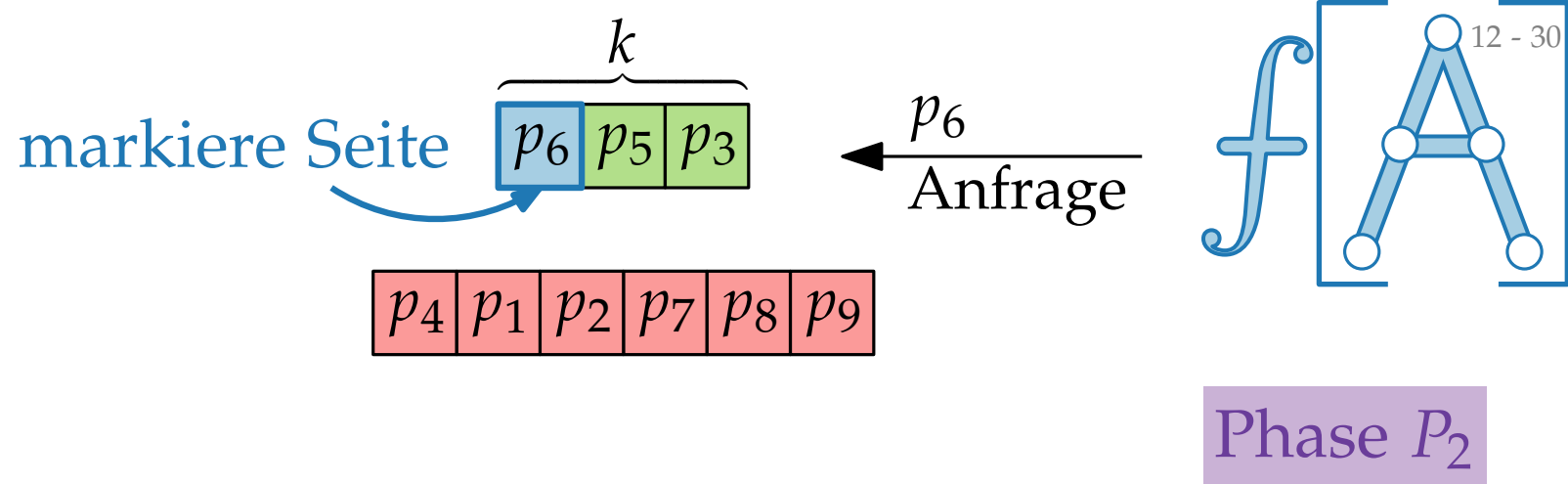
Randomisierte Strategie



Randomisierte Strategie: MARKING

- Geht in Phasen vor
- Am Anfang jeder Phase sind alle Seiten unmarkiert.
- Wenn eine Seite angefragt wird, wird sie **markiert**.
- Unter den unmarkierten Seiten wird eine **zufällig gleichverteilt** entnommen.
- Wenn alle Seiten markiert sind und ein Seitenfehler auftritt, entferne Markierungen und starte neue Phase.

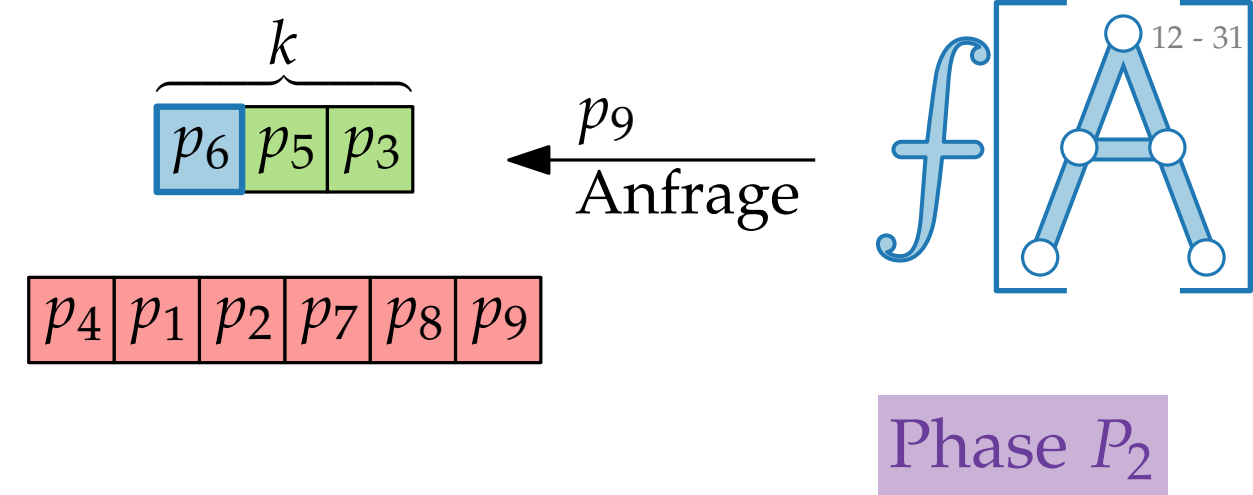
Randomisierte Strategie



Randomisierte Strategie: MARKING

- Geht in Phasen vor
- Am Anfang jeder Phase sind alle Seiten unmarkiert.
- Wenn eine Seite angefragt wird, wird sie **markiert**.
- Unter den unmarkierten Seiten wird eine **zufällig gleichverteilt** entnommen.
- Wenn alle Seiten markiert sind und ein Seitenfehler auftritt, entferne Markierungen und starte neue Phase.

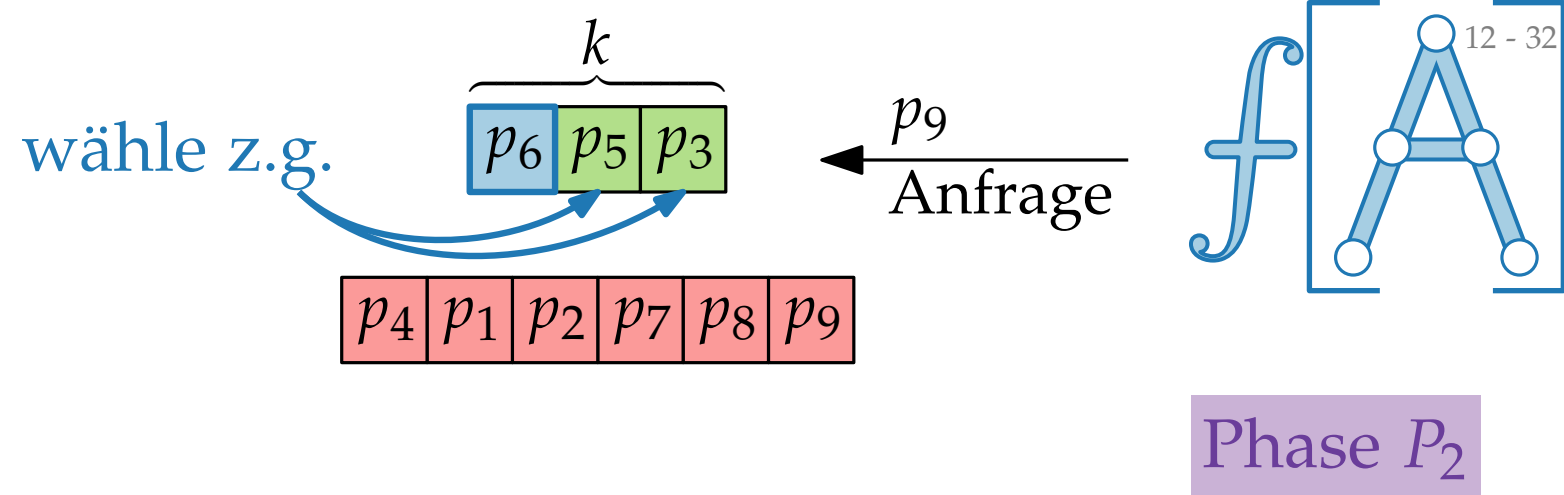
Randomisierte Strategie



Randomisierte Strategie: MARKING

- Geht in Phasen vor
- Am Anfang jeder Phase sind alle Seiten unmarkiert.
- Wenn eine Seite angefragt wird, wird sie **markiert**.
- Unter den unmarkierten Seiten wird eine **zufällig gleichverteilt** entnommen.
- Wenn alle Seiten markiert sind und ein Seitenfehler auftritt, entferne Markierungen und starte neue Phase.

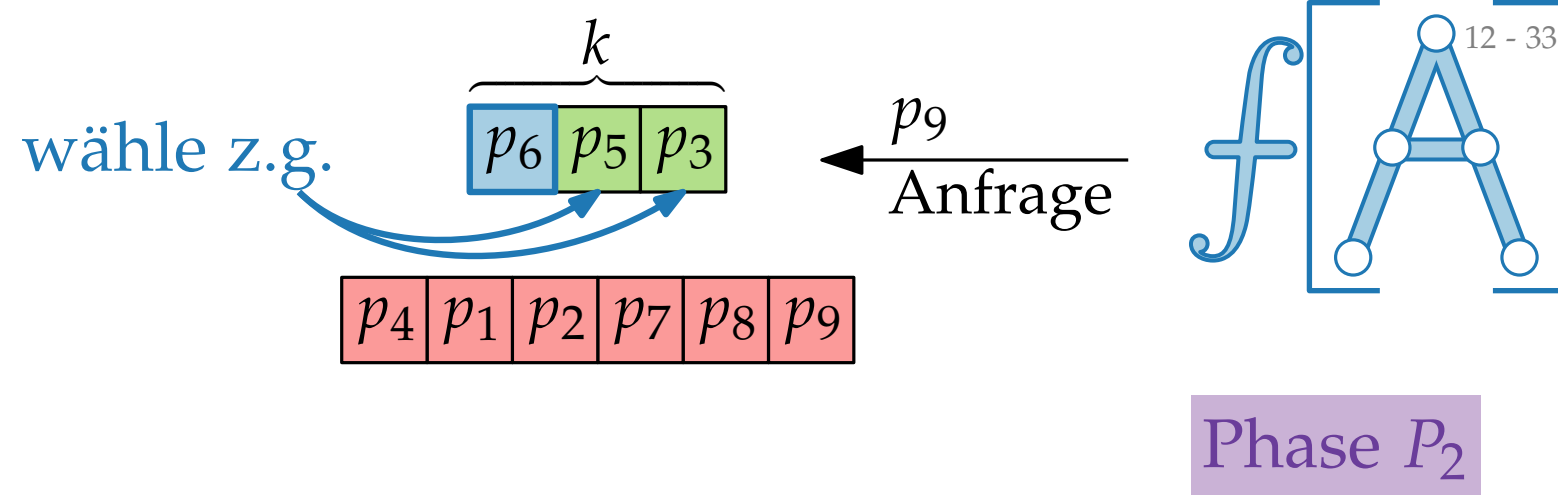
Randomisierte Strategie



Randomisierte Strategie: MARKING

- Geht in Phasen vor
- Am Anfang jeder Phase sind alle Seiten unmarkiert.
- Wenn eine Seite angefragt wird, wird sie **markiert**.
- Unter den unmarkierten Seiten wird eine **zufällig gleichverteilt** entnommen.
- Wenn alle Seiten markiert sind und ein Seitenfehler auftritt, entferne Markierungen und starte neue Phase.

Randomisierte Strategie

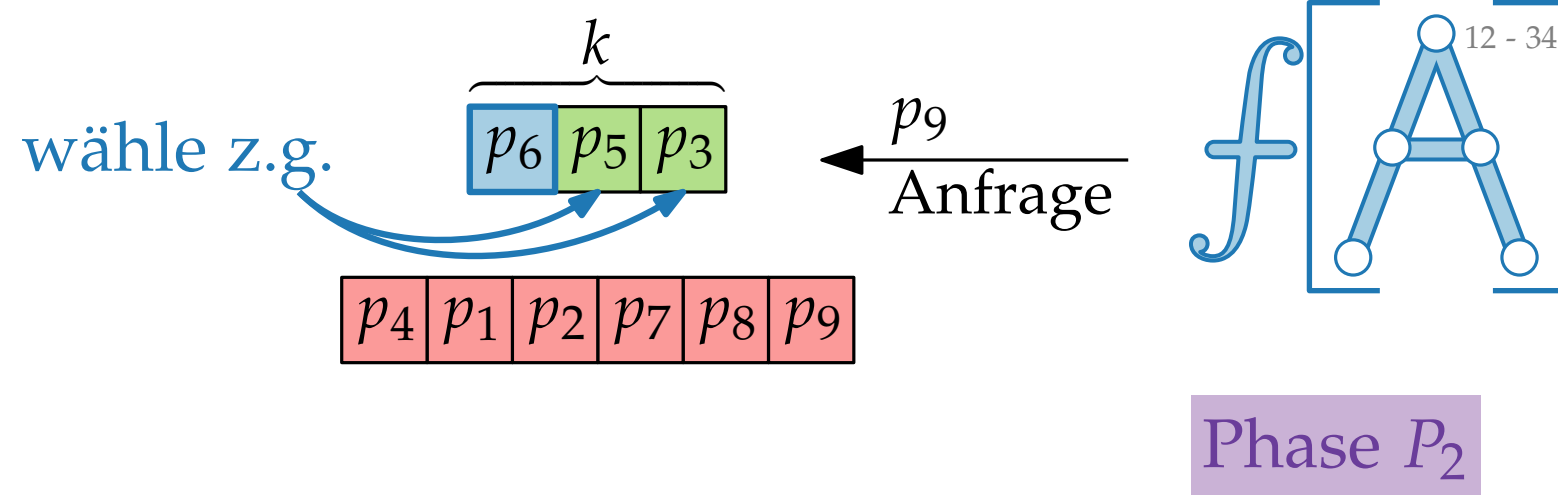


Randomisierte Strategie: MARKING

- Geht in Phasen vor
- Am Anfang jeder Phase sind alle Seiten unmarkiert.
- Wenn eine Seite angefragt wird, wird sie **markiert**.
- Unter den unmarkierten Seiten wird eine **zufällig gleichverteilt** entnommen.
- Wenn alle Seiten markiert sind und ein Seitenfehler auftritt, entferne Markierungen und starte neue Phase.

Satz. MARKING ist $2H_k$ -kompetitiv.

Randomisierte Strategie



Randomisierte Strategie: MARKING

- Geht in Phasen vor
- Am Anfang jeder Phase sind alle Seiten unmarkiert.
- Wenn eine Seite angefragt wird, wird sie **markiert**.
- Unter den unmarkierten Seiten wird eine **zufällig gleichverteilt** entnommen.
- Wenn alle Seiten markiert sind und ein Seitenfehler auftritt, entferne Markierungen und starte neue Phase.

Satz. MARKING ist $2H_k$ -kompetitiv.

harmonische Reihe

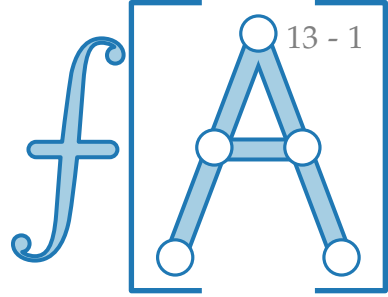
$$\ln(k+1) \leq H_k \leq \ln(k) + 1.$$

Analyse

nicht
prüfungsrelevant

Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.



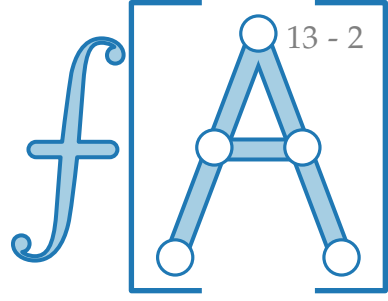
Analyse

nicht
prüfungsrelevant

Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

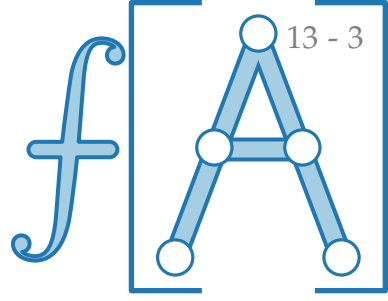
Betrachte
Phase P_i .



Analyse

nicht
prüfungsrelevant

Betrachte
Phase P_i .



Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

- Eine Seite heißt **schal**, wenn sie unmarkiert ist, aber in P_{i-1} markiert war.

Analyse

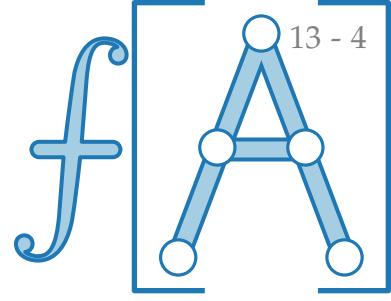
nicht
prüfungsrelevant

Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

- Eine Seite heißt **schal**, wenn sie unmarkiert ist, aber in P_{i-1} markiert war.

Betrachte
Phase P_i .

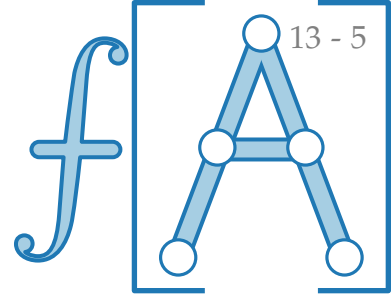


→ Am Anfang von P_i im Cache.

Analyse

nicht
prüfungsrelevant

Betrachte
Phase P_i .



Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

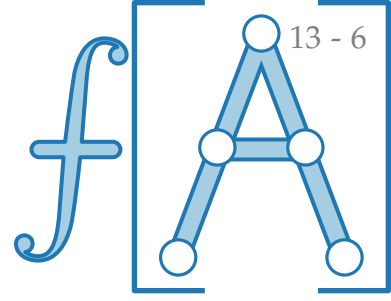
- Eine Seite heißt **schal**, wenn sie unmarkiert ist, aber in P_{i-1} markiert war.
- Eine Seite ist **sauber**, wenn sie unmarkiert, aber nicht **schal** ist.

→ Am Anfang von P_i im Cache.

Analyse

nicht
prüfungsrelevant

Betrachte
Phase P_i .



Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

- Eine Seite heißt **schal**, wenn sie unmarkiert ist, aber in P_{i-1} markiert war.
- Eine Seite ist **sauber**, wenn sie unmarkiert, aber nicht **schal** ist.

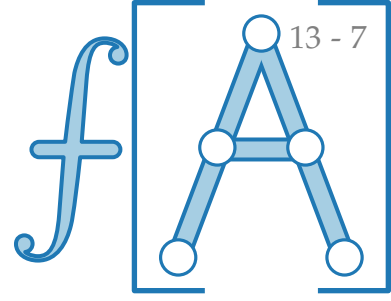
→ Am Anfang von P_i im Cache.

→ nicht im Cache!

Analyse

nicht
prüfungsrelevant

Betrachte
Phase P_i .



Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

- Eine Seite heißt **schal**, wenn sie unmarkiert ist, aber in P_{i-1} markiert war.
- Eine Seite ist **sauber**, wenn sie unmarkiert, aber nicht **schal** ist.
- $S_{\text{MARK}} (S_{\text{OPT}})$: Menge der Seiten im Cache von MARKING (OPT).

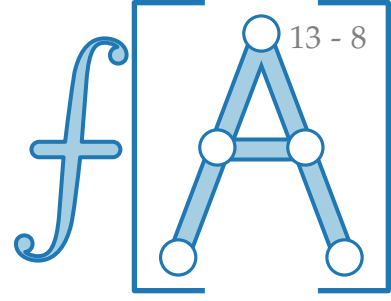
→ Am Anfang von P_i im Cache.

→ nicht im Cache!

Analyse

nicht
prüfungsrelevant

Betrachte
Phase P_i .



Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

- Eine Seite heißt **schal**, wenn sie unmarkiert ist, aber in P_{i-1} markiert war.
- Eine Seite ist **sauber**, wenn sie unmarkiert, aber nicht **schal** ist.
- $S_{\text{MARK}} (S_{\text{OPT}})$: Menge der Seiten im Cache von MARKING (OPT).
- d_{begin} : $|S_{\text{OPT}} \setminus S_{\text{MARK}}|$ zu Beginn von P_i .

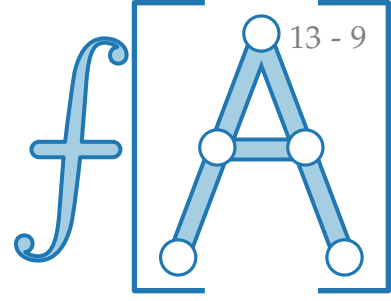
→ Am Anfang von P_i im Cache.

→ nicht im Cache!

Analyse

nicht
prüfungsrelevant

Betrachte
Phase P_i .



Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

■ Eine Seite heißt **schal**, wenn sie unmarkiert ist, aber in P_{i-1} markiert war.

→ Am Anfang von P_i im Cache.

■ Eine Seite ist **sauber**, wenn sie unmarkiert, aber nicht **schal** ist. → nicht im Cache!

■ $S_{\text{MARK}}(S_{\text{OPT}})$: Menge der Seiten im Cache von MARKING (OPT).

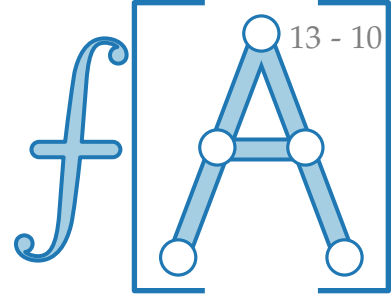
■ d_{begin} : $|S_{\text{OPT}} \setminus S_{\text{MARK}}|$ zu Beginn von P_i .

■ d_{end} : $|S_{\text{OPT}} \setminus S_{\text{MARK}}|$ am Ende von P_i .

Analyse

nicht
prüfungsrelevant

Betrachte
Phase P_i .



Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

→ Am Anfang von P_i im Cache.

■ Eine Seite heißt **schal**, wenn sie unmarkiert ist, aber in P_{i-1} markiert war.

■ Eine Seite ist **sauber**, wenn sie unmarkiert, aber nicht **schal** ist. → nicht im Cache!

■ $S_{\text{MARK}} (S_{\text{OPT}})$: Menge der Seiten im Cache von MARKING (OPT).

■ d_{begin} : $|S_{\text{OPT}} \setminus S_{\text{MARK}}|$ zu Beginn von P_i .

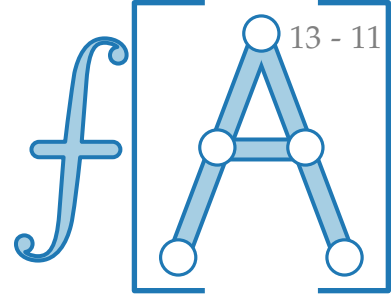
■ d_{end} : $|S_{\text{OPT}} \setminus S_{\text{MARK}}|$ am Ende von P_i .

■ c : Anzahl der **sauberen Seiten**, die in P_i angefragt werden.

Analyse

nicht
prüfungsrelevant

Betrachte
Phase P_i .



Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

■ Eine Seite heißt **schal**, wenn sie unmarkiert ist, aber in P_{i-1} markiert war.

→ Am Anfang von P_i im Cache.

■ Eine Seite ist **sauber**, wenn sie unmarkiert, aber nicht **schal** ist. → nicht im Cache!

■ $S_{\text{MARK}} (S_{\text{OPT}})$: Menge der Seiten im Cache von MARKING (OPT).

■ d_{begin} : $|S_{\text{OPT}} \setminus S_{\text{MARK}}|$ zu Beginn von P_i .

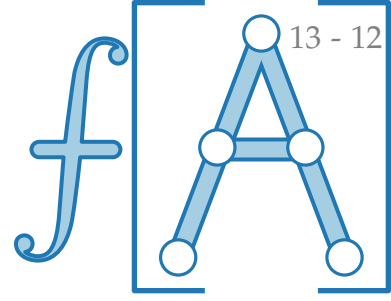
■ d_{end} : $|S_{\text{OPT}} \setminus S_{\text{MARK}}|$ am Ende von P_i .

■ c : Anzahl der **sauberen Seiten**, die in P_i angefragt werden → jede führt zu Fehler in MARKING

Analyse

nicht
prüfungsrelevant

Betrachte
Phase P_i .



Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

■ Eine Seite heißt **schal**, wenn sie unmarkiert ist, aber in P_{i-1} markiert war.

→ Am Anfang von P_i im Cache.

■ Eine Seite ist **sauber**, wenn sie unmarkiert, aber nicht **schal** ist. → nicht im Cache!

■ $S_{\text{MARK}} (S_{\text{OPT}})$: Menge der Seiten im Cache von MARKING (OPT).

■ d_{begin} : $|S_{\text{OPT}} \setminus S_{\text{MARK}}|$ zu Beginn von P_i .

■ d_{end} : $|S_{\text{OPT}} \setminus S_{\text{MARK}}|$ am Ende von P_i .

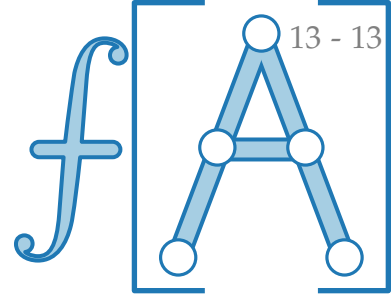
■ c : Anzahl der **sauberen Seiten**, die in P_i angefragt werden → jede führt zu Fehler in MARKING

■ OPT hat $\geq$ Fehler.

Analyse

nicht
prüfungsrelevant

Betrachte
Phase P_i .



Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

→ Am Anfang von P_i im Cache.

■ Eine Seite heißt **schal**, wenn sie unmarkiert ist, aber in P_{i-1} markiert war.

■ Eine Seite ist **sauber**, wenn sie unmarkiert, aber nicht **schal** ist. → nicht im Cache!

■ $S_{\text{MARK}} (S_{\text{OPT}})$: Menge der Seiten im Cache von MARKING (OPT).

■ d_{begin} : $|S_{\text{OPT}} \setminus S_{\text{MARK}}|$ zu Beginn von P_i .

■ d_{end} : $|S_{\text{OPT}} \setminus S_{\text{MARK}}|$ am Ende von P_i .

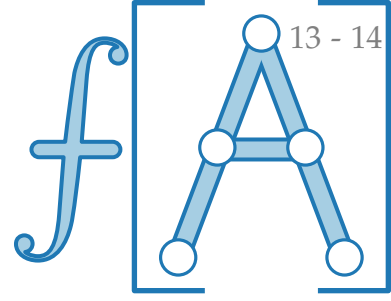
■ c : Anzahl der **sauberen Seiten**, die in P_i angefragt werden → jede führt zu Fehler in MARKING

■ OPT hat $\geq c - d_{\text{begin}}$ Fehler.

Analyse

nicht
prüfungsrelevant

Betrachte
Phase P_i .



Satz. MARKING ist $2H_k$ -kompetitiv.

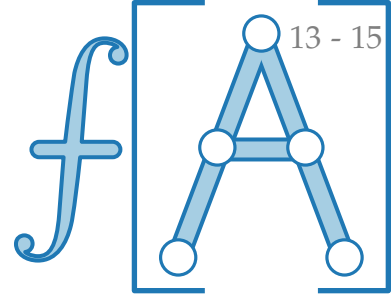
Beweis.

- Eine Seite heißt **schal**, wenn sie unmarkiert ist, aber in P_{i-1} markiert war. → Am Anfang von P_i im Cache.
- Eine Seite ist **sauber**, wenn sie unmarkiert, aber nicht **schal** ist. → nicht im Cache!
- $S_{\text{MARK}} (S_{\text{OPT}})$: Menge der Seiten im Cache von MARKING (OPT).
- d_{begin} : $|S_{\text{OPT}} \setminus S_{\text{MARK}}|$ zu Beginn von P_i .
- d_{end} : $|S_{\text{OPT}} \setminus S_{\text{MARK}}|$ am Ende von P_i .
- c : Anzahl der **sauberen Seiten**, die in P_i angefragt werden → jede führt zu Fehler in MARKING
- OPT hat $\geq \max(c - d_{\text{begin}}, d_{\text{end}})$ Fehler.

Analyse

nicht
prüfungsrelevant

Betrachte
Phase P_i .



Satz. MARKING ist $2H_k$ -kompetitiv.

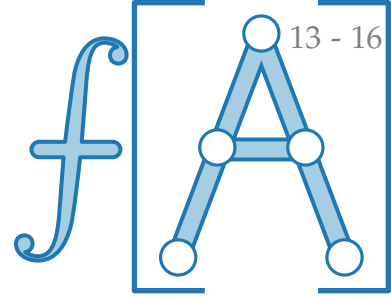
Beweis.

- Eine Seite heißt **schal**, wenn sie unmarkiert ist, aber in P_{i-1} markiert war. → Am Anfang von P_i im Cache.
- Eine Seite ist **sauber**, wenn sie unmarkiert, aber nicht **schal** ist. → nicht im Cache!
- $S_{\text{MARK}} (S_{\text{OPT}})$: Menge der Seiten im Cache von MARKING (OPT).
- d_{begin} : $|S_{\text{OPT}} \setminus S_{\text{MARK}}|$ zu Beginn von P_i .
- d_{end} : $|S_{\text{OPT}} \setminus S_{\text{MARK}}|$ am Ende von P_i .
- c : Anzahl der **sauberen Seiten**, die in P_i angefragt werden → jede führt zu Fehler in MARKING
- OPT hat $\geq \max(c - d_{\text{begin}}, d_{\text{end}}) \geq \frac{1}{2}(c - d_{\text{begin}} + d_{\text{end}})$ Fehler.

Analyse

nicht
prüfungsrelevant

Betrachte
Phase P_i .



Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

→ Am Anfang von P_i im Cache.

■ Eine Seite heißt **schal**, wenn sie unmarkiert ist, aber in P_{i-1} markiert war.

■ Eine Seite ist **sauber**, wenn sie unmarkiert, aber nicht **schal** ist. → nicht im Cache!

■ $S_{\text{MARK}}(S_{\text{OPT}})$: Menge der Seiten im Cache von MARKING (OPT).

■ d_{begin} : $|S_{\text{OPT}} \setminus S_{\text{MARK}}|$ zu Beginn von P_i .

■ d_{end} : $|S_{\text{OPT}} \setminus S_{\text{MARK}}|$ am Ende von P_i .

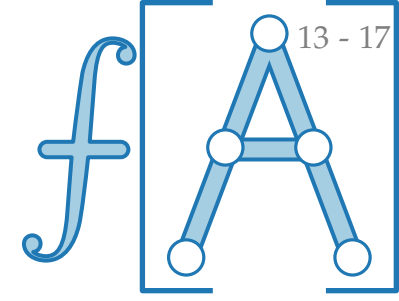
■ c : Anzahl der **sauberen Seiten**, die in P_i angefragt werden → jede führt zu Fehler in MARKING

■ OPT hat $\geq \max(c - d_{\text{begin}}, d_{\text{end}}) \geq \frac{1}{2}(c - d_{\text{begin}} + d_{\text{end}}) = \frac{c}{2} - \frac{d_{\text{begin}}}{2} + \frac{d_{\text{end}}}{2}$ Fehler.

Analyse

nicht
prüfungsrelevant

Betrachte
Phase P_i .



Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

→ Am Anfang von P_i im Cache.

■ Eine Seite heißt **schal**, wenn sie unmarkiert ist, aber in P_{i-1} markiert war.

■ Eine Seite ist **sauber**, wenn sie unmarkiert, aber nicht **schal** ist. → nicht im Cache!

■ $S_{\text{MARK}} (S_{\text{OPT}})$: Menge der Seiten im Cache von MARKING (OPT).

■ d_{begin} : $|S_{\text{OPT}} \setminus S_{\text{MARK}}|$ zu Beginn von P_i .

■ d_{end} : $|S_{\text{OPT}} \setminus S_{\text{MARK}}|$ am Ende von P_i .

■ c : Anzahl der **sauberen Seiten**, die in P_i angefragt werden → jede führt zu Fehler in MARKING

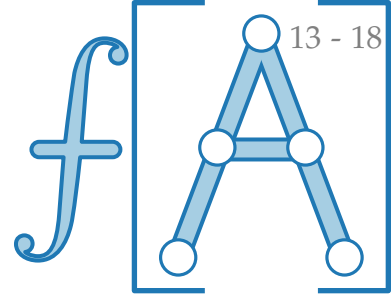
■ OPT hat $\geq \max(c - d_{\text{begin}}, d_{\text{end}}) \geq \frac{1}{2}(c - d_{\text{begin}} + d_{\text{end}}) = \frac{c}{2} - \frac{d_{\text{begin}}}{2} + \frac{d_{\text{end}}}{2}$ Fehler.

Über alle Phasen heben sich $\frac{d_{\text{begin}}}{2}$ und $\frac{d_{\text{end}}}{2}$ auf, außer dem ersten $\frac{d_{\text{begin}}}{2}$ und dem letzten $\frac{d_{\text{end}}}{2}$.

Analyse

nicht
prüfungsrelevant

Betrachte
Phase P_i .



Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

→ Am Anfang von P_i im Cache.

■ Eine Seite heißt **schal**, wenn sie unmarkiert ist, aber in P_{i-1} markiert war.

■ Eine Seite ist **sauber**, wenn sie unmarkiert, aber nicht **schal** ist. → nicht im Cache!

■ $S_{\text{MARK}} (S_{\text{OPT}})$: Menge der Seiten im Cache von MARKING (OPT).

■ d_{begin} : $|S_{\text{OPT}} \setminus S_{\text{MARK}}|$ zu Beginn von P_i .

■ d_{end} : $|S_{\text{OPT}} \setminus S_{\text{MARK}}|$ am Ende von P_i .

■ c : Anzahl der **sauberen Seiten**, die in P_i angefragt werden → jede führt zu Fehler in MARKING

■ OPT hat $\geq \max(c - d_{\text{begin}}, d_{\text{end}}) \geq \frac{1}{2}(c - d_{\text{begin}} + d_{\text{end}}) = \frac{c}{2} - \frac{d_{\text{begin}}}{2} + \frac{d_{\text{end}}}{2}$ Fehler.

Über alle Phasen heben sich $\frac{d_{\text{begin}}}{2}$ und $\frac{d_{\text{end}}}{2}$ auf, außer dem ersten $\frac{d_{\text{begin}}}{2}$ und dem letzten $\frac{d_{\text{end}}}{2}$.

■ Weil das erste $d_{\text{begin}} = 0$, hat OPT **mindestens $\frac{c}{2}$ Fehler pro Phase.**

Analyse

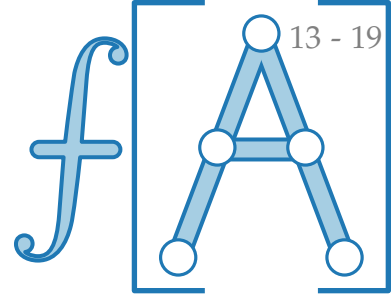
nicht
prüfungsrelevant

Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

- Für saubere Seiten hat MARKING c Fehler.

Betrachte
Phase P_i .



schal: unmarkiert, aber in P_{i-1} markiert
sauber: unmarkiert, aber nicht schal
OPT min. $c/2$ Fehler pro Phase

Analyse

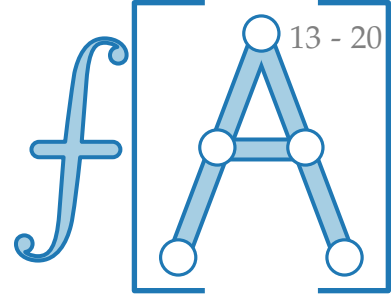
nicht
prüfungsrelevant

Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

- Für saubere Seiten hat MARKING c Fehler.
- Für schale Seiten gibt es $s = k - c \leq k$ Anfragen.

Betrachte
Phase P_i .



schal: unmarkiert, aber in P_{i-1} markiert
sauber: unmarkiert, aber nicht schal
OPT min. $c/2$ Fehler pro Phase

Analyse

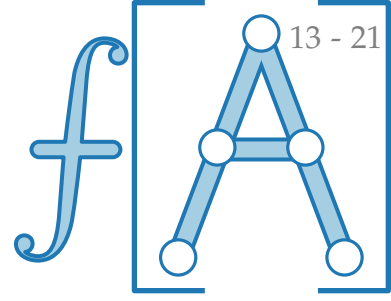
nicht
prüfungsrelevant

Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

- Für saubere Seiten hat MARKING c Fehler.
- Für schale Seiten gibt es $s = k - c \leq k$ Anfragen.
- Für Anfragen $j = 1, \dots, s$ auf schale Seiten, betrachte die erwartete Fehleranzahl $E[F_j]$.

Betrachte
Phase P_i .

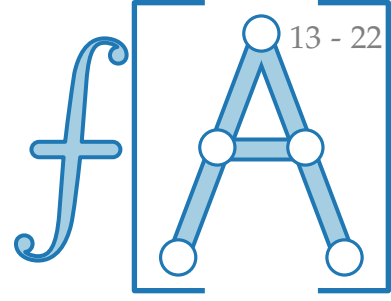


schal: unmarkiert, aber in P_{i-1} markiert
sauber: unmarkiert, aber nicht schal
OPT min. $c/2$ Fehler pro Phase

Analyse

nicht
prüfungsrelevant

Betrachte
Phase P_i .



Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

- Für **saubere Seiten** hat MARKING c Fehler.
- Für **schale Seiten** gibt es $s = k - c \leq k$ Anfragen.
- Für Anfragen $j = 1, \dots, s$ auf **schale Seiten**, betrachte die erwartete Fehleranzahl $E[F_j]$.
- $c(j)$: # **saubere Seiten**, die bisher in dieser Phase angefragt wurden

schal: unmarkiert, aber in P_{i-1} markiert
sauber: unmarkiert, aber nicht schal
OPT min. $c/2$ Fehler pro Phase

Analyse

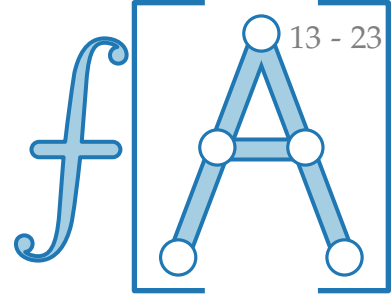
nicht
prüfungsrelevant

Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

- Für saubere Seiten hat MARKING c Fehler.
- Für schale Seiten gibt es $s = k - c \leq k$ Anfragen.
- Für Anfragen $j = 1, \dots, s$ auf schale Seiten, betrachte die erwartete Fehleranzahl $E[F_j]$.
- $c(j)$: # saubere Seiten, die bisher in dieser Phase angefragt wurden
 - für jede davon wurde eine schale Seite rausgeworfen

Betrachte
Phase P_i .

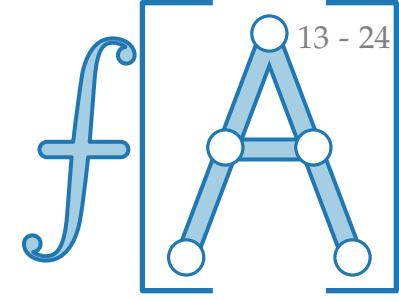


schal: unmarkiert, aber in P_{i-1} markiert
sauber: unmarkiert, aber nicht schal
OPT min. $c/2$ Fehler pro Phase

Analyse

nicht
prüfungsrelevant

Betrachte
Phase P_i .



Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

- Für saubere Seiten hat MARKING c Fehler.
- Für schale Seiten gibt es $s = k - c \leq k$ Anfragen.
- Für Anfragen $j = 1, \dots, s$ auf schale Seiten, betrachte die erwartete Fehleranzahl $E[F_j]$.
- $c(j)$: # saubere Seiten, die bisher in dieser Phase angefragt wurden
 $s(j)$: # schale Seiten, die bisher noch nicht angefragt wurden

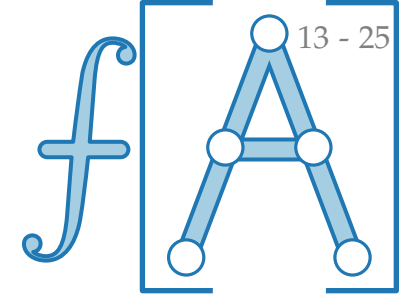
schal: unmarkiert, aber in P_{i-1} markiert
sauber: unmarkiert, aber nicht schal
OPT min. $c/2$ Fehler pro Phase

→ für jede davon
wurde eine schale
Seite rausgeworfen

Analyse

nicht
prüfungsrelevant

Betrachte
Phase P_i .



Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

- Für saubere Seiten hat MARKING c Fehler.
- Für schale Seiten gibt es $s = k - c \leq k$ Anfragen.
- Für Anfragen $j = 1, \dots, s$ auf schale Seiten, betrachte die erwartete Fehleranzahl $E[F_j]$.
- $c(j)$: # saubere Seiten, die bisher in dieser Phase angefragt wurden
 $s(j)$: # schale Seiten, die bisher noch nicht angefragt wurden
- $E[F_j] =$

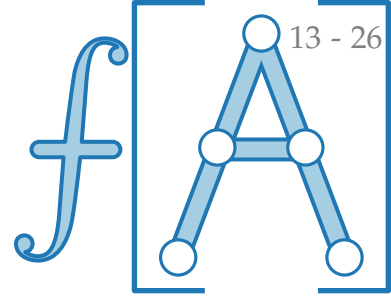
schal: unmarkiert, aber in P_{i-1} markiert
sauber: unmarkiert, aber nicht schal
OPT min. $c/2$ Fehler pro Phase

→ für jede davon
wurde eine schale
Seite rausgeworfen

Analyse

nicht
prüfungsrelevant

Betrachte
Phase P_i .



Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

- Für **saubere Seiten** hat MARKING c Fehler.
- Für **schale Seiten** gibt es $s = k - c \leq k$ Anfragen.
- Für Anfragen $j = 1, \dots, s$ auf **schale Seiten**, betrachte die erwartete Fehleranzahl $E[F_j]$.
- $c(j)$: # **saubere Seiten**, die bisher in dieser Phase angefragt wurden
 $s(j)$: # **schale Seiten**, die bisher noch nicht angefragt wurden
- $E[F_j] = \quad \cdot 0 + \quad \cdot 1 \leq$

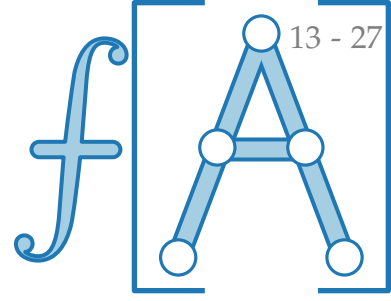
schal: unmarkiert, aber in P_{i-1} markiert
sauber: unmarkiert, aber nicht schal
OPT min. $c/2$ Fehler pro Phase

→ für jede davon
wurde eine schale
Seite rausgeworfen

Analyse

nicht
prüfungsrelevant

Betrachte
Phase P_i .



Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

- Für **saubere Seiten** hat MARKING c Fehler.
- Für **schale Seiten** gibt es $s = k - c \leq k$ Anfragen.
- Für Anfragen $j = 1, \dots, s$ auf **schale Seiten**, betrachte die erwartete Fehleranzahl $E[F_j]$.
- $c(j)$: # **saubere Seiten**, die bisher in dieser Phase angefragt wurden
 $s(j)$: # **schale Seiten**, die bisher noch nicht angefragt wurden
- $E[F_j] = \quad \cdot 0 + \frac{c(j)}{s(j)} \cdot 1 \leq$

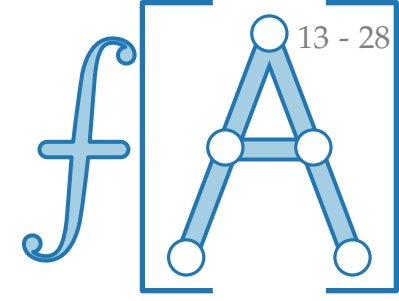
schal: unmarkiert, aber in P_{i-1} markiert
sauber: unmarkiert, aber nicht schal
OPT min. $c/2$ Fehler pro Phase

→ für jede davon
wurde eine schale
Seite rausgeworfen

Analyse

nicht
prüfungsrelevant

Betrachte
Phase P_i .



Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

- Für **saubere Seiten** hat MARKING c Fehler.
- Für **schale Seiten** gibt es $s = k - c \leq k$ Anfragen.
- Für Anfragen $j = 1, \dots, s$ auf **schale Seiten**, betrachte die erwartete Fehleranzahl $E[F_j]$.
- $c(j)$: # **saubere Seiten**, die bisher in dieser Phase angefragt wurden
 $s(j)$: # **schale Seiten**, die bisher noch nicht angefragt wurden
- $E[F_j] = \frac{s(j)-c(j)}{s(j)} \cdot 0 + \frac{c(j)}{s(j)} \cdot 1 \leq$

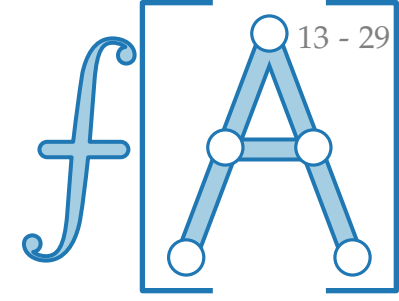
schal: unmarkiert, aber in P_{i-1} markiert
sauber: unmarkiert, aber nicht schal
OPT min. $c/2$ Fehler pro Phase

→ für jede davon
wurde eine schale
Seite rausgeworfen

Analyse

nicht
prüfungsrelevant

Betrachte
Phase P_i .



Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

- Für **saubere Seiten** hat MARKING c Fehler.
- Für **schale Seiten** gibt es $s = k - c \leq k$ Anfragen.
- Für Anfragen $j = 1, \dots, s$ auf **schale Seiten**, betrachte die erwartete Fehleranzahl $E[F_j]$.
- $c(j)$: # **saubere Seiten**, die bisher in dieser Phase angefragt wurden
 $s(j)$: # **schale Seiten**, die bisher noch nicht angefragt wurden
- $E[F_j] = \frac{s(j)-c(j)}{s(j)} \cdot 0 + \frac{c(j)}{s(j)} \cdot 1 \leq \frac{c}{k+1-j}$
 $s(j) \geq k+1-j$

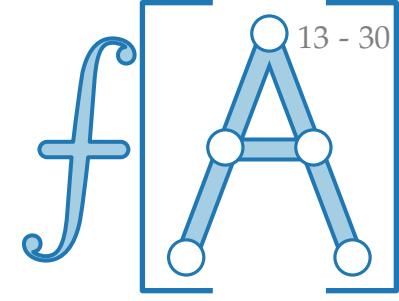
schal: unmarkiert, aber in P_{i-1} markiert
sauber: unmarkiert, aber nicht schal
OPT min. $c/2$ Fehler pro Phase

→ für jede davon
wurde eine schale
Seite rausgeworfen

Analyse

nicht
prüfungsrelevant

Betrachte
Phase P_i .



Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

- Für **saubere Seiten** hat MARKING c Fehler.
- Für **schale Seiten** gibt es $s = k - c \leq k$ Anfragen.
- Für Anfragen $j = 1, \dots, s$ auf **schale Seiten**, betrachte die erwartete Fehleranzahl $E[F_j]$.
- $c(j)$: # **saubere Seiten**, die bisher in dieser Phase angefragt wurden
 $s(j)$: # **schale Seiten**, die bisher noch nicht angefragt wurden
- $E[F_j] = \frac{s(j)-c(j)}{s(j)} \cdot 0 + \frac{c(j)}{s(j)} \cdot 1 \leq \frac{c}{k+1-j}$
 $s(j) \geq k+1-j$
- $\sum_{j=1}^s E[F_j]$

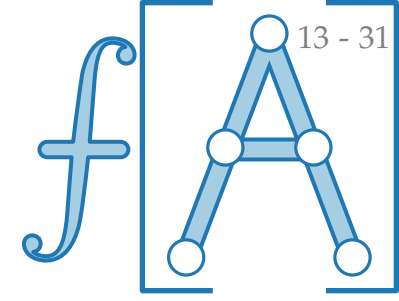
schal: unmarkiert, aber in P_{i-1} markiert
sauber: unmarkiert, aber nicht schal
OPT min. $c/2$ Fehler pro Phase

→ für jede davon
wurde eine schale
Seite rausgeworfen

Analyse

nicht
prüfungsrelevant

Betrachte
Phase P_i .



Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

- Für **saubere Seiten** hat MARKING c Fehler.
- Für **schale Seiten** gibt es $s = k - c \leq k$ Anfragen.
- Für Anfragen $j = 1, \dots, s$ auf **schale Seiten**, betrachte die erwartete Fehleranzahl $E[F_j]$.
- $c(j)$: # **saubere Seiten**, die bisher in dieser Phase angefragt wurden
 $s(j)$: # **schale Seiten**, die bisher noch nicht angefragt wurden
- $E[F_j] = \frac{s(j)-c(j)}{s(j)} \cdot 0 + \frac{c(j)}{s(j)} \cdot 1 \leq \frac{c}{k+1-j}$
 $s(j) \geq k+1-j$
- $\sum_{j=1}^s E[F_j] \leq \sum_{j=1}^s \frac{c}{k+1-j}$

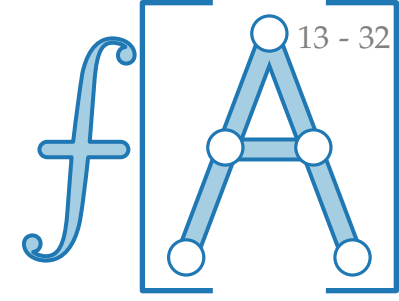
schal: unmarkiert, aber in P_{i-1} markiert
sauber: unmarkiert, aber nicht schal
OPT min. $c/2$ Fehler pro Phase

→ für jede davon
wurde eine schale
Seite rausgeworfen

Analyse

nicht
prüfungsrelevant

Betrachte
Phase P_i .



Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

- Für **saubere Seiten** hat MARKING c Fehler.
- Für **schale Seiten** gibt es $s = k - c \leq k$ Anfragen.
- Für Anfragen $j = 1, \dots, s$ auf **schale Seiten**, betrachte die erwartete Fehleranzahl $E[F_j]$.
- $c(j)$: # **saubere Seiten**, die bisher in dieser Phase angefragt wurden
 $s(j)$: # **schale Seiten**, die bisher noch nicht angefragt wurden
- $E[F_j] = \frac{s(j)-c(j)}{s(j)} \cdot 0 + \frac{c(j)}{s(j)} \cdot 1 \leq \frac{c}{k+1-j}$
 $s(j) \geq k+1-j$
- $\sum_{j=1}^s E[F_j] \leq \sum_{j=1}^s \frac{c}{k+1-j}$

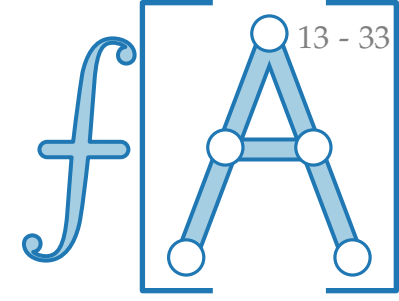
schal: unmarkiert, aber in P_{i-1} markiert
sauber: unmarkiert, aber nicht schal
OPT min. $c/2$ Fehler pro Phase

→ für jede davon
wurde eine schale
Seite rausgeworfen

Analyse

nicht
prüfungsrelevant

Betrachte
Phase P_i .



Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

- Für **saubere Seiten** hat MARKING c Fehler.
- Für **schale Seiten** gibt es $s = k - c \leq k$ Anfragen.
- Für Anfragen $j = 1, \dots, s$ auf **schale Seiten**, betrachte die erwartete Fehleranzahl $E[F_j]$.
- $c(j)$: # **saubere Seiten**, die bisher in dieser Phase angefragt wurden
 $s(j)$: # **schale Seiten**, die bisher noch nicht angefragt wurden
- $E[F_j] = \frac{s(j)-c(j)}{s(j)} \cdot 0 + \frac{c(j)}{s(j)} \cdot 1 \leq \frac{c}{k+1-j}$
 $s(j) \geq k+1-j$
- $\sum_{j=1}^s E[F_j] \leq \sum_{j=1}^s \frac{c}{k+1-j} = \sum_{j=1}^{k-c} \frac{c}{k+1-j}$

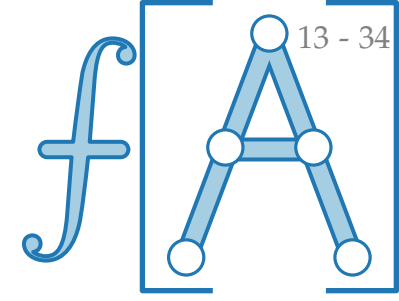
schal: unmarkiert, aber in P_{i-1} markiert
sauber: unmarkiert, aber nicht schal
OPT min. $c/2$ Fehler pro Phase

→ für jede davon
wurde eine schale
Seite rausgeworfen

Analyse

nicht
prüfungsrelevant

Betrachte
Phase P_i .



Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

- Für **saubere Seiten** hat MARKING c Fehler.
- Für **schale Seiten** gibt es $s = k - c \leq k$ Anfragen.
- Für Anfragen $j = 1, \dots, s$ auf **schale Seiten**, betrachte die erwartete Fehleranzahl $E[F_j]$.
- $c(j)$: # **saubere Seiten**, die bisher in dieser Phase angefragt wurden
 $s(j)$: # **schale Seiten**, die bisher noch nicht angefragt wurden
- $E[F_j] = \frac{s(j)-c(j)}{s(j)} \cdot 0 + \frac{c(j)}{s(j)} \cdot 1 \leq \frac{c}{k+1-j}$
 $s(j) \geq k+1-j$
- $\sum_{j=1}^s E[F_j] \leq \sum_{j=1}^s \frac{c}{k+1-j} = \sum_{j=1}^{k-c} \frac{c}{k+1-j} = \sum_{j=c+1}^k \frac{c}{j}$

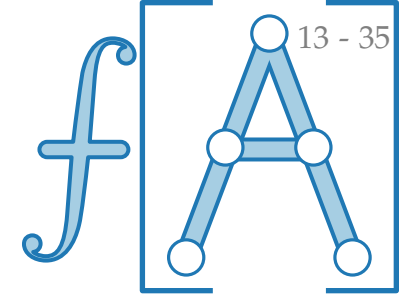
schal: unmarkiert, aber in P_{i-1} markiert
sauber: unmarkiert, aber nicht schal
OPT min. $c/2$ Fehler pro Phase

→ für jede davon
wurde eine schale
Seite rausgeworfen

Analyse

nicht
prüfungsrelevant

Betrachte
Phase P_i .



Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

- Für **saubere Seiten** hat MARKING c Fehler.
- Für **schale Seiten** gibt es $s = k - c \leq k$ Anfragen.
- Für Anfragen $j = 1, \dots, s$ auf **schale Seiten**, betrachte die erwartete Fehleranzahl $E[F_j]$.
- $c(j)$: # **saubere Seiten**, die bisher in dieser Phase angefragt wurden
 $s(j)$: # **schale Seiten**, die bisher noch nicht angefragt wurden
- $E[F_j] = \frac{s(j)-c(j)}{s(j)} \cdot 0 + \frac{c(j)}{s(j)} \cdot 1 \leq \frac{c}{k+1-j}$
 $s(j) \geq k+1-j$
- $\sum_{j=1}^s E[F_j] \leq \sum_{j=1}^s \frac{c}{k+1-j} = \sum_{j=1}^{k-c} \frac{c}{k+1-j} = \sum_{j=c+1}^k \frac{c}{j} = c \cdot (H_k - H_c)$

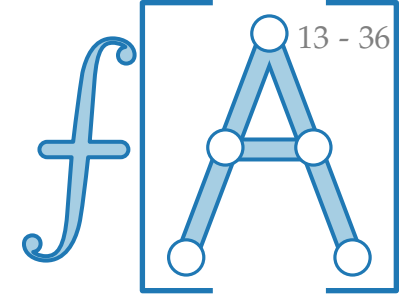
schal: unmarkiert, aber in P_{i-1} markiert
sauber: unmarkiert, aber nicht schal
OPT min. $c/2$ Fehler pro Phase

→ für jede davon
wurde eine schale
Seite rausgeworfen

Analyse

nicht
prüfungsrelevant

Betrachte
Phase P_i .



Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

- Für **saubere Seiten** hat MARKING c Fehler.
- Für **schale Seiten** gibt es $s = k - c \leq k$ Anfragen.
- Für Anfragen $j = 1, \dots, s$ auf **schale Seiten**, betrachte die erwartete Fehleranzahl $E[F_j]$.
- $c(j)$: # **saubere Seiten**, die bisher in dieser Phase angefragt wurden
 $s(j)$: # **schale Seiten**, die bisher noch nicht angefragt wurden
- $E[F_j] = \frac{s(j)-c(j)}{s(j)} \cdot 0 + \frac{c(j)}{s(j)} \cdot 1 \leq \frac{c}{k+1-j}$
 $s(j) \geq k+1-j$
- $\sum_{j=1}^s E[F_j] \leq \sum_{j=1}^s \frac{c}{k+1-j} = \sum_{j=1}^{k-c} \frac{c}{k+1-j} = \sum_{j=c+1}^k \frac{c}{j} = c \cdot (H_k - H_c) \leq c \cdot H_k$

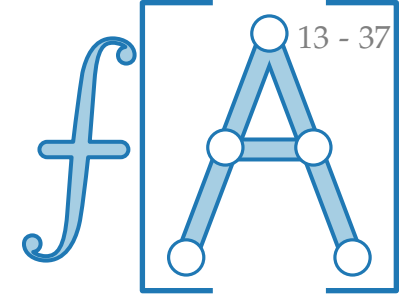
schal: unmarkiert, aber in P_{i-1} markiert
sauber: unmarkiert, aber nicht schal
OPT min. $c/2$ Fehler pro Phase

→ für jede davon
wurde eine schale
Seite rausgeworfen

Analyse

nicht
prüfungsrelevant

Betrachte
Phase P_i .



Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

- Für **saubere Seiten** hat MARKING c Fehler.
- Für **schale Seiten** gibt es $s = k - c \leq k$ Anfragen.
- Für Anfragen $j = 1, \dots, s$ auf **schale Seiten**, betrachte die erwartete Fehleranzahl $E[F_j]$.
- $c(j)$: # **saubere Seiten**, die bisher in dieser Phase angefragt wurden
 $s(j)$: # **schale Seiten**, die bisher noch nicht angefragt wurden
- $E[F_j] = \frac{s(j)-c(j)}{s(j)} \cdot 0 + \frac{c(j)}{s(j)} \cdot 1 \leq \frac{c}{k+1-j}$
 $s(j) \geq k+1-j$
- $\sum_{j=1}^s E[F_j] \leq \sum_{j=1}^s \frac{c}{k+1-j} = \sum_{j=1}^{k-c} \frac{c}{k+1-j} = \sum_{j=c+1}^k \frac{c}{j} = c \cdot (H_k - H_c) \leq c \cdot H_k$
- Also ist das kompetitive Verhältnis von MARKING

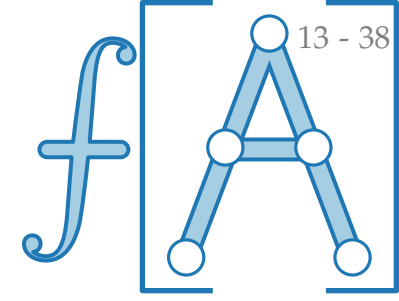
schal: unmarkiert, aber in P_{i-1} markiert
sauber: unmarkiert, aber nicht schal
OPT min. $c/2$ Fehler pro Phase

→ für jede davon
wurde eine schale
Seite rausgeworfen

Analyse

nicht
prüfungsrelevant

Betrachte
Phase P_i .



Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

- Für **saubere Seiten** hat MARKING c Fehler.
- Für **schale Seiten** gibt es $s = k - c \leq k$ Anfragen.
- Für Anfragen $j = 1, \dots, s$ auf **schale Seiten**, betrachte die erwartete Fehleranzahl $E[F_j]$.
- $c(j)$: # **saubere Seiten**, die bisher in dieser Phase angefragt wurden
 $s(j)$: # **schale Seiten**, die bisher noch nicht angefragt wurden
- $E[F_j] = \frac{s(j)-c(j)}{s(j)} \cdot 0 + \frac{c(j)}{s(j)} \cdot 1 \leq \frac{c}{k+1-j}$
 $s(j) \geq k+1-j$
- $\sum_{j=1}^s E[F_j] \leq \sum_{j=1}^s \frac{c}{k+1-j} = \sum_{j=1}^{k-c} \frac{c}{k+1-j} = \sum_{j=c+1}^k \frac{c}{j} = c \cdot (H_k - H_c) \leq c \cdot H_k$
- Also ist das kompetitive Verhältnis von MARKING $\leq c$

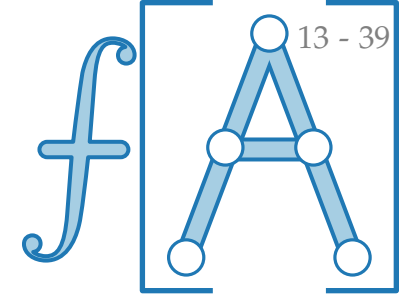
schal: unmarkiert, aber in P_{i-1} markiert
sauber: unmarkiert, aber nicht schal
OPT min. $c/2$ Fehler pro Phase

→ für jede davon
wurde eine schale
Seite rausgeworfen

Analyse

nicht
prüfungsrelevant

Betrachte
Phase P_i .



Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

- Für **saubere Seiten** hat MARKING c Fehler.
- Für **schale Seiten** gibt es $s = k - c \leq k$ Anfragen.
- Für Anfragen $j = 1, \dots, s$ auf **schale Seiten**, betrachte die erwartete Fehleranzahl $E[F_j]$.
- $c(j)$: # **saubere Seiten**, die bisher in dieser Phase angefragt wurden
 $s(j)$: # **schale Seiten**, die bisher noch nicht angefragt wurden
- $E[F_j] = \frac{s(j)-c(j)}{s(j)} \cdot 0 + \frac{c(j)}{s(j)} \cdot 1 \leq \frac{c}{k+1-j}$
 $s(j) \geq k+1-j$
- $\sum_{j=1}^s E[F_j] \leq \sum_{j=1}^s \frac{c}{k+1-j} = \sum_{j=1}^{k-c} \frac{c}{k+1-j} = \sum_{j=c+1}^k \frac{c}{j} = c \cdot (H_k - H_c) \leq c \cdot H_k$
- Also ist das kompetitive Verhältnis von MARKING $\frac{c}{c} + c \cdot H_k$

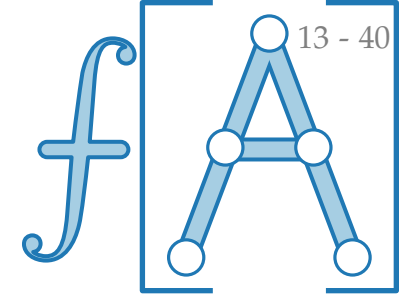
schal: unmarkiert, aber in P_{i-1} markiert
sauber: unmarkiert, aber nicht schal
OPT min. $c/2$ Fehler pro Phase

→ für jede davon
wurde eine schale
Seite rausgeworfen

Analyse

nicht
prüfungsrelevant

Betrachte
Phase P_i .



Satz. MARKING ist $2H_k$ -kompetitiv.

Beweis.

- Für **saubere Seiten** hat MARKING c Fehler.
- Für **schale Seiten** gibt es $s = k - c \leq k$ Anfragen.
- Für Anfragen $j = 1, \dots, s$ auf **schale Seiten**, betrachte die erwartete Fehleranzahl $E[F_j]$.
- $c(j)$: # **saubere Seiten**, die bisher in dieser Phase angefragt wurden
 $s(j)$: # **schale Seiten**, die bisher noch nicht angefragt wurden
- $E[F_j] = \frac{s(j)-c(j)}{s(j)} \cdot 0 + \frac{c(j)}{s(j)} \cdot 1 \leq \frac{c}{k+1-j}$
 $s(j) \geq k+1-j$
- $\sum_{j=1}^s E[F_j] \leq \sum_{j=1}^s \frac{c}{k+1-j} = \sum_{j=1}^{k-c} \frac{c}{k+1-j} = \sum_{j=c+1}^k \frac{c}{j} = c \cdot (H_k - H_c) \leq c \cdot H_k$
- Also ist das kompetitive Verhältnis von MARKING $\frac{c + c \cdot H_k}{c/2}$

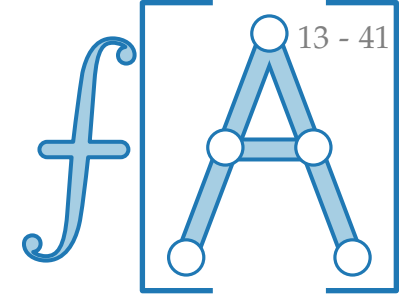
schal: unmarkiert, aber in P_{i-1} markiert
sauber: unmarkiert, aber nicht schal
OPT min. $c/2$ Fehler pro Phase

→ für jede davon
wurde eine schale
Seite rausgeworfen

Analyse

nicht
prüfungsrelevant

Betrachte
Phase P_i .



Satz. MARKING ist $2H_k$ -kompetitiv.

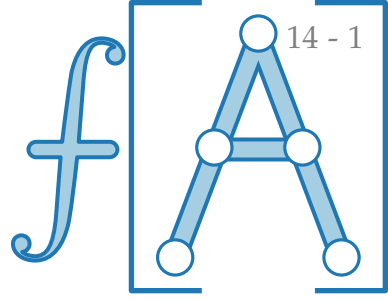
Beweis.

- Für **saubere Seiten** hat MARKING c Fehler.
- Für **schale Seiten** gibt es $s = k - c \leq k$ Anfragen.
- Für Anfragen $j = 1, \dots, s$ auf **schale Seiten**, betrachte die erwartete Fehleranzahl $E[F_j]$.
- $c(j)$: # **saubere Seiten**, die bisher in dieser Phase angefragt wurden
 $s(j)$: # **schale Seiten**, die bisher noch nicht angefragt wurden
- $E[F_j] = \frac{s(j)-c(j)}{s(j)} \cdot 0 + \frac{c(j)}{s(j)} \cdot 1 \leq \frac{c}{k+1-j}$
 $s(j) \geq k+1-j$
- $\sum_{j=1}^s E[F_j] \leq \sum_{j=1}^s \frac{c}{k+1-j} = \sum_{j=1}^{k-c} \frac{c}{k+1-j} = \sum_{j=c+1}^k \frac{c}{j} = c \cdot (H_k - H_c) \leq c \cdot H_k$
- Also ist das kompetitive Verhältnis von MARKING $\frac{c+c \cdot H_k}{c/2} = 2H_k$. $\square$

schal: unmarkiert, aber in P_{i-1} markiert
sauber: unmarkiert, aber nicht schal
OPT min. $c/2$ Fehler pro Phase

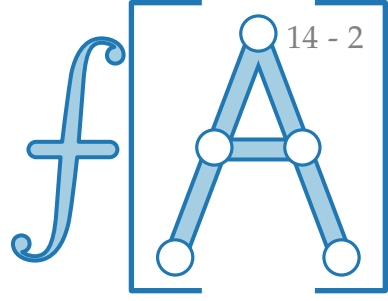
→ für jede davon
wurde eine schale
Seite rausgeworfen

Diskussion



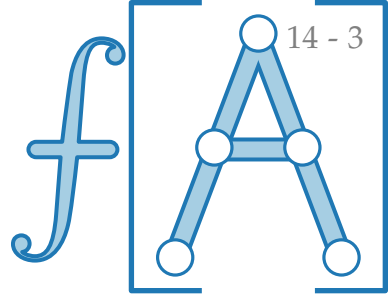
- Online Algorithmen operieren in einem anderen Rahmen als klassische Algorithmen. Unvollständige Informationen sind sehr natürlich und kommen of in praktischen Anwendungen vor. Können Sie weitere Beispiele finden?

Diskussion



- Online Algorithmen operieren in einem anderen Rahmen als klassische Algorithmen. Unvollständige Informationen sind sehr natürlich und kommen of in praktischen Anwendungen vor. Können Sie weitere Beispiele finden?
- Wir können auch klassische Probleme mit unvollständigen Informationen in online Probleme umwandeln. Z.B. Matching Probleme für Fahrgemeinschaften.

Diskussion



- Online Algorithmen operieren in einem anderen Rahmen als klassische Algorithmen. Unvollständige Informationen sind sehr natürlich und kommen oft in praktischen Anwendungen vor. Können Sie weitere Beispiele finden?
- Wir können auch klassische Probleme mit unvollständigen Informationen in online Probleme umwandeln. Z.B. Matching Probleme für Fahrgemeinschaften.
- Randomisierung kann helfen, das Verhalten in worst-case Instanzen zu verbessern: wir sind weniger vorhersehbar für einen Konkurrenten.